

GENERATIVE AI FOR DATA SCIENCE

Boosting Data Science Workflows
with Generative AI Techniques

KEY TOPICS



Generative AI applications
in Data Science



Data preparation and
synthetic data generation



Data augmentation and
feature engineering



Exploratory Data Analysis (EDA)



Machine Learning
workflow enhancement



Data visualization with AI



Responsible AI
and Data Ethics



Generative Model
Architectures



Generative Adversarial
Networks (GANs)



Autoencoders



AI tools: GPT-3.5,
ChatCSV, and tomat.ai



Real-world applications of
Generative AI in Data Science



KASHIF MAQBOOL JOIYA

Data Scientist | AI Engineer



<https://github.com/KashifMaqbool>



Table of Content

1	GENERATIVE AI AND DATA SCIENCE	1
1.1	LEARNING OBJECTIVES	1
1.2	WHAT IS GENERATIVE AI?	1
1.3	HOW GENERATIVE AI OPERATES	1
1.4	APPLICATIONS OF GENERATIVE AI	1
1.5	HOW DATA SCIENTISTS USE GENERATIVE AI	2
1.5.1	SYNTHETIC DATA GENERATION	2
1.5.2	CODE GENERATION FOR ANALYTICAL MODELS	2
1.5.3	BUSINESS INSIGHTS AND REPORTING	2
1.6	REAL-WORLD EXAMPLE	3
1.7	KEY TAKEAWAYS	3
2	EXPERT VIEWPOINTS: HOW GENERATIVE AI ENHANCES DATA SCIENCE WORKFLOWS	3
2.1	INTRODUCTION	3
2.2	HOW GENERATIVE AI ENHANCES DATA SCIENCE WORKFLOWS	3
2.3	GENERATIVE AI FOR CODING TASKS	4
2.4	DATA PREPARATION AND AUTOMATION	4
2.4.1	AUTOMATED DATA IMPUTATION	4
2.5	POPULAR GENERATIVE AI TOOLS	5
2.5.1	CHATGPT	5
2.5.2	GITHUB COPILOT	5
2.6	TECHNICAL DOCUMENTATION	5
2.7	FEATURE ENGINEERING	5
2.8	VISUALIZATION AND REPORTING	6
2.9	RAPID MODEL PROTOTYPING	6
2.10	EXPERT EXAMPLES OF USING GENERATIVE AI IN DATA SCIENCE	6
2.10.1	DATA EXTRACTION AND TRANSFORMATION	6
2.10.2	SYNTHETIC DATA GENERATION	6
2.10.3	AUTOMATED REPORT GENERATION	7
2.10.4	DOCUMENTATION AND CODE GENERATION	7
2.11	RESEARCH ASSISTANCE	7
2.12	ENTERPRISE KNOWLEDGE CHATBOTS	7
2.13	COMPUTER VISION APPLICATIONS	7
2.14	DESIGN OPTIMIZATION	7
2.15	FINANCIAL SERVICES APPLICATIONS	8
2.16	DOCUMENT UNDERSTANDING	8
2.17	MACHINE LEARNING CODE GENERATION	8
2.18	MODEL METADATA	8

3	GENERATIVE AI TOOLS FOR DATA SCIENTISTS	9
3.1	OVERVIEW	9
3.2	SYNTHETIC DATA GENERATION	9
3.3	IMAGE GENERATION AND MANIPULATION	9
3.4	NATURAL LANGUAGE GENERATION	10
3.5	MUSIC AND AUDIO SYNTHESIS	10
3.6	SIMULATION AND DATA AUGMENTATION	11
3.7	CONTENT GENERATION	11
3.8	ANOMALY DETECTION	12
3.9	DATA AUGMENTATION	12
3.10	HUMAN-COMPUTER INTERACTION	13
3.11	CONCLUSION	13
4	GENERATIVE AI'S IMPACT ACROSS INDUSTRIES	13
4.1	LEARNING OBJECTIVES	13
4.2	INTRODUCTION	14
4.3	IMPACT OF GENERATIVE AI ON HEALTHCARE	14
4.3.1	DRUG DISCOVERY	14
4.3.2	MEDICAL IMAGE ANALYSIS	14
4.3.3	PERSONALIZED TREATMENT	14
4.4	IMPACT OF GENERATIVE AI ON FINANCE	15
4.4.1	RISK MANAGEMENT	15
4.4.2	FRAUD DETECTION	15
4.4.3	INVESTMENT STRATEGIES	15
4.5	IMPACT OF GENERATIVE AI ON RETAIL	15
4.5.1	CUSTOMER EXPERIENCE	16
4.5.2	PRODUCT DEVELOPMENT	16
4.5.3	SUPPLY CHAIN OPTIMIZATION	16
4.6	IMPACT OF GENERATIVE AI ON MANUFACTURING	16
4.6.1	PRODUCTION OPTIMIZATION	16
4.6.2	PRODUCT DESIGN	17
4.6.3	QUALITY CONTROL	17
4.7	IMPACT OF GENERATIVE AI ON MEDIA AND ENTERTAINMENT	17
4.7.1	CONTENT CREATION	17
4.7.2	PERSONALIZED USER EXPERIENCE	17
4.7.3	CREATIVE ASSISTANCE	18
4.8	IMPACT OF GENERATIVE AI ON EDUCATION	18
4.8.1	PERSONALIZED LEARNING	18
4.8.2	REAL-TIME FEEDBACK	18
4.8.3	ADAPTIVE LEARNING MATERIALS	19
4.9	IMPACT OF GENERATIVE AI ON TRANSPORTATION	19
4.9.1	TRAFFIC MANAGEMENT	19

4.9.2	TRANSPORTATION SYSTEM OPTIMIZATION	19
4.9.3	TRANSPORTATION SAFETY	19
4.10	KEY TAKEAWAYS	20
5	SKILLS DATA PROFESSIONALS SHOULD DEVELOP TO EFFECTIVELY LEVERAGE GENERATIVE AI (GENAI)	20
5.1	INTRODUCTION	20
5.2	PROGRAMMING SKILLS	20
5.3	MACHINE LEARNING FUNDAMENTALS	20
5.4	MATHEMATICAL FOUNDATIONS	21
5.5	DEEP LEARNING FOUNDATIONS	21
5.6	GENERATIVE AI MODEL ARCHITECTURES	21
5.7	UNDERSTANDING HOW GENAI WORKS	22
5.8	PROMPT ENGINEERING	22
5.9	ADVANCED AI WORKFLOWS	22
5.10	MODEL FINE-TUNING	23
5.11	AI LIBRARIES AND FRAMEWORKS	23
5.12	DATA VISUALIZATION SKILLS	23
5.13	MODEL DEPLOYMENT	23
5.14	EVALUATING AI-GENERATED SOLUTIONS	23
5.15	ESSENTIAL SOFT SKILLS	24
5.16	KEY TAKEAWAYS	24
6	LEVERAGING GENERATIVE AI IN THE DATA SCIENCE LIFE CYCLE	24
6.1	LEARNING OBJECTIVES	24
6.2	INTRODUCTION	25
6.3	PROBLEM DEFINITION AND BUSINESS UNDERSTANDING	25
6.3.1	REAL-WORLD EXAMPLE	25
6.4	DATA ACQUISITION AND PREPARATION	25
6.4.1	REAL-WORLD EXAMPLE	26
6.5	MODEL DEVELOPMENT AND TRAINING	26
6.5.1	REAL-WORLD EXAMPLE	26
6.6	MODEL EVALUATION AND REFINEMENT	26
6.6.1	REAL-WORLD EXAMPLE	27
6.7	MODEL DEPLOYMENT AND MONITORING	27
6.7.1	REAL-WORLD EXAMPLE	27
6.8	SUMMARY	27
6.8.1	HOW GENERATIVE AI ENHANCES EACH PHASE	27
7	TYPES OF GENERATIVE AI MODELS	28
7.1	LEARNING OBJECTIVES	28

7.2	INTRODUCTION	28
7.3	GENERATIVE ADVERSARIAL NETWORKS (GANs)	28
7.3.1	STRENGTHS OF GANs	28
7.3.2	APPLICATIONS OF GANs	29
7.3.3	EXAMPLE	29
7.4	VARIATIONAL AUTOENCODERS (VAEs)	29
7.4.1	STRENGTHS OF VAEs	29
7.4.2	APPLICATIONS OF VAEs	29
7.4.3	EXAMPLE	30
7.5	AUTOREGRESSIVE MODELS	30
7.5.1	STRENGTHS OF AUTOREGRESSIVE MODELS	30
7.5.2	APPLICATIONS OF AUTOREGRESSIVE MODELS	30
7.5.3	EXAMPLE	30
7.6	FLOW-BASED MODELS	31
7.6.1	STRENGTHS OF FLOW-BASED MODELS	31
7.6.2	APPLICATIONS OF FLOW-BASED MODELS	31
7.6.3	EXAMPLE	31
7.7	SUMMARY	31
7.8	KEY TAKEAWAYS	32
8	GUIDE TO CHOOSING A GENERATIVE AI MODEL TYPE	32
8.1	TYPES OF GENERATIVE AI MODELS	32
8.2	COMPARISON OF MODELS ON DIFFERENT CONSIDERATIONS	34
9	CASE STUDY: SUCCESSFUL IMPLEMENTATION OF GENERATIVE AI	34
9.1	INTRODUCTION	34
9.2	DATA INGESTION AND PREPROCESSING	35
9.2.1	TEXT ANALYSIS WITH GENERATIVE PRE-TRAINED TRANSFORMERS (GPTs)	35
9.2.2	AUTOMATIC GENRE AND SUBGENRE TAGGING	35
9.2.3	AI-POWERED ANOMALY DETECTION	35
9.3	FEATURE ENGINEERING AND MODEL TRAINING	35
9.3.1	GENERATIVE ADVERSARIAL NETWORKS (GANs)	35
9.3.2	COUNTERFACTUAL REASONING	35
9.3.3	EXPLAINABLE AI (XAI) FOR MODEL INTERPRETABILITY	35
9.4	DEPLOYMENT AND MONITORING	35
9.4.1	ADAPTIVE CONTENT RECOMMENDATIONS	35
9.4.2	A/B TESTING WITH GENERATIVE MODELS	36
9.4.3	CONTINUOUS FEEDBACK LOOP	36
9.5	CONCLUSION	36
10	EXPLORE A SIMPLE GENERATIVE TOOL	36

10.1	LEARNING OBJECTIVES	36
10.2	INTRODUCTION	36
10.3	EXERCISE 1: SIGN UP IN DATA ROBOT	37
10.3.1	STEP	37
10.4	EXERCISE 2: ADD A DATASET	37
10.4.1	STEPS	37
10.5	EXERCISE 3: WORK ON DATA MODELING	39
10.5.1	STEPS	39
10.6	SUMMARY	42
11	GENERATIVE AI TOOLS FOR DATA SCIENTISTS	42
11.1	OVERVIEW	42
11.2	CLAUDE AI BY ANTHROPIC	42
11.3	CATEGORIES OF GENERATIVE AI TOOLS	43
11.4	OPEN-SOURCE GENERATIVE AI TOOLS	43
11.4.1	HUGGING FACE TRANSFORMERS	43
11.4.2	DATABRICKS WITH GENAI	43
11.4.3	LANGCHAIN	43
11.4.4	HUGGING FACE DATASETS	43
11.4.5	LABEL STUDIO	44
11.4.6	DAML (DATA ANNOTATION FOR MACHINE LEARNING)	44
11.4.7	ALBUMENTATIONS	44
11.4.8	PANDAS PROFILING	44
11.4.9	TENSORFLOW	44
11.4.10	PYTORCH	44
11.4.11	MLFLOW	44
11.5	PROPRIETARY GENERATIVE AI TOOLS	45
11.5.1	OPENAI GPT MODELS	45
11.5.2	CHATGPT	45
11.5.3	MICROSOFT COPILOT	45
11.5.4	GITHUB COPILOT	45
11.5.5	IBM WATSONX CODE ASSISTANT	45
11.5.6	TABLEAU GPT	45
11.5.7	POWER BI COPILOT	46
11.5.8	DATA ROBOT	46
11.6	CLOUD-BASED AI PLATFORMS	46
11.7	ADDITIONAL AI TOOLS	46
11.7.1	GOOGLE'S BOT MODEL	46
11.7.2	HTU.AI	46
11.8	KEY TAKEAWAYS	46
12	GENERATIVE AI AS A USEFUL TOOL FOR DATA GENERATION AND AUGMENTATION	47

12.1	LEARNING OBJECTIVES	47
12.2	INTRODUCTION	47
12.3	GENERATIVE AI FOR DIFFERENT TYPES OF DATA	47
12.3.1	STRUCTURED DATA	47
12.3.2	SEMI-STRUCTURED DATA	48
12.3.3	UNSTRUCTURED DATA	48
12.4	DATA GENERATION USING UNIVERSAL DATA	48
12.4.1	EXAMPLE PROMPT	48
12.5	DATA GENERATION USING CHATGPT	49
12.5.1	EXAMPLE PROMPT	49
12.6	DATA GENERATION USING BARD	49
12.7	DATA GENERATION USING MOSTLYAI	50
12.7.1	STEPS	50
12.8	DATA GENERATION USING CTGAN IN GOOGLE COLABORATORY	51
12.8.1	DATASET USED	51
12.8.2	STEPS	52
12.8.3	EVALUATING THE GENERATED DATA	53
12.9	KEY TAKEAWAYS	53
13	GENERATIVE AI FOR DATA GENERATION AND AUGMENTATION	54
13.1	LEARNING OBJECTIVES	54
13.2	INTRODUCTION	54
13.3	INSURANCE DATASET OVERVIEW	54
14	EXERCISE 1: DATA AUGMENTATION USING ILOVECSV	54
14.1	PROCEDURE	54
14.2	DATA AUGMENTATION	54
15	EXERCISE 2: DATA AUGMENTATION USING TONIC FABRICATE	55
15.1	PROCEDURE	55
15.2	PROMPT FOR SYNTHETIC DATA GENERATION	55
15.3	DOWNLOAD THE SYNTHETIC DATASET	56
15.4	ANALYZE THE GENERATED REPORT	56
15.4.1	PROMPT FOR REPORT GENERATION	56
15.5	SUMMARY	56
16	GENERATIVE AI FOR DATA PREPARATION AND DATA QUERYING	57
16.1	LEARNING OBJECTIVES	57
16.2	INTRODUCTION	57

16.3	GENERATIVE AI FOR DATA PREPARATION	57
16.3.1	HANDLING MISSING VALUES	57
16.3.2	DETECTING OUTLIERS	58
16.3.3	NOISE REDUCTION	58
16.3.4	DATA TRANSLATION	58
16.4	GENERATIVE AI FOR DATA QUERYING	59
16.4.1	NATURAL LANGUAGE QUERYING	59
16.4.2	QUERY RECOMMENDATION	59
16.4.3	QUERY OPTIMIZATION	59
16.5	SUMMARY	60
17	GENERATIVE AI FOR STREAMLINING THE DATA PREPARATION PROCESS	60
17.1	LEARNING OBJECTIVES	60
17.2	HOW GENERATIVE AI STREAMLINES DATA PREPARATION	60
17.2.1	CHATCSV FOR DATA CLEANING	61
17.3	DATA INTEGRATION AND TABLE MERGING	61
17.4	NATURAL LANGUAGE TO SQL	61
17.4.1	KNOWLEDGE-AUGMENTED FOUNDATION MODELS	61
17.5	DATA CLEANING AND DATA QUALITY IMPROVEMENT	61
17.6	DATA NORMALIZATION AND TRANSFORMATION	62
17.7	POTENTIAL PITFALLS OF GENERATIVE AI	62
17.7.1	LACK OF DOMAIN UNDERSTANDING	63
17.7.2	FALSE SENSE OF DATA QUALITY	63
17.7.3	OVERFITTING NOISE	63
17.7.4	BIAS IN GENERATED DATA	63
17.7.5	IMPORTANCE OF HUMAN OVERSIGHT	63
17.8	GENERATIVE AI FOR COMPLEX DATA QUERIES	64
17.9	GEN SQL	64
17.10	KEY TAKEAWAYS	65
18	GENERATIVE AI FOR DATA PREPARATION	65
18.1	LEARNING OBJECTIVES	65
18.2	INTRODUCTION TO DATA PREPARATION	66
18.3	DEMONSTRATION 1: DATA PREPARATION USING CHATCSV	66
18.3.1	UPLOADING THE DATASET	66
18.3.2	REPLACING MISSING VALUES	66
18.3.3	FINDING OUTLIERS	67
18.4	DEMONSTRATION 2: DATA PREPARATION USING TOMAT.AI	68
18.4.1	UPLOADING THE DATASET	68
18.4.2	CALCULATING AVERAGE VALUES	68
18.4.3	CREATING A DATA FLOW	69
18.4.4	MERGING MULTIPLE TABLES	69

18.4.5	FILTERING DATA	69
18.5	USING THE AI ASSISTANT	69
18.6	CREATING AN IF-THEN RULE	70
18.7	EXPORTING THE PREPARED DATASET	70
18.8	KEY TAKEAWAYS	71
19	GENERATIVE AI FOR DATA PREPARATION	71
19.1	LEARNING OBJECTIVES	71
19.2	INTRODUCTION	71
19.3	EXERCISE 1: IMPORT THE DATASET	71
19.3.1	PROMPT	71
19.4	EXERCISE 2: HANDLE MISSING DATA	72
19.4.1	IDENTIFY MISSING VALUES	72
19.4.2	REPLACE MISSING VALUES	72
19.5	EXERCISE 3: MODIFY DATA TYPES	72
19.5.1	PROMPT	72
19.6	EXERCISE 4: STANDARDIZATION AND NORMALIZATION	72
19.6.1	CONVERT MEASUREMENT UNITS	72
19.6.2	NORMALIZE CPU FREQUENCY	73
19.7	EXERCISE 5: CONVERT CATEGORICAL DATA TO NUMERICAL DATA	73
19.7.1	PROMPT	73
19.8	PRACTICE EXERCISES	73
19.9	SUMMARY	74
20	GENERATIVE AI FOR QUERYING DATABASES	74
20.1	LEARNING OBJECTIVES	74
20.2	INTRODUCTION	74
20.3	EXTRACTING INSIGHTS FROM MASSIVE DATASETS USING GENERATIVE AI	74
20.3.1	SETTING UP THE DATABASE	75
20.4	GENERATING SQL QUERIES USING NATURAL LANGUAGE	75
20.4.1	VIEWING COLUMN NAMES	75
20.4.2	COUNTING THE NUMBER OF ROWS	76
20.4.3	FINDING THE AVERAGE AGE	76
20.4.4	FINDING ROWS BASED ON A TAX RANGE	76
20.4.5	REPLACING ZERO VALUES IN THE ZN COLUMN	76
20.4.6	SORTING THE TABLE BY THE MEDV COLUMN	76
20.4.7	INSERTING NEW ROWS	76
20.4.8	GENERATING A CONDITION-BASED QUERY	76
20.4.9	CREATING A SUB-TABLE	77
20.5	IMPORTANCE OF QUERYING DATABASES	77
20.6	KEY TAKEAWAYS	77

21	GENERATIVE AI FOR QUERYING DATABASES	78
21.1	LEARNING OBJECTIVES	78
21.2	INTRODUCTION	78
21.3	EXERCISE 1: PROVIDE CONTEXT TO THE GENERATIVE AI MODEL	78
21.3.1	PROMPT INSTRUCTIONS	78
21.3.2	ATTRIBUTE VALUE DEFINITIONS	78
21.4	EXERCISE 2: GENERATE SQL QUERIES USING PROMPTS	79
21.4.1	AGE DISTRIBUTION	79
21.4.2	GENDER ANALYSIS	79
21.4.3	CHEST PAIN TYPE FREQUENCY	80
21.4.4	AGE GROUP ANALYSIS AND TARGET VARIABLE	80
21.5	PRACTICE EXERCISES	81
21.6	SUMMARY	81
22	MODULE 1 SUMMARY: DATA SCIENCE AND GENERATIVE AI	81
22.1	MODULE COMPLETION	81
23	MODULE 1 CHEATSHEET: DATA SCIENCE AND GENERATIVE AI	82
23.1	POPULAR GENERATIVE AI TOOLS	82
23.2	IMPORTANT PROMPTS FOR DATA PREPARATION	82
24	GENERATIVE AI FOR DRAWING DATA INSIGHTS	83
24.1	LEARNING OBJECTIVES	83
24.2	USING GENERATIVE AI TO GENERATE PYTHON CODE FOR DATA ANALYSIS	83
24.3	ENHANCING DATA ANALYSIS WITH ADVANCED STATISTICAL INSIGHTS	84
24.4	EXTENDING THE ANALYSIS WITH FEATURE SELECTION AND FEATURE ENGINEERING	84
24.5	USING HAL9 TO GENERATE DATA INSIGHTS	84
24.6	DISTRIBUTION OF STUDENT AGES ACROSS SCHOOLS	85
24.7	FINDING MISSING VALUES	85
24.8	GENERATING STATISTICAL INSIGHTS	85
24.9	KEY TAKEAWAYS	86
25	GENERATIVE AI FOR DATA INSIGHTS	86
25.1	LEARNING OBJECTIVES	86
25.2	INTRODUCTION	86
25.3	EXERCISE 1: GENERATE A STATISTICAL DESCRIPTION OF THE DATASET	86
25.3.1	PROMPT	87
25.4	EXERCISE 2: CREATE PARAMETER VISUALIZATIONS	87

25.4.1	CONTINUOUS ATTRIBUTES	87
25.4.2	CATEGORICAL ATTRIBUTES	87
25.4.3	PROMPT	87
25.5	EXERCISE 3: EVALUATE DEPENDENCE	87
25.5.1	PROMPT	88
25.6	EXERCISE 4: GROUPING AND PIVOT TABLES	88
25.6.1	PROMPT	88
25.7	SUMMARY	88
26	GENERATIVE AI FOR DATA VISUALIZATION	88
26.1	LEARNING OBJECTIVES	88
26.2	INTRODUCTION	89
26.3	CREATING VISUALIZATIONS WITH COLUMNS.AI	89
26.3.1	UPLOADING AND EXPLORING A DATASET	89
26.3.2	GENERATING A BAR CHART	89
26.3.3	CREATING A PIE CHART	90
26.3.4	GENERATING INSIGHTS	90
26.3.5	CUSTOMIZING AND EXPORTING CHARTS	90
26.4	EXPLORING DATA WITH AKKIO	90
26.4.1	CREATING A SCATTER PLOT	90
26.4.2	GENERATING A BAR CHART	91
26.4.3	CREATING A CORRELATION MATRIX	91
26.4.4	IDENTIFYING OUTLIERS	91
26.4.5	UNDERSTANDING DATA DISTRIBUTION	91
26.5	KEY TAKEAWAYS	91
27	GENERATIVE AI FOR STORYTELLING	92
27.1	LEARNING OBJECTIVES	92
27.2	INTRODUCTION	92
27.3	EXERCISE 1: SIGN UP FOR Zoho ANALYTICS	92
27.4	EXERCISE 2: UPLOAD A DATASET	92
27.5	EXERCISE 3: USE ZIA AI TO EXPLORE DATA AND GENERATE REPORTS	92
27.5.1	PROMPTS	92
27.6	EXERCISE 4: FINALIZE THE DASHBOARD	93
27.7	EXERCISE 5: SAVE AND SHARE THE REPORT	93
27.8	SUMMARY	94
28	GENERATIVE AI TOOLS FOR MODEL DEVELOPMENT	94
28.1	LEARNING OBJECTIVES	94
28.2	INTRODUCTION	94
28.3	DATA ROBOT	94

28.3.1	MERITS	95
28.3.2	DEMERITS	95
28.4	AUTOGLUON	95
28.4.1	MERITS	95
28.4.2	DEMERITS	95
28.5	H2O DRIVERLESS AI	95
28.5.1	MERITS	95
28.5.2	DEMERITS	96
28.6	AMAZON SAGEMAKER AUTOPILOT	96
28.6.1	MERITS	96
28.6.2	DEMERITS	96
28.7	GOOGLE VERTEX AI	96
28.7.1	MERITS	96
28.7.2	DEMERITS	96
28.8	USING CHATGPT AND GOOGLE BARD FOR MODEL DEVELOPMENT	97
28.8.1	EXAMPLE PROMPT	97
28.8.2	IMPORTANT NOTE	97
28.9	KEY TAKEAWAYS	97
29	GENERATIVE AI FOR MODELS DEVELOPMENT	98
29.1	LEARNING OBJECTIVES	98
29.2	INTRODUCTION	98
29.3	IMPORT THE DATASET	98
29.3.1	PROMPT	98
29.4	LINEAR REGRESSION WITH A SINGLE VARIABLE	98
29.4.1	PROMPT	98
29.5	LINEAR REGRESSION WITH MULTIPLE VARIABLES	99
29.5.1	PROMPT	99
29.6	POLYNOMIAL REGRESSION	99
29.6.1	PROMPT	99
29.7	CREATE A MACHINE LEARNING PIPELINE	99
29.7.1	PROMPT	99
29.8	GRID SEARCH AND RIDGE REGRESSION	100
29.8.1	PROMPT	100
29.8.2	MODEL CONFIGURATION	100
29.9	SUMMARY	100
30	GENERATIVE AI FOR UNDERSTANDING DATA AND MODEL DEVELOPMENT	101
30.1	LEARNING OBJECTIVES	101
30.2	GENERATIVE AI FOR EXPLORATORY DATA ANALYSIS (EDA)	101
30.2.1	STATISTICAL DATA DESCRIPTION	101
30.2.2	UNIVARIATE ANALYSIS	101

30.2.3	BIVARIATE ANALYSIS	101
30.2.4	MULTIVARIATE ANALYSIS	101
30.2.5	FEATURE ENGINEERING	102
30.2.6	HYPOTHESIS GENERATION	102
30.3	ADVANTAGES OF GENERATIVE AI FOR MODEL DEVELOPMENT	102
30.3.1	MODEL ARCHITECTURE SELECTION	102
30.3.2	FEATURE SELECTION	102
30.3.3	ENSEMBLE MODEL GENERATION	103
30.3.4	MODEL INTERPRETABILITY	103
30.3.5	IMPROVING GENERALIZATION	103
30.4	KEY TAKEAWAYS	103
31	GENERATIVE AI FOR IDENTIFYING PATTERNS AND INSIGHTS IN LARGE DATASETS	104
31.1	LEARNING OBJECTIVES	104
31.2	HOW GENERATIVE AI IDENTIFIES PATTERNS AND INSIGHTS	104
31.2.1	PROCESSING LARGE VOLUMES OF DATA	104
31.2.2	FOUNDATION MODELS VS. TRADITIONAL MACHINE LEARNING MODELS	105
31.2.3	THE ROLE OF THE ATTENTION MECHANISM	105
31.2.4	LEARNING COMPLEX RELATIONSHIPS	106
31.3	ADVANTAGES OF GENERATIVE AI FOR MODEL BUILDING AND VALIDATION	106
31.3.1	SYNTHETIC DATA GENERATION	106
31.3.2	AUTOMATION AND EFFICIENCY	106
31.3.3	ADAPTIVE LEARNING	107
31.3.4	DOING MORE WITH LESS	107
31.4	LIMITATIONS OF GENERATIVE AI IN PREDICTIVE ANALYTICS	107
31.4.1	HIGH COMPUTATIONAL COST	107
31.4.2	BLACK-BOX NATURE	107
31.4.3	MODEL BIAS	107
31.4.4	DEPENDENCE ON DATA QUALITY	108
31.4.5	COMPUTATIONAL HARDNESS	108
31.4.6	LACK OF DOMAIN CONTEXT	108
31.4.7	RISK OF OVERFITTING	108
31.4.8	CHALLENGES IN REGULATED INDUSTRIES	108
31.5	KEY TAKEAWAYS	108
32	CONSIDERATIONS WHILE USING GENERATIVE AI IN INDUSTRIES	109
32.1	LEARNING OBJECTIVES	109
32.2	INTRODUCTION	109
32.3	DATA CONSIDERATIONS	109
32.4	MODEL CONSIDERATIONS	109
32.5	ETHICAL CONSIDERATIONS	110
32.6	INDUSTRY-SPECIFIC CONSIDERATIONS	110

32.6.1	FINANCE INDUSTRY	110
32.6.2	HEALTHCARE INDUSTRY	111
32.6.3	RETAIL INDUSTRY	112
32.7	KEY TAKEAWAYS	113

33 CHALLENGES WHILE USING GENERATIVE AI **113**

33.1	LEARNING OBJECTIVES	113
33.2	OVERVIEW	113
33.3	TECHNICAL CHALLENGES	113
33.3.1	TRAINING DATA QUALITY AND AVAILABILITY	114
33.3.2	MODEL INTERPRETABILITY	114
33.3.3	AI HALLUCINATIONS	114
33.3.4	COMPUTATIONAL RESOURCE REQUIREMENTS	114
33.3.5	LACK OF STANDARDIZATION	114
33.4	ORGANIZATIONAL CHALLENGES	114
33.4.1	COPYRIGHT AND INTELLECTUAL PROPERTY ISSUES	115
33.4.2	SKILL GAPS	115
33.4.3	INTEGRATION WITH EXISTING SYSTEMS	115
33.4.4	CHANGE MANAGEMENT	115
33.4.5	MEASURING RETURN ON INVESTMENT (ROI)	116
33.5	CULTURAL CHALLENGES	116
33.5.1	RISK AVERSION	116
33.5.2	DATA SHARING AND COLLABORATION	116
33.5.3	TRUST AND TRANSPARENCY	116
33.5.4	CONTINUOUS LEARNING AND ADAPTATION	117
33.6	KEY TAKEAWAYS	117

34 RESPONSIBLE GENERATIVE AI FOR DATA PROFESSIONALS **117**

34.1	INTRODUCTION	117
34.2	RESPONSIBLE AI	118
34.3	LEGAL AND REGULATORY COMPLIANCE	119
34.4	SOCIAL IMPACT	119
34.5	RESPONSIBLE AI PRACTICES	120
34.6	FURTHER READING	120
34.6.1	IBM RESPONSIBLE AI	120
34.6.2	GOVERNMENT GUIDANCE ON RESPONSIBLE AI	120

35 CASE STUDY: CONSIDERATIONS WHILE USING GENERATIVE AI IN HEALTHCARE **121**

35.1	OVERVIEW	121
35.2	DATA AUGMENTATION	121
35.2.1	KEY CONSIDERATIONS	121

35.2.2	DATA PROFESSIONALS' APPROACH	121
35.3	DIAGNOSTICS	121
35.3.1	KEY CONSIDERATIONS	122
35.3.2	DATA PROFESSIONALS' APPROACH	122
35.4	DRUG DESIGN	122
35.4.1	KEY CONSIDERATIONS	122
35.4.2	DATA PROFESSIONALS' APPROACH	122
35.5	PERSONALIZED HEALTHCARE	123
35.5.1	KEY CONSIDERATIONS	123
35.5.2	DATA PROFESSIONALS' APPROACH	123
35.6	KEY TAKEAWAYS	123
36	RESPONSIBLE USE OF GENERATIVE AI IN REAL-WORLD APPLICATIONS	124
36.1	OVERVIEW	124
36.2	LEARNING OBJECTIVES	124
36.3	EXERCISE 1	124
36.3.1	SCENARIO	124
36.3.2	QUESTION	124
36.3.3	SOLUTION	124
36.3.4	ADDITIONAL INFORMATION	125
36.4	EXERCISE 2	125
36.4.1	SCENARIO	125
36.4.2	QUESTION	125
36.4.3	SOLUTION	125
36.5	EXERCISE 3	125
36.5.1	SCENARIO	125
36.5.2	QUESTION	125
36.5.3	SOLUTION	126
36.6	KEY TAKEAWAYS	126
37	MODULE 2 SUMMARY: USE OF GENERATIVE AI FOR DATA SCIENCE	126
38	MODULE 2 CHEAT SHEET: USE OF GENERATIVE AI FOR DATA SCIENCE	127
38.1	POPULAR GENERATIVE AI TOOLS	127
38.2	IMPORTANT PROMPTS FOR GENERATING DATA INSIGHTS AND VISUALIZATIONS	127
38.3	IMPORTANT PROMPTS FOR MODEL DEVELOPMENT AND REFINEMENT	128
39	REVIEW WHAT WE LEARNED	129
39.1	OVERVIEW	129
39.2	KEY TAKEAWAYS	129

40	FINAL PROJECT OVERVIEW	130
40.1	INTRODUCTION	130
40.2	PROJECT SCENARIO	130
40.3	PROJECT OBJECTIVES	131
40.4	CLIENT-SPECIFIC ANALYSIS REQUIREMENTS	131
40.5	MODEL DEVELOPMENT REQUIREMENTS	131
40.6	ABOUT THE DATASET	131
40.7	DATASET ATTRIBUTES	132
40.8	PROJECT TASKS	132
41	DATA LOADING & PREPARATION – PART 1	132
41.1	LEARNING OBJECTIVES	132
41.2	INTRODUCTION	132
41.3	EXERCISE 1: IMPORT THE DATASET	133
41.3.1	OBJECTIVE	133
41.3.2	PROMPT	133
41.3.3	PYTHON CODE	133
41.4	EXERCISE 2: DATA PREPARATION	133
41.4.1	DATA CLEANING	133
41.4.2	PROMPT	133
41.4.3	PYTHON CODE	133
41.5	SUMMARY	134
42	DATA INSIGHTS AND VISUALIZATION – PART 2	134
42.1	INTRODUCTION	134
42.2	EXERCISE 3: DATA INSIGHTS AND VISUALIZATION	134
42.2.1	IDENTIFY THE TOP 5 CORRELATED FEATURES	134
42.2.2	COUNT THE NUMBER OF CARS BY FUEL TYPE	135
42.2.3	CREATE A BOX PLOT	135
42.2.4	CREATE A REGRESSION PLOT	136
42.3	SUMMARY	136
43	MODEL DEVELOPMENT AND EVALUATION – PART 3	137
43.1	INTRODUCTION	137
43.2	EXERCISE 4: MODEL DEVELOPMENT AND EVALUATION	137
43.2.1	LINEAR REGRESSION USING A SINGLE FEATURE	137
43.2.2	LINEAR REGRESSION USING MULTIPLE FEATURES	137
43.2.3	CREATE A MACHINE LEARNING PIPELINE	138
43.3	SUMMARY	139

44	MODEL OPTIMIZATION – PART 4	139
44.1	INTRODUCTION	139
44.2	EXERCISE 5: MODEL OPTIMIZATION	140
44.2.1	RIDGE REGRESSION	140
44.2.2	POLYNOMIAL RIDGE REGRESSION	140
44.2.3	GRID SEARCH WITH RIDGE REGRESSION	141
44.3	SUMMARY	142
44.4	FINAL PROJECT SUMMARY	143
45	HOW GENERATIVE AI COULD TRANSFORM DATA SCIENCE OVER THE NEXT FIVE YEARS	143
45.1	OVERVIEW	143
45.2	HOW GENERATIVE AI WILL TRANSFORM DATA SCIENCE	143
45.3	AUTOMATION OF REPETITIVE DATA SCIENCE TASKS	144
45.4	MORE ABSTRACTION IN DATA SCIENCE WORKFLOWS	144
45.5	DEMOCRATIZATION OF DATA SCIENCE	145
45.6	EXAMPLE: SYNTHETIC TABULAR DATA GENERATION	145
45.6.1	TRADITIONAL DATA SCIENCE APPROACH	145
45.6.2	GENERATIVE AI APPROACH	145
45.7	SYNTHETIC DATA AND DATA AUGMENTATION	146
45.8	STAYING UPDATED WITH GENERATIVE AI	146
45.9	LEARN THROUGH HANDS-ON PRACTICE	147
45.10	CONTINUOUS LEARNING RESOURCES	147
45.11	READ RESEARCH PAPERS	147
45.12	PARTICIPATE IN THE AI COMMUNITY	148
45.13	CONTINUE EXPERIMENTING	148
45.14	KEY TAKEAWAY	148
46	GENERATIVE AI FOR DATA SCIENCE SKILLS	149
46.1	LEARNING OBJECTIVES	149
46.2	SKILLS REQUIRED TO BECOME A SUCCESSFUL DATA SCIENTIST	149
46.2.1	MATHEMATICAL AND STATISTICAL FOUNDATION	149
46.2.2	PROGRAMMING SKILLS	149
46.2.3	DATA PREPARATION	149
46.2.4	STATISTICAL ANALYSIS	150
46.2.5	MACHINE LEARNING KNOWLEDGE	150
46.2.6	HANDS-ON PROJECT EXPERIENCE	151
46.2.7	STAYING UPDATED WITH INDUSTRY DEVELOPMENTS	151
46.2.8	PRACTICE AND CONTINUOUS LEARNING	151
46.3	KEY TAKEAWAYS	152

Module 1: Data Science and Generative AI

1 Generative AI and Data Science

1.1 Learning Objectives

After studying this topic, we will be able to:

- Describe Generative AI.
- Explain how data scientists use Generative AI in data science.

1.2 What is Generative AI?

Generative AI is a subset of artificial intelligence that focuses on producing new data rather than just analyzing existing data.

It allows machines to create content, including:

- Images
- Music
- Language
- Computer code
- Other forms of content

These creations mimic content produced by people.

1.3 How Generative AI Operates

Deep learning models like **Generative Adversarial Networks (GANs)** and **Variational Autoencoders (VAEs)** are at the foundation of this technique.

These models create new instances that replicate the underlying distribution of the original data by learning patterns from enormous volumes of data.

1.4 Applications of Generative AI

Generative AI has found diverse applications across various industries.

Some examples include:

- **Natural Language Processing (NLP):** OpenAI's GPT-3 can generate human-like text, revolutionizing content creation and chatbots.
- **Healthcare:** Generative AI can synthesize medical images, aiding in the training of medical professionals.
- **Art and Creativity:** Generative AI can create unique and visually stunning artworks, generating endless creative visual compositions.

- **Game Development:** Game developers use Generative AI to generate realistic environments, characters, and game levels.
- **Fashion:** Generative AI assists in fashion by designing new styles and creating personalized shopping recommendations.

1.5 How Data Scientists Use Generative AI

1.5.1 Synthetic Data Generation

Training and testing a model takes lots of data.

Sometimes, the data we want to study does not have enough observations to build a model.

Interest in synthetic data as a tool for analysis and model creation has increased due to recent developments in Generative AI.

Data scientists can augment their datasets using Generative AI to create synthetic data.

It creates this data with similar properties as the real data, such as:

- Distribution
- Clustering
- Many other factors the AI learned about the real dataset

Data scientists can then use the synthetic data along with the real data for model training and testing.

1.5.2 Code Generation for Analytical Models

Data scientists, researchers, and analysts frequently find themselves confined by time when examining data patterns and insights.

Due to this restriction, they can only conceive, develop, and evaluate a small number of hypotheses, leaving many other possibilities untested.

With Generative AI, data scientists can leverage Generative AI to generate and test software code for constructing analytical models.

Coding automation has the potential to revolutionize the field of analytics, allowing the data scientist to focus on higher-level tasks such as:

- Identifying and clarifying the problem the models intend to solve.
- Evaluating hypotheses from a wider range of data sources.

1.5.3 Business Insights and Reporting

Generative AI can generate accurate business insights and comprehensive reports, making it possible to update these insights as the data evolves.

Furthermore, it can autonomously explore data to uncover hidden patterns and insights that might go unnoticed during manual analysis and enhance decision-making.

1.6 Real-World Example

IBM's Cognos Analytics, which provides AI-powered automation, enables us to unlock the full potential of our data with the help of its natural language AI assistant.

We describe the question we are looking to answer or the hypothesis we want to test, and it helps generate the insights we need.

1.7 Key Takeaways

- Generative AI is a subset of artificial intelligence that focuses on producing new data rather than analyzing existing data.
- It uses deep learning models such as **Generative Adversarial Networks (GANs)** and **Variational Autoencoders (VAEs)** to generate new data that follows the patterns of the original dataset.
- Generative AI has applications in natural language processing, healthcare, art, game development, and fashion.
- Data scientists use Generative AI to generate synthetic data for model training and testing when real-world data is limited.
- Generative AI helps automate software code generation for analytical models, allowing data scientists to focus on higher-level analytical tasks.
- It can generate business insights, produce comprehensive reports, and continuously update them as new data becomes available.
- AI-powered tools such as **IBM Cognos Analytics** assist in exploring data, uncovering hidden patterns, testing hypotheses, and improving decision-making.
- Generative AI augments data science efforts, enabling deeper insights, addressing data limitations, and improving the overall quality of data-driven outcomes.

2 Expert Viewpoints: How Generative AI Enhances Data Science Workflows

2.1 Introduction

In this topic, we explore expert perspectives on how **Generative AI (Gen AI)** is currently being used to enhance data science workflows. Experts discuss how Gen AI automates repetitive tasks, accelerates development, improves model performance, and supports various stages of the data science lifecycle.

2.2 How Generative AI Enhances Data Science Workflows

Generative AI helps streamline data science workflows by automating repetitive tasks.

Some key applications include:

- Creating **synthetic data** to balance datasets.

- Generating new features to improve model performance.
- Summarizing large documents.
- Translating languages.
- Automating repetitive processes so that data scientists can focus on more critical aspects of projects, such as analysis and decision-making.

2.3 Generative AI for Coding Tasks

One of the major applications of Generative AI in data science is assisting with coding tasks.

Certain **foundation models** (large AI models trained on vast amounts of data that can perform many different tasks) are capable of:

- Writing code.
- Debugging code.
- Translating code.
- Explaining code.

For example, certain models understand Python libraries such as:

- **Pandas**
- **Scikit-learn**

As a result, they help accelerate the development of data science scripts.

Instead of starting with a blank Jupyter Notebook from scratch, Generative AI can generate a **boilerplate template** (a reusable starting template containing the basic project structure) to help us begin quickly.

2.4 Data Preparation and Automation

Currently, Generative AI is widely used for numerous data preparation tasks.

These include:

- Generating synthetic data, especially when datasets involve privacy concerns.
- Automating data imputation (statistical process of replacing missing or inconsistent values in a dataset with estimated substitutions).
- Detecting outliers.
- Performing data normalization.

2.4.1 Automated Data Imputation

Generative AI models such as:

- Autoencoders
- GANs (Generative Adversarial Networks)

can generate plausible values for missing or corrupted data.

This eliminates the need for manual imputation techniques.

2.5 Popular Generative AI Tools

2.5.1 ChatGPT

When discussing Generative AI, ChatGPT is one of the most widely recognized tools.

It provides valuable assistance by:

- Answering questions.
- Generating code snippets.
- Brainstorming ideas.
- Supporting various technical tasks.

2.5.2 GitHub Copilot

GitHub Copilot is an AI-powered coding assistant developed by GitHub.

It helps by providing:

- Code suggestions.
- Auto-completion.
- Documentation assistance.

Its Visual Studio Code (VS Code) extension includes a chat window that helps us find suitable code solutions for development tasks.

2.6 Technical Documentation

Generative AI also accelerates the creation of technical documentation.

It can assist with:

- Cleaning up GitHub repositories.
- Writing one-page documents.
- Preparing playback presentations.
- Creating solution architectures.

By providing reference data, Generative AI can generate these materials much faster than creating them manually.

2.7 Feature Engineering

Generative AI supports feature engineering by generating additional features using existing data.

These newly generated features can improve the performance of machine learning models.

2.8 Visualization and Reporting

Generative AI helps with:

- Advanced visualizations.
- Report generation.

It enables data scientists to quickly create plots and visualizations without spending excessive time searching documentation or researching suitable visualization techniques on websites such as Stack Overflow.

2.9 Rapid Model Prototyping

Generative AI is also used to rapidly prototype machine learning models.

For example:

- We have a problem.
- We have its corresponding dataset.
- We use Generative AI to quickly iterate through multiple algorithms.
- We determine which algorithm is best suited for solving the problem on the available data.

This significantly reduces experimentation time.

2.10 Expert Examples of Using Generative AI in Data Science

Experts shared several real-world examples of how they use Generative AI in their daily work.

2.10.1 Data Extraction and Transformation

Generative AI is used throughout the data science workflow, including:

- Data extraction.
- Data transformation.

Since no amount of data is ever enough for every problem, Generative AI helps generate synthetic datasets that match project requirements.

2.10.2 Synthetic Data Generation

Experts have observed Generative AI being used to:

- Generate synthetic data when available data is limited.
- Create datasets tailored to specific problem statements.
- Address privacy concerns while maintaining useful training data.

2.10.3 Automated Report Generation

Generative AI can automatically generate reports from raw data, making it easier to present findings to stakeholders.

2.10.4 Documentation and Code Generation

Generative AI reduces time spent on routine tasks by automating:

- Documentation.
- Code generation.

2.11 Research Assistance

Previously, finding suitable research papers required spending considerable time searching platforms such as:

- Google Scholar
- arXiv

With the emergence of Generative AI, researchers now use it to identify the best resources for their research topics more efficiently.

2.12 Enterprise Knowledge Chatbots

One expert described implementing company-specific documents into a language model and deploying a chatbot.

As a result:

- Employees no longer spent long hours searching documents.
- They could retrieve specific information directly from a Generative AI-powered chatbot.

2.13 Computer Vision Applications

Generative AI is used for image synthesis in computer vision projects.

One example involves detecting manufacturing defects.

Benefits include:

- Expanding training datasets.
- Improving the performance of models that identify faults or irregularities in machines.

2.14 Design Optimization

Generative AI can assist in design optimization by:

- Generating new machine component designs.
- Optimizing performance.
- Reducing weight.
- Making materials more efficient.

2.15 Financial Services Applications

For financial services clients, Generative AI is extensively used for Natural Language Processing (NLP).

Examples include:

- Sentiment analysis.
- Entity extraction.
- Topic classification.
- Additional NLP tasks using Large Language Models (LLMs).

2.16 Document Understanding

Generative AI also supports computer vision-based document understanding.

Examples include:

- Reading bank statements and extracting key topics or values to populate databases.
- Reading Requests for Proposal (RFPs) (formal documents organizations use to request project proposals from vendors) and summarizing key concepts.
- Analyzing tables and graphs to extract key insights.

2.17 Machine Learning Code Generation

Generative AI can automatically generate machine learning code.

We simply describe the problem, such as:

- Classification.
- Regression.
- Using Scikit-learn.
- Using a neural network.

Generative AI then generates the complete shell code (the initial project code structure that provides a starting point for development).

2.18 Model Metadata

Generative AI is also useful for collecting model metadata (information that describes a machine learning model, such as its configuration, parameters, performance metrics, and training details).

3 Generative AI Tools for Data Scientists

3.1 Overview

Generative Artificial Intelligence (AI) has revolutionized data science by enabling the creation of new data samples that resemble the original dataset. From generating synthetic images to creating realistic text and simulating complex datasets, these tools serve various purposes in the hands of data scientists.

This topic explores the different tasks data scientists perform using generative AI tools, the purposes of these AI tools, and their applications across diverse domains.

3.2 Synthetic Data Generation

Tasks

- Creating artificial data samples that mimic real-world datasets.

Tools

- TensorFlow Probability
- PyTorch
- SDV (Synthetic Data Vault)
- GANs (Generative Adversarial Networks)

Purpose and Applications

- TensorFlow Probability and PyTorch enable probabilistic modeling.
- SDV provides a framework for generating synthetic data based on statistical models.
- GANs (Generative Adversarial Networks) are a well-known generative modeling technique that excels in creating realistic data samples.
- These tools are used in healthcare for creating synthetic patient records.
- They are used in finance for creating simulated financial transactions.
- They are also used for training machine learning models when the original data is scarce or sensitive.

3.3 Image Generation and Manipulation

Tasks

- Generating synthetic images.
- Modifying visual attributes.
- Creating new designs.

Tools

- StyleGAN
- DALL·E
- BigGAN

Purpose and Applications

- StyleGAN, a specialized implementation and extension of GANs, can create high-quality images.
- DALL·E can create images based on text descriptions.
- BigGAN can generate diverse and realistic images.
- These tools have applications in the art and design, content creation, fashion, and gaming industries.
- They enable the creation of unique images and enhance creative workflows.

3.4 Natural Language Generation

Tasks

- Generating human-like text.
- Creating stories, articles, or dialogue.

Tools

- OpenAI's GPT (Generative Pre-Trained Transformer) models
- Hugging Face's Transformers

Purpose and Applications

- GPT models can generate coherent and context-appropriate text.
- Transformers provide flexibility in text generation and fine-tuning specific tasks.
- Data professionals use these tools in:
 - Chatbots
 - Content generation
 - Automated summarization
 - Conversational AI
- These applications are widely used in media, customer service, and content creation.

3.5 Music and Audio Synthesis

Tasks

- Generating musical compositions.
- Synthesizing audio samples.

Tools

- Magenta
- Jukebox
- NSynth

Purpose and Applications

- Magenta can generate melodies, harmonies, and musical compositions using deep learning techniques.
- Jukebox can create new songs in various genres.
- NSynth can generate new sounds by combining existing ones.
- Artists use these tools in:
 - Music production
 - Gaming
 - Entertainment
- These tools help create original compositions, sound effects, and adaptive soundtracks.

3.6 Simulation and Data Augmentation

Tasks

- Simulating scenarios.
- Augmenting datasets for machine learning models.

Tools

- Unity ML-Agents
- NVIDIA's SimNet
- Augmentor

Purpose and Applications

- Unity ML-Agents can create intelligent agents for simulations.
- SimNet can simulate realistic data.
- Augmentor provides data augmentation techniques.
- Data professionals use these tools in:
 - Robotics
 - Gaming
 - Autonomous vehicles
 - Simulations
- These tools support training AI models and testing algorithms in different environments.

3.7 Content Generation

Tasks

- Creating human-like content such as:
 - Text
 - Images
 - Music

Tools

- OpenAI's GPT models
- DeepDream

- StyleGAN

Purpose and Applications

- GPT models excel in generating coherent text.
- DeepDream creates surreal images.
- StyleGAN ensures realistic image generation.
- Content creators use these tools in:
 - Content creation
 - Storytelling
 - Art
 - Entertainment industries

3.8 Anomaly Detection

Tasks

- Identifying outliers or anomalies in datasets.

Tools

- Autoencoders
- Isolation Forest
- GANs

Purpose and Applications

- Autoencoders can detect anomalies or outliers in the data.
- Isolation Forest can effectively handle anomaly detection in high-dimensional data.
- GANs can generate normal data distributions.
- Data professionals use these tools to detect:
 - Financial fraud
 - Manufacturing errors
 - Cybersecurity threats

3.9 Data Augmentation

Tasks

- Enhancing training datasets by generating variations of existing data.

Tools

- CycleGAN
- Augmentor
- Neural Style Transfer

Purpose and Applications

- CycleGAN can perform image-to-image translation.
- Augmentor can generate augmented images.
- Neural Style Transfer allows the artistic transformation of images based on the style of one image and the content of another.
- Data professionals use these tools in:
 - Computer vision
 - Medical imaging
 - Data augmentation for machine learning models

3.10 Human-Computer Interaction

Tasks

- Enabling human-like interactions through:
 - Chatbots
 - Assistants
 - Avatars

Tools

- Dialogflow
- Rasa
- RunwayML

Purpose and Applications

- Dialogflow and Rasa effectively build conversational AI.
- RunwayML is suited for creative coding.
- These tools are used in:
 - Customer service
 - Virtual assistants
 - Gaming industries
- They help enhance the user experience.

3.11 Conclusion

Generative AI tools allow data scientists to expand datasets, create new content, and simulate scenarios to push the boundaries of innovation and enable advancements across all industries.

These tools are constantly evolving, allowing data scientists to explore new possibilities and make significant discoveries in the field of artificial intelligence and data science.

4 Generative AI's Impact Across Industries

4.1 Learning Objectives

After studying this topic, we will be able to:

- Summarize the impact of Generative AI across industries.

- List the impact of Generative AI on the following industries:
 - Healthcare
 - Finance
 - Retail
 - Manufacturing
 - Media and Entertainment
 - Education
 - Transportation

4.2 Introduction

Generative AI is an impactful branch of Artificial Intelligence that has revolutionized the field of data science by opening up new ways to generate new data, insights, and solutions.

Using deep learning algorithms, data scientists train Generative AI models to create new data that is statistically similar to the original data. This generated data provides valuable insights and helps tackle complex problems across various industries.

The following sections explain the impact of Generative AI on healthcare, finance, retail, manufacturing, media and entertainment, education, and transportation.

4.3 Impact of Generative AI on Healthcare

Generative AI models have a significant impact on the healthcare industry because they can help:

- Discover new drugs.
- Analyze medical images.
- Personalize treatment plans.

4.3.1 Drug Discovery

By analyzing the molecular structure of known medications and their impact on biological targets, Generative AI models can predict new drug candidates with the required properties in a much shorter time.

4.3.2 Medical Image Analysis

Generative AI models can analyze medical images such as:

- X-rays
- MRIs (Magnetic Resonance Imaging)
- CT scans (Computed Tomography scans)

These models help detect possible abnormalities and support earlier detection of diseases.

4.3.3 Personalized Treatment

Generative AI models can predict the risk of developing certain diseases by analyzing:

- Lifestyle factors
- Medical history
- Genetic information

This improves the effectiveness of treatment and reduces adverse drug reactions (unwanted or harmful effects caused by medications).

4.4 Impact of Generative AI on Finance

The financial industry is embracing Generative AI models to:

- Manage risks.
- Detect fraud.
- Create better investment strategies.

4.4.1 Risk Management

Generative AI models can analyze:

- Market trends
- Historical data
- Economic indicators

They can also simulate various financial scenarios, such as:

- Market crashes
- Economic downturns

These simulations help assess potential risks and develop strategies to minimize losses.

4.4.2 Fraud Detection

Generative AI models can flag potentially fraudulent activities by:

- Analyzing transaction patterns.
- Identifying anomalies (unusual or unexpected patterns).

This helps prevent losses due to fraud.

4.4.3 Investment Strategies

Generative AI models can analyze financial data and investment patterns to help financial advisors recommend personalized and profitable investment portfolios for their clients.

4.5 Impact of Generative AI on Retail

The retail industry utilizes Generative AI models to:

- Enhance customer experience.

- Develop products.
- Optimize the supply chain.

4.5.1 Customer Experience

Generative AI models analyze:

- Customer behavior
- Purchase patterns
- Customer feedback

This enables retailers to personalize:

- Product recommendations
- Marketing campaigns
- Customer service interactions

4.5.2 Product Development

Generative AI models identify popular features and styles to help product designers create products that resonate with the target audience.

4.5.3 Supply Chain Optimization

Generative AI can:

- Predict demand patterns.
- Identify potential disruptions.

This capability helps retailers create effective inventory management strategies.

4.6 Impact of Generative AI on Manufacturing

The manufacturing industry is leveraging Generative AI models to:

- Optimize production processes.
- Improve product design.
- Enhance quality control.

4.6.1 Production Optimization

Generative AI can simulate various production scenarios and identify bottlenecks (points that slow down production) to help manufacturers:

- Increase production efficiency.
- Reduce waste.
- Minimize downtime.

4.6.2 Product Design

These models help design functional and cost-effective products by analyzing:

- Engineering data
- Material properties
- Manufacturing constraints

4.6.3 Quality Control

Generative AI models help manufacturers enhance quality control by:

- Analyzing product data.
- Identifying defects.
- Predicting potential failures.

4.7 Impact of Generative AI on Media and Entertainment

The media and entertainment industry is embracing Generative AI models to:

- Create new forms of content.
- Personalize user experiences.
- Enhance creative workflows.

4.7.1 Content Creation

These models can generate realistic:

- Images
- Videos
- Music

The entertainment industry can use this content in:

- Television shows
- Movies
- Video games

4.7.2 Personalized User Experience

Generative AI models analyze:

- User preferences
- Viewing history
- Demographic data

This helps:

- Recommend personalized content.

- Tailor advertisements.
- Create customized user interfaces.

4.7.3 Creative Assistance

Generative AI models assist:

- Writers
- Artists
- Musicians

They help by:

- Developing new ideas.
- Generating creative variations.
- Exploring uncharted territories in their respective fields.

This enables creators to produce original and engaging content for their audiences.

4.8 Impact of Generative AI on Education

The education sector is using Generative AI models to:

- Personalize learning experiences.
- Provide real-time feedback.
- Create adaptive learning materials.

4.8.1 Personalized Learning

Generative AI models analyze student data such as:

- Test scores
- Learning styles
- Individual needs

These models can:

- Create tailored learning plans.
- Recommend appropriate resources.
- Provide personalized feedback.

4.8.2 Real-Time Feedback

Generative AI models can:

- Assess student comprehension.
- Identify areas of difficulty.
- Provide real-time feedback.

This helps students identify their strengths and weaknesses.

4.8.3 Adaptive Learning Materials

Generative AI models can create customized learning materials that adapt to individual learning speeds, helping educators develop adaptive learning materials for their students.

4.9 Impact of Generative AI on Transportation

The transportation industry is exploring the potential of Generative AI models to:

- Improve traffic flow.
- Optimize transportation systems.
- Enhance safety.

4.9.1 Traffic Management

Generative AI models can predict traffic patterns based on:

- Historical data
- Real-time traffic sensors
- Weather forecasts

The models help:

- Optimize traffic signals.
- Adjust speed limits.
- Reroute traffic.

These capabilities help reduce traffic congestion.

4.9.2 Transportation System Optimization

Generative AI models analyze transportation networks, including:

- Public transit systems
- Road infrastructure
- Logistics networks

This helps identify potential bottlenecks and suggest improvements.

4.9.3 Transportation Safety

Generative AI models analyze:

- Accident data
- Patterns
- Risk factors

This helps:

- Reduce the number of accidents.
- Improve overall safety in the transportation sector.

4.10 Key Takeaways

- Generative AI can tackle complex problems across various industries.
- The healthcare industry uses Generative AI models to discover drugs, diagnose diseases, and personalize treatment plans.
- The financial industry uses these models to manage risks, detect fraud, and create better investment strategies.
- The retail industry utilizes Generative AI models to enhance customer experience, develop products, and optimize the supply chain.
- The manufacturing industry leverages these models to optimize production processes, improve product design, and enhance quality control.
- The media and entertainment industry embraces these models to create new content, personalize user experiences, and enhance creative workflows.
- The education sector uses Generative AI models to personalize learning experiences, provide real-time feedback, and create adaptive learning materials.
- The transportation industry is exploring these models to improve traffic flow, optimize transportation systems, and enhance safety.

5 Skills Data Professionals Should Develop to Effectively Leverage Generative AI (GenAI)

5.1 Introduction

To effectively leverage or use Generative AI (GenAI) in projects, data professionals should focus on developing a combination of technical, analytical, and soft skills. Building expertise across programming, machine learning, deep learning, prompt engineering, AI workflows, deployment, and ethical decision-making enables us to successfully develop and implement AI solutions in real-world projects.

5.2 Programming Skills

Programming forms the foundation for working with Generative AI.

- Learn programming languages, especially **Python**, as it is the best choice for most AI and Data Science-related frameworks.
- Develop proficiency in **R**, which is also widely used for working with Generative AI tools.

5.3 Machine Learning Fundamentals

A strong understanding of Machine Learning is essential before working with Generative AI.

We should develop knowledge of:

- Machine Learning algorithms and how to implement them.
- Regression.
- Classification.
- AI model training.
- Data manipulation.
- Evaluating AI-generated outcomes.

To implement Machine Learning models, we should gain experience with frameworks such as:

- TensorFlow
- PyTorch
- Keras

5.4 Mathematical Foundations

Understanding the mathematical concepts behind AI helps us better understand how Generative AI models work.

Important topics include:

- Linear Algebra.
- Matrix Computations.

5.5 Deep Learning Foundations

Before learning modern Generative AI models, we should understand the deep learning architectures that paved the way for them.

Key concepts include:

- Recurrent Neural Networks (RNNs)
- Convolutional Neural Networks (CNNs)
- Attention Mechanism
- Natural Language Processing (NLP) concepts

5.6 Generative AI Model Architectures

We should become familiar with the major model architectures associated with Generative AI, including:

- Transformers
- Encoder Models
- Decoder Models
- Embeddings (numerical representations of data that capture semantic meaning)
- Diffusion Models
- GANs (Generative Adversarial Networks)

Understanding these architectures provides a solid foundation for learning the complete Generative AI ecosystem.

5.7 Understanding How GenAI Works

The first and foremost requirement is understanding how Generative AI works.

We should understand:

- How GenAI is trained.
- What kind of data it is trained on.
- Its strengths.
- Its limitations.
- Where GenAI should be used.
- Where GenAI should not be used.

This knowledge helps us make informed decisions about when and how to apply Generative AI effectively.

5.8 Prompt Engineering

Prompt Engineering is one of the most important skills for working with Generative AI.

Each model has different best practices for prompting, so we should learn how to effectively communicate with different AI models to obtain the desired output.

Important Prompt Engineering concepts include:

- Prompt Engineering.
- Few-shot Prompting (providing a few examples within the prompt to guide the model).
- Creating memory for chatbots.
- Chaining prompts using **LangChain** (a framework for connecting multiple prompts, tools, and AI components into workflows).

As of now, prompt engineering is the primary medium of communication with Generative AI models.

Remember:

With GenAI, it's not about doing, it's about getting your work done.

5.9 Advanced AI Workflows

After mastering the fundamentals, we should explore advanced AI techniques, including:

- Retrieval-Augmented Generation (RAG) (combining external knowledge retrieval with AI-generated responses).
- Embeddings.
- Agentic Workflows (AI systems capable of planning and performing tasks autonomously).

5.10 Model Fine-Tuning

Develop skills in improving foundation models through various tuning techniques.

Important topics include:

- Prompt Tuning.
- Fine-Tuning.
- PEFT (Parameter-Efficient Fine-Tuning).
- Instruction Tuning of a Base Model.

5.11 AI Libraries and Frameworks

Develop practical experience with widely used AI libraries and frameworks, including:

- TensorFlow
- PyTorch
- Keras
- Hugging Face
- Scikit-learn
- LangChain

5.12 Data Visualization Skills

Data visualization helps us interpret AI-generated results more effectively.

Useful visualization tools include:

- Matplotlib
- Plotly
- Cmode

5.13 Model Deployment

Beyond model development, we should learn how to deploy AI solutions.

Important deployment technologies include:

- Docker
- Kubernetes
- Cloud Platforms

5.14 Evaluating AI-Generated Solutions

Data professionals should develop the ability to quickly evaluate:

- AI-generated code.
- AI-generated solutions.

The faster we evaluate outputs, the more iterations we can perform, leading to better and more refined solutions.

5.15 Essential Soft Skills

Technical expertise alone is not enough. Successful AI professionals should also develop strong soft skills, including:

- Problem Solving.
- Adaptability.
- Ethical Decision-Making.

These skills play a significant role in implementing AI successfully in real-world applications.

5.16 Key Takeaways

To effectively leverage Generative AI in projects, we should develop expertise in the following areas:

- Python and R programming.
- Machine Learning algorithms.
- Linear Algebra and Matrix Computations.
- Deep Learning fundamentals.
- Transformers, Encoder Models, Decoder Models, Embeddings, Diffusion Models, and GANs.
- Understanding how GenAI works, including its strengths and limitations.
- Prompt Engineering and advanced prompting techniques.
- LangChain and chatbot memory creation.
- Retrieval-Augmented Generation (RAG).
- Agentic Workflows.
- Prompt Tuning, Fine-Tuning, PEFT, and Instruction Tuning.
- TensorFlow, PyTorch, Keras, Hugging Face, and Scikit-learn.
- Data Visualization using Matplotlib, Cmode, and Plotly.
- Model deployment using Docker, Kubernetes, and Cloud Platforms.
- Rapid evaluation of AI-generated code and solutions.
- Problem Solving, Adaptability, and Ethical Decision-Making.

6 Leveraging Generative AI in the Data Science Life Cycle

6.1 Learning Objectives

After completing this topic, we will be able to:

- List the five phases of the data science life cycle.
- Discuss how Generative AI provides innovative tools to improve every phase of the data science life cycle.

6.2 Introduction

The **data science life cycle** is a structured approach to delivering knowledge and insights from data. It is a series of interconnected phases that transform raw data into actionable insights.

Generative AI is a branch of artificial intelligence that generates new data. It has proven to be a transformative force in the data science life cycle by providing data scientists with powerful tools to analyze data, uncover hidden insights, and develop innovative solutions.

The data science life cycle comprises the following five phases:

- Problem Definition and Business Understanding
- Data Acquisition and Preparation
- Model Development and Training
- Model Evaluation and Refinement
- Model Deployment and Monitoring

Generative AI has revolutionized the data science life cycle by providing innovative tools to improve every process phase.

6.3 Problem Definition and Business Understanding

The **Problem Definition and Business Understanding** phase involves clearly defining the problem and understanding the business context of the data.

In this phase, Generative AI can:

- Generate new ideas and solutions by mimicking existing product descriptions, marketing campaigns, or successful solutions in other industries.
- Generate synthetic customer profiles (artificially created customer data that resembles real customer characteristics) to understand diverse needs and preferences, informing product development and targeted marketing strategies.
- Simulate economic conditions, competitor actions, and market trends to assess opportunities and potential risks before investing in data gathering or model development.

6.3.1 Real-World Example

A **pharmaceutical company** uses Generative AI to analyze synthetic patient profiles and generate potential drug targets for rare diseases.

6.4 Data Acquisition and Preparation

The **Data Acquisition and Preparation** phase includes gathering accurate and consistent data from various sources and running preprocessing steps for preparing data for modeling and analysis.

In this phase, Generative AI can:

- Fill in missing values in datasets to improve data quality and model training accuracy.
- Augment data by generating synthetic data points to:
 - Balance skewed datasets.
 - Expand training datasets.
 - Improve model generalizability (the ability of a model to perform well on unseen data) to unseen scenarios.
- Detect anomalies by training generative models on standard data patterns to identify outliers (data points that differ significantly from the rest of the data) and potential security threats in real-time data streams.

6.4.1 Real-World Example

A **manufacturing company** uses Generative AI to fill in missing sensor data on production lines for predictive maintenance and anomaly detection.

6.5 Model Development and Training

The **Model Development and Training** phase includes selecting and training appropriate machine learning algorithms to generate insights and patterns from the data.

In this phase, Generative AI can:

- Perform feature engineering by generating diverse and representative features to address feature scarcity and improve model performance.
- Explore many hyperparameter combinations and accelerate model optimization efficiently.
- Generate textual explanations or visual representations of complex model predictions, improving interpretability (making model decisions easier to understand) and trust in AI-driven decisions.

6.5.1 Real-World Example

A **financial institution** uses Generative AI to explore different feature combinations and optimize a fraud detection model with higher accuracy and explainability.

6.6 Model Evaluation and Refinement

The **Model Evaluation and Refinement** phase involves evaluating the performance of trained models to identify areas for improvement and ensure their generalizability.

In this phase, Generative AI can:

- Generate adversarial or edge cases (challenging or unusual test cases designed to expose model weaknesses) to test the model's robustness against malicious attacks or unusual scenarios.
- Train a generative model to mimic model uncertainty, highlighting cases where predictions are unreliable and require further scrutiny.

- Perform counterfactual reasoning (evaluating "what if" scenarios by changing one or more variables) by modeling existing data points to assess the impact of different variables on model predictions and refine decision-making strategies.

6.6.1 Real-World Example

A **self-driving car company** uses Generative AI to test its models against extreme weather conditions and assess potential risks before real-world deployment.

6.7 Model Deployment and Monitoring

The **Model Deployment and Monitoring** phase includes integrating trained models into real-world applications or systems and continuously monitoring their performance.

In this phase, Generative AI can:

- Continuously monitor real-time data using a generative AI model trained on the initial training data to detect data drift (changes in the statistical properties of incoming data over time) and trigger model retraining when necessary.
- Provide personalized experiences by generating dynamic content or recommendations tailored to individual user preferences or contexts, enriching user experience and engagement.
- Perform A/B testing by generating variations of marketing campaigns or product features and testing them on small subgroups of users before wider rollout, optimizing performance based on real-time feedback.

6.7.1 Real-World Example

A **streaming service** uses Generative AI to recommend personalized content to each user based on their unique viewing histories and preferences.

6.8 Summary

The data science life cycle comprises the following five phases:

- Problem Definition and Business Understanding
- Data Acquisition and Preparation
- Model Development and Training
- Model Evaluation and Refinement
- Model Deployment and Monitoring

6.8.1 How Generative AI Enhances Each Phase

Data Science Life Cycle Phase	How Generative AI Helps
Problem Definition and Business Understanding	Idea generation, customer segmentation, simulation scenarios
Data Acquisition and Preparation	Missing data imputation, data augmentation, anomaly detection

Model Development and Training	Feature engineering, AutoML exploration, hyperparameter tuning, improved interpretability
Model Evaluation and Refinement	Stress testing, uncertainty estimation, counterfactual reasoning (what-if scenario analysis)
Model Deployment and Monitoring	Data drift detection, personalized experiences, A/B testing

7 Types of Generative AI Models

7.1 Learning Objectives

After completing this topic, we will be able to:

- List the four common types of generative AI models.
- Discuss the strengths of each generative AI model.
- Explain the applications of each model in data science.

7.2 Introduction

Generative AI models represent a fascinating domain within the field of machine learning. Data scientists can use these models to create new content, including text, images, audio, and other data types.

The four common types of generative AI models are:

- Generative Adversarial Networks (GANs)
- Variational Autoencoders (VAEs)
- Autoregressive Models
- Flow-Based Models

The following sections discuss each model, its strengths, applications, and real-world examples.

7.3 Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are renowned for their ability to imitate and transcend reality.

A GAN consists of two neural networks:

- **Generator** – Produces increasingly realistic data.
- **Discriminator** – Distinguishes between real and fake samples.

This process drives the generator to continually refine its outputs, creating remarkably authentic data.

7.3.1 Strengths of GANs

- Generate highly realistic and diverse data that is often indistinguishable from genuine data.

- Generate complex data across multiple modalities, including:
 - Images
 - Videos
 - Music

7.3.2 Applications of GANs

GANs are widely used for:

- Generating new images.
- Editing existing images.
- Enhancing image quality.
- Composing original music.
- Personalizing playlists.
- Generating text.
- Producing creative content.
- Translating languages.
- Summarizing text.
- Data augmentation (expanding limited datasets for training other machine learning models).

7.3.3 Example

StyleGAN is a notable GAN architecture specifically designed for generating high-fidelity images of faces with diverse styles and attributes.

7.4 Variational Autoencoders (VAEs)

Variational Autoencoders (VAEs) discover the underlying patterns that govern data organization.

They encode data into a **latent representation** (a compressed representation that captures the essential characteristics of the original data) and preserve the inherent structure of the original data.

This latent representation serves as the foundation for generating new data samples.

7.4.1 Strengths of VAEs

- Discover the underlying structure of data.
- Generate new samples that follow inherent patterns.
- Efficient and scalable for handling large datasets.
- Effective for anomaly detection by identifying outliers and unexpected patterns.
- Compress datasets without compromising information content.

7.4.2 Applications of VAEs

VAEs are commonly used for:

- Anomaly detection.
- Data compression.
- Collaborative filtering (recommending products, movies, or music based on user preferences).
- Image style transfer (transforming the style of one image into another).

7.4.3 Example

VAEGAN is a hybrid model that combines VAEs and GANs to generate high-quality and diverse images of faces.

7.5 Autoregressive Models

Autoregressive models are sequential data champions.

These models excel at handling sequential data such as:

- Text
- Time series

They generate data one element at a time by predicting the next element based on previously generated elements, enabling the creation of coherent and consistent sequences.

7.5.1 Strengths of Autoregressive Models

- Simple and interpretable, making them easier to understand and debug.
- Highly effective for sequential data.
- Produce coherent and grammatically correct sequences.

7.5.2 Applications of Autoregressive Models

Autoregressive models are commonly used for:

- Generating poetry.
- Generating scripts.
- Generating emails.
- Converting text into natural-sounding speech.
- Time series forecasting (predicting future trends and patterns in time-dependent data).
- Machine translation with fluent and accurate language generation.

7.5.3 Example

Generative Pre-trained Transformers (GPTs) are prominent autoregressive models that:

- Generate human-quality text.
- Translate languages.
- Produce diverse creative content.

7.6 Flow-Based Models

Flow-based models unveil the probability landscape of data.

These models directly model the **probability distribution** (the mathematical representation of how data values are distributed) of data, enabling efficient sampling and generation.

They transform complex data into more straightforward representations while generating new data that follows the original probability distribution.

7.6.1 Strengths of Flow-Based Models

- Directly model probability distributions.
- Enable efficient sampling and data generation.
- Offer flexible and adaptable architectures that can be tailored to specific data characteristics and task requirements.

7.6.2 Applications of Flow-Based Models

Flow-based models are widely used for:

- Generating high-quality images with realistic details and textures.
- Simulating synthetic data.
- Identifying unusual patterns in data distribution that may indicate fraudulent activity.
- Density estimation (estimating the probability density function to gain insights into data distribution).

7.6.3 Example

RealNVP is a well-known flow-based model that generates high-quality images of human faces.

7.7 Summary

The four common generative AI models are:

Model	Major Strengths	Common Applications	Example
Generative Adversarial Networks (GANs)	Highly realistic and diverse data generation	Image generation, image editing, image enhancement, music generation, text generation, language translation, text summarization, data augmentation	StyleGAN
Variational Autoencoders (VAEs)	Discover underlying data structure, scalable, efficient, anomaly	Anomaly detection, data compression, collaborative filtering, image style transfer	VAEGAN

	detection, data compression		
Autoregressive Models	Effective for sequential data, simple, interpretable	Text generation, speech synthesis, time series forecasting, machine translation	Generative Pre-trained Transformers (GPTs)
Flow-Based Models	Direct probability modeling, efficient sampling, flexible architecture	Image generation, synthetic data generation, anomaly detection, density estimation	RealNVP

7.8 Key Takeaways

- Generative AI models create new content such as text, images, audio, and other data types.
- **GANs** excel at image, music, and text generation, as well as data augmentation.
- **VAEs** are effective for anomaly detection, data compression, collaborative filtering, and style transfer.
- **Autoregressive models** are well suited for text generation, speech synthesis, time series forecasting, and machine translation.
- **Flow-based models** are suitable for image generation, synthetic data generation, anomaly detection, and density estimation.

8 Guide to Choosing a Generative AI Model Type

8.1 Types of Generative AI Models

Model	Key Features	Applications
Generative Adversarial Networks (GANs)	• Two competing neural networks: generator and discriminator. • The generator learns to create realistic data, while the discriminator learns to distinguish real from fake. • The adversarial training process continuously improves both networks. • Can be challenging to train and achieve stable results.	• Image generation: faces, landscapes, objects • Text generation: poems, code, scripts • Video generation: realistic videos, animation • Drug discovery: generate molecules with intended properties • Music generation: composing new songs
Variational Autoencoders (VAEs)	• Encode input data into a lower-dimensional latent space. • Learn a probability distribution over the latent space. • Decode samples from the latent space to generate new data points.	• Image compression: efficiently stores and transmits images

	• Focus on learning a meaningful representation of the data.	• Anomaly detection: identify unusual data points • Dimensionality reduction: compress high-dimensional data • Text summarization: generate concise summaries of text documents
Autoregressive Models	• Generate data point by point, conditioned on previously generated points. • Use recurrent neural networks (RNNs) or transformers to capture long-term dependencies. • Can be computationally expensive for long sequences.	• Text generation: realistic and coherent text sequences • Music generation: generating music that follows genre and style • Time series forecasting: predicting future values of a time series • Image inpainting: filling in missing parts of an image
Diffusion Models	• Start with simple noise and gradually de-noise it into realistic data. • Use a U-Net architecture (a neural network architecture designed for image segmentation and generation) with skip connections (connections that preserve important information by bypassing intermediate layers). • Can be more stable and easier to train than GANs, but often slower.	• Image generation: high-quality and diverse images • Text generation: coherent and grammatically correct text • Audio generation: realistic and musical audio • Inpainting and denoising: improving the quality of images or audio
Flow-based Models	• Transform a simple distribution (Gaussian (a normal probability distribution)) into a complex one using invertible transformations (transformations that can be reversed without losing information).	• Image generation: realistic and diverse images • Density estimation: modeling the

	• Learn the parameters of these transformations from the data. • Can be efficient and accurate for high-dimensional data, but training can be challenging.	probability distribution of data • Dimensionality reduction: compress high-dimensional data • Anomaly detection: identify unusual data points
--	---	--

8.2 Comparison of Models on Different Considerations

Feature	GANs	VAEs	Autoregressive Models	Diffusion Models	Flow-based Models
Data Type	Images, text, audio	Images, text, continuous data	Images, text, sequences	Images, text	Images, continuous data
Task Objective	High-fidelity generation, data augmentation	Encoding/decoding, representation learning	Sequence generation, text-to-image translation	Image generation, editing, inpainting	Image generation, conditional generation
Quality of Samples	High-fidelity, diverse	Often blurry, less realistic	Sharp, high-resolution	High-fidelity, diverse	High-fidelity, controllable
Control over Generation	Limited	Moderate	High	Moderate	High
Training Complexity	High	Moderate	High	Moderate	High
Interpretability	Low	Moderate	High	Moderate	Low

9 Case Study: Successful Implementation of Generative AI

9.1 Introduction

StreamCraft, a rapidly growing online streaming platform, faced a critical challenge: user engagement was plateauing. Traditional recommendation engines struggled to break through the "**filter bubble**" (a situation in which users repeatedly receive similar content based on their previous preferences), thereby offering predictable content and failing to entice viewers to explore new genres or hidden gems.

To address this challenge, StreamCraft decided to integrate **Generative AI (GAI)** into its data science workflow at every stage of the content discovery process.

9.2 Data Ingestion and Preprocessing

9.2.1 Text Analysis with Generative Pre-trained Transformers (GPTs)

GPTs were used to analyze movie descriptions, reviews, and social media mentions, extracting hidden semantic relationships and identifying emotional tones. This enriched metadata, allowing StreamCraft to categorize content beyond genre labels while capturing nuanced themes and moods.

9.2.2 Automatic Genre and Subgenre Tagging

GPTs further enabled the creation of dynamic genre tags that evolved with the content landscape. This allowed StreamCraft to identify emerging trends and niche subgenres, catering to specific viewer preferences.

9.2.3 AI-Powered Anomaly Detection

StreamCraft employed GAI to identify and flag unusual patterns in user behavior and content engagement. This facilitated proactive intervention against fraudulent activity or potential technical glitches, ensuring a smooth user experience.

9.3 Feature Engineering and Model Training

9.3.1 Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) were used to create synthetic user profiles, mimicking the behavior of actual viewers with specific interests. This expanded the training data, enabling the recommendation engine to learn from a wider range of preferences and make more accurate predictions.

9.3.2 Counterfactual Reasoning

GAI models were trained to predict what users would watch under hypothetical scenarios, simulating their response to different content offerings. This allowed StreamCraft to test various recommendation strategies and optimize content placement without impacting real users.

9.3.3 Explainable AI (XAI) for Model Interpretability

StreamCraft implemented **Explainable AI (XAI)** techniques to understand the reasoning behind the recommendations generated by their GAI models. This enhanced transparency, allowing for trust-building with users and enabling targeted improvements to the recommendation system.

9.4 Deployment and Monitoring

9.4.1 Adaptive Content Recommendations

StreamCraft's GAI-powered engine delivered dynamic, context-aware recommendations, adapting to real-time user behavior and external factors such as trending topics or seasonal

events. This personalized experience kept viewers engaged and encouraged exploration beyond their usual preferences.

9.4.2 A/B Testing with Generative Models

GAI facilitated rapid testing of different recommendation algorithms and content layouts. StreamCraft could generate multiple versions of its user interface and test them with simulated user data, identifying the most effective layouts and content combinations before deploying them to real users.

9.4.3 Continuous Feedback Loop

StreamCraft incorporated user feedback into its GAI models, allowing the system to learn and adapt over time. This ensured the recommendations remained relevant and user-centric, creating a virtuous cycle of improved engagement and retention.

9.5 Conclusion

StreamCraft's GAI implementation resulted in a significant increase in user engagement:

- Viewership time per user grew by **25%**.
- Discovery of new content genres increased by **30%**.
- The platform experienced a **15% reduction in churn** (the percentage of users who stopped using it), indicating a stronger relationship with its viewers.

This success story showcases the transformative potential of **Generative AI (GAI)** in the online streaming industry, paving the way for personalized, dynamic, and ultimately more engaging experiences for viewers.

10 Explore a Simple Generative Tool

10.1 Learning Objectives

After completing these exercises, we will be able to:

- Sign up for the DataRobot platform.
- Add a dataset to a use case.
- Build and explore machine learning models using DataRobot.

10.2 Introduction

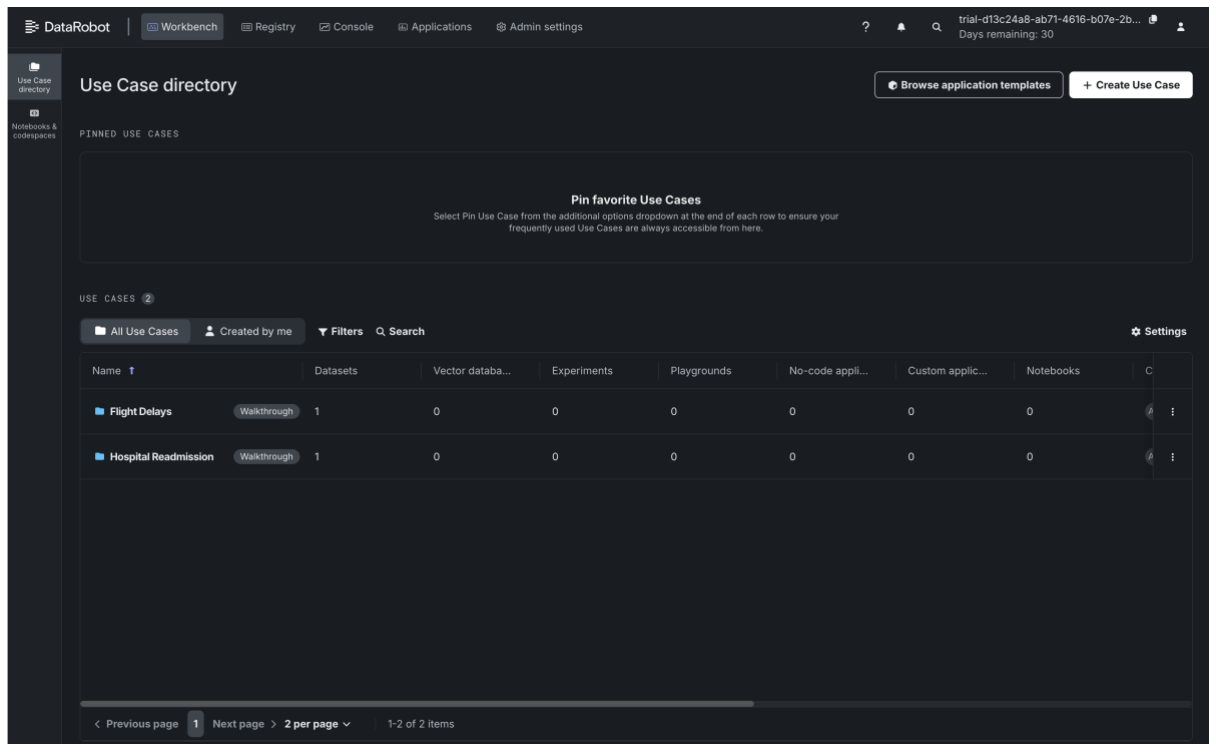
Generative AI tools simplify many aspects of data science and machine learning by automating data preparation, model building, prediction, and analysis. In this topic, we explore the DataRobot platform to understand how automated machine learning (AutoML) can streamline the model development workflow.

10.3 Exercise 1: Sign Up in DataRobot

10.3.1 Step

- Sign up for the DataRobot platform.

Note: DataRobot requires a work email address for registration. Alternatively, we can create a GitHub account using a Gmail address and use GitHub credentials to sign in to DataRobot. Also, we can choose the Sign in with Google method to sign in to the platform.



10.4 Exercise 2: Add a Dataset

In this exercise, we create a new use case and add a dataset for model development.

10.4.1 Steps

- Open **Workbench**.
- Create a new **Use Case**.
- Browse the available datasets.
- Select the sample dataset **HOSPITAL_READMISSION_DATA**.
- Preview the dataset and its features.
- Add the dataset to the use case.
- Explore the dataset features using the **All Features** menu.

DataRobot
Workbench
Registry
Console
Applications
Admin settings
?
trial-d13c24a8-ab71-4616-b07e-2b...
Days remaining: 30

Browse data
Cancel
Add to Use Case

DataRobot assets
Data Registry
Datasets
3

Data connections
+ Add connection
Demo Connection
Snowflake

HOSPITAL_READMISSION_DATA
Data Registry / HOSPITAL_READMISSIO...
Features
Data preview

race	gender	age	weight	admission_type_id	discharge_disposition...	
Caucasian	Male	[50-60]		Emergency	Discharged to home	
Caucasian	Female	[70-80]		Urgent	Discharged/transferred to...	
Caucasian	Male	[60-70]		Elective		
Caucasian	Male	[60-70]			Discharged to home	
Caucasian	Female	[40-50]		Not Available	Discharged to home	
Caucasian	Male	[90-100]		Emergency	Discharged/transferred to...	
Caucasian	Male	[70-80]		Urgent	Discharged/transferred to...	
Caucasian	Female	[80-90]		Urgent	Discharged to home	
Hispanic	Female	[50-60]		Emergency	Discharged to home	
Caucasian	Male	[80-90]		Emergency	Discharged to home	
Caucasian	Male	[60-70]		Urgent	Discharged/transferred to...	
Caucasian	Female	[50-60]	[75-100]	Elective	Discharged to home	
Caucasian	Male	[60-70]		Urgent	Discharged to home	
Caucasian	Male	[20-30]		Urgent	Left AMA	
	Male	[50-60]		Emergency	Discharged/transferred to...	
Caucasian	Male	[30-40]		Urgent	Discharged to home	
AfricanAmerican	Female	[60-70]		Elective	Discharged to home	
Caucasian	Male	[50-60]		Urgent	Discharged/transferred to...	

Snapshot sample
51 features
1,511 rows

DataRobot
Workbench
Registry
Console
Applications
Admin settings
?
trial-d13c24a8-ab71-4616-b07e-2b...
Days remaining: 30

Use Case directory / Use_Case_1 / HOSPITAL_READMISSION_DATA

HOSPITAL_READMISSION_DATA
Jul 26th, 2026 11:16 AM
Snapshot
Data actions

Show insights
Show features from: All Features
+ Create feature list
Search
Show summary

race	gender	age	weight	admission_type_id	discharge_disposition...	admission_source_id	time...
Caucasian	Female	[70-80]	26%	Emergency	Discharged to home	Emergency Room	2.0
AfricanAmerican	Male	[60-70]	22%	Urgent	Discharged/transferred to...	Physician Referral	3.0
Other		Other	52%	Other	Other	Other	Other
Caucasian	Male	[50-60]		Emergency	Discharged to home		1
Caucasian	Female	[70-80]		Urgent	Discharged/transferred to...	Clinic Referral	10
Caucasian	Male	[60-70]		Elective		Physician Referral	7
Caucasian	Male	[60-70]			Discharged to home	Emergency Room	4

38

10.5 Exercise 3: Work on Data Modeling

In this exercise, we build a machine learning model using the uploaded dataset and explore the generated results.

The screenshot shows the DataRobot Workbench interface for the 'HOSPITAL_READMISSION_DATA' dataset. The 'Data actions' menu is open, with 'Start modeling' highlighted. The interface displays various feature distributions and a summary of the dataset.

Dataset Summary:

- DATE ADDED: July 29, 2024
- ADDED BY: Pratiksha Verma
- FEATURES: 51
- ROWS: 10,000
- SIZE: 5.48 MB
- DATABASE: Snowflake / Demo Connection
- SCHEMA: TRIAL_READONLY
- TABLE: HOSPITAL_READMISSION_DATA

Feature Distributions:

- race:** Caucasian (74%), AfricanAmerican (21%), Other (6%)
- gender:** Female (54%), Male (46%)
- age:** [70-80] (26%), [60-70] (22%), [50-60] (22%), [40-50] (19%), [30-40] (19%), [20-30] (19%), [10-20] (19%), [0-10] (19%)
- weight:** ==Missing== (96%), [75-100] (2%), Other (2%)
- admission_type:** Emergency (49%), Urgent (19%), Discharge (32%), Other (2%)

The screenshot shows the 'Set up new experiment' interface in DataRobot Workbench. The 'Modeling' tab is selected, and the experiment is configured for supervised learning. The target feature is 'readmitted'. The experiment summary on the right shows the dataset name, target feature, and modeling settings.

Experiment Configuration:

- Learning type:** Supervised
- Target feature:** readmitted
- Target type:** Binary classification
- Positive class:** 1
- Modeling mode:** Quick Autopilot
- Optimization metric:** LogLoss (Accuracy) (Recommended)
- Training feature list:** Informative Features

Experiment Summary:

- Name:** HOSPITAL_READMISSION_DATA - 2026-07-26 11:43:24
- Description:** Add experiment description.
- Dataset:** HOSPITAL_READMISSION_DATA
- Rows:** 10,000
- Features:** 51
- Modeling:** Learning type: Supervised, Target feature: readmitted, Target type: Binary classification, Positive class: 1, Modeling mode: Quick Autopilot, Optimization metric: LogLoss, Training feature list: Informative Features
- Partitioning:** Method: Stratified sampling, Validation type: Cross-validation, Details: 5 cross-validation folds, 20% holdout
- Time series modeling:** Not available
- Additional settings:** No settings applied

10.5.1 Steps

- Select **Modeling** from **Data Actions**.
- Prepare the dataset for modeling.
- Select **readmitted** as the target feature.
- Configure additional model settings if required.
- Start the modeling process.
- Review the generated models after training is complete.
- Explore the **Model Overview**, including:

- Blueprint
- Feature Impact
- Other model insights
- Generate predictions using unseen or test data.
- Generate and download the compliance report for the selected model.

DataRobot | Workbench | Registry | Console | Applications | Admin settings

trial-d13c24a8-ab71-4616-b07e-2b... Days remaining: 30

Set up new experiment

Dataset Modeling Additional settings

Exit < Back Start modeling

Data partitioning / Incremental modeling / Time series modeling / Additional settings

Partitioning method
Select the method for assigning rows to partitions when training models.

Stratified sampling
Rows are assigned to ensure similar target distribution across each partition.

Validation type

☒ **Cross-validation**
Trains models on a specified number of folds, maximizing data use but also increasing run time.

☐ **Training-validation-holdout**
Splits data into three partitions: trains models on the training set, assess performance on the validation set, and evaluates the model on unseen data in the holdout set.

Cross-validation folds
Enter a value from 2 - 50.
5 folds

Holdout percentage
Set the subset of data that is unavailable during training and validation. Enter a value from 0-99%.
20 percent

CV folds Holdout

0% 50% 100%

Experiment summary

Name
HOSPITAL_READMISSION_DATA - 2026-07-26 11:43:24

Description
Add experiment description.

Dataset

Name HOSPITAL_READMISSION_DATA

Rows 10,000

Features 51

Modeling

Learning type Supervised

Target feature readmitted

Target type Binary classification

Positive class 1

Modeling mode Quick Autopilot

Optimization metric LogLoss

Training feature list Informative Features

Partitioning

Method Stratified sampling

Validation type Cross-validation

Details 5 cross-validation folds
20% holdout

☐ Time series modeling
Not available

Additional settings
No settings applied

DataRobot | Workbench | Registry | Console | Applications | Admin settings

trial-d13c24a8-ab71-4616-b07e-2b... Days remaining: 30

Use Case directory / Use_Case_1 / HOSPITAL_READMISSION_DATA - 2026-07-26 11:43:24

Setup

Data preview

Features

Feature lists

Data insights

Blueprint repository

Model Leaderboard

Experiment insights

Model comparison

Workers

Experiment allocation

CPU

- 2 +

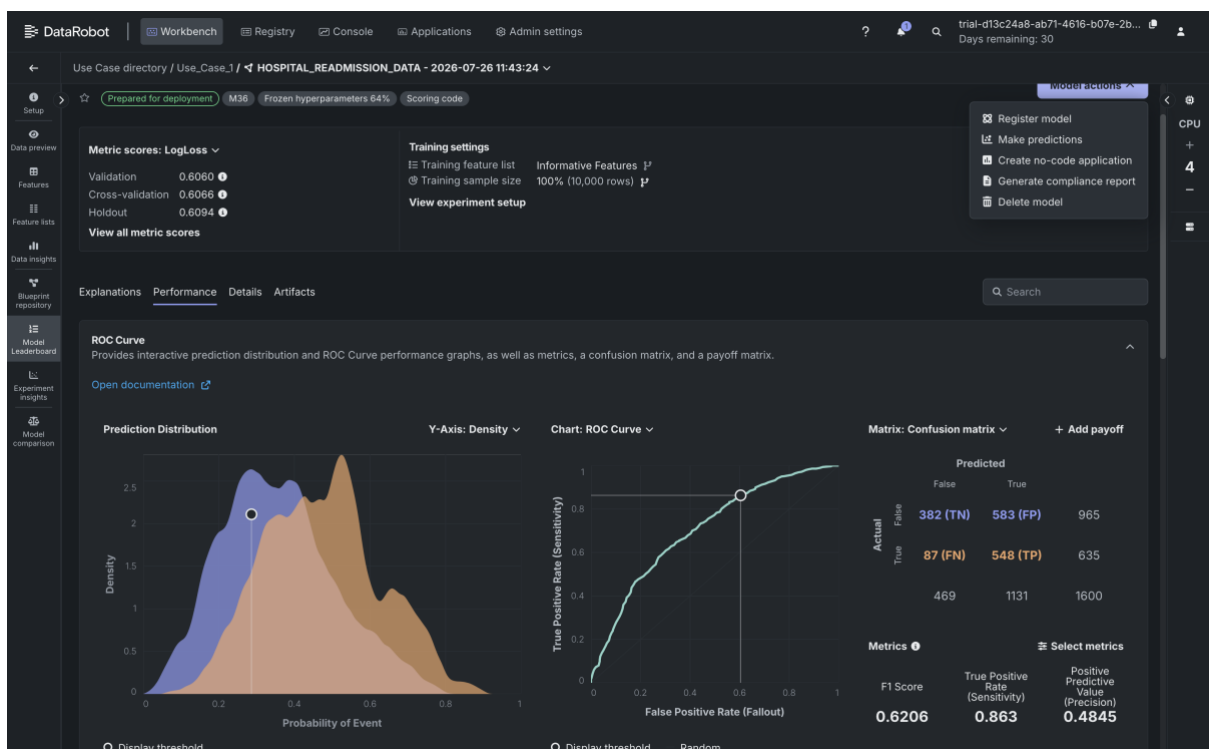
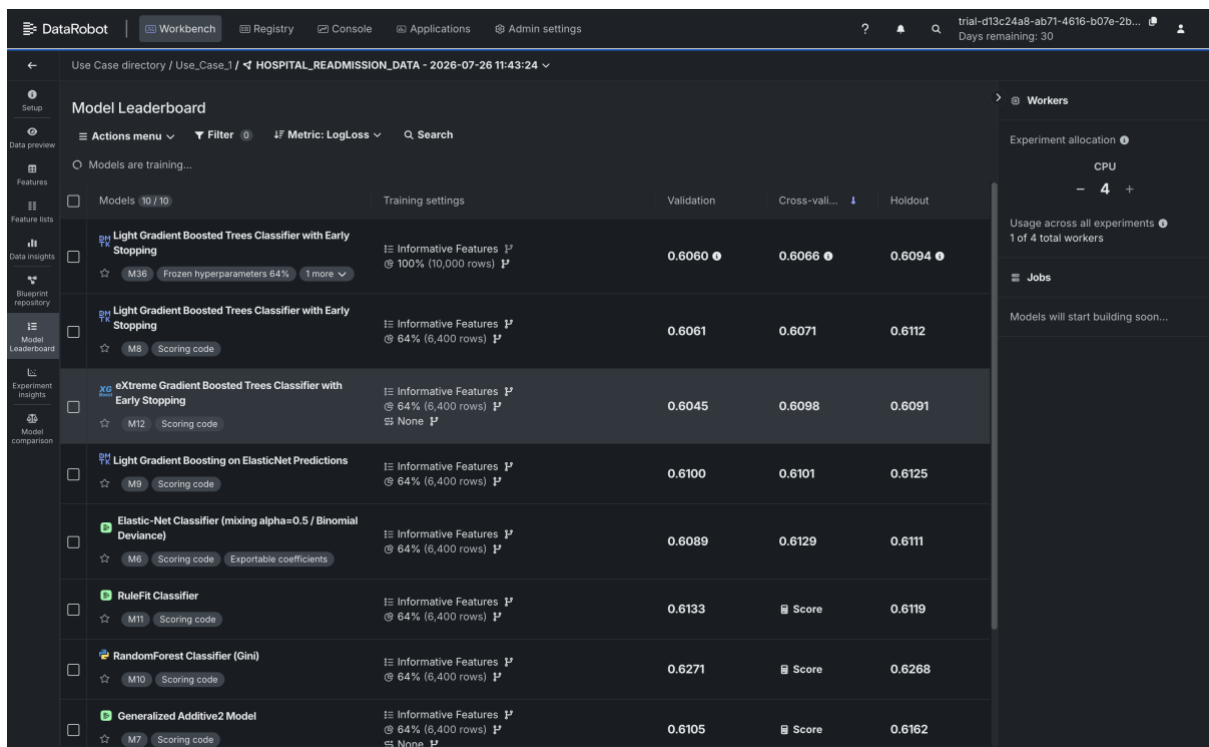
Usage across all experiments

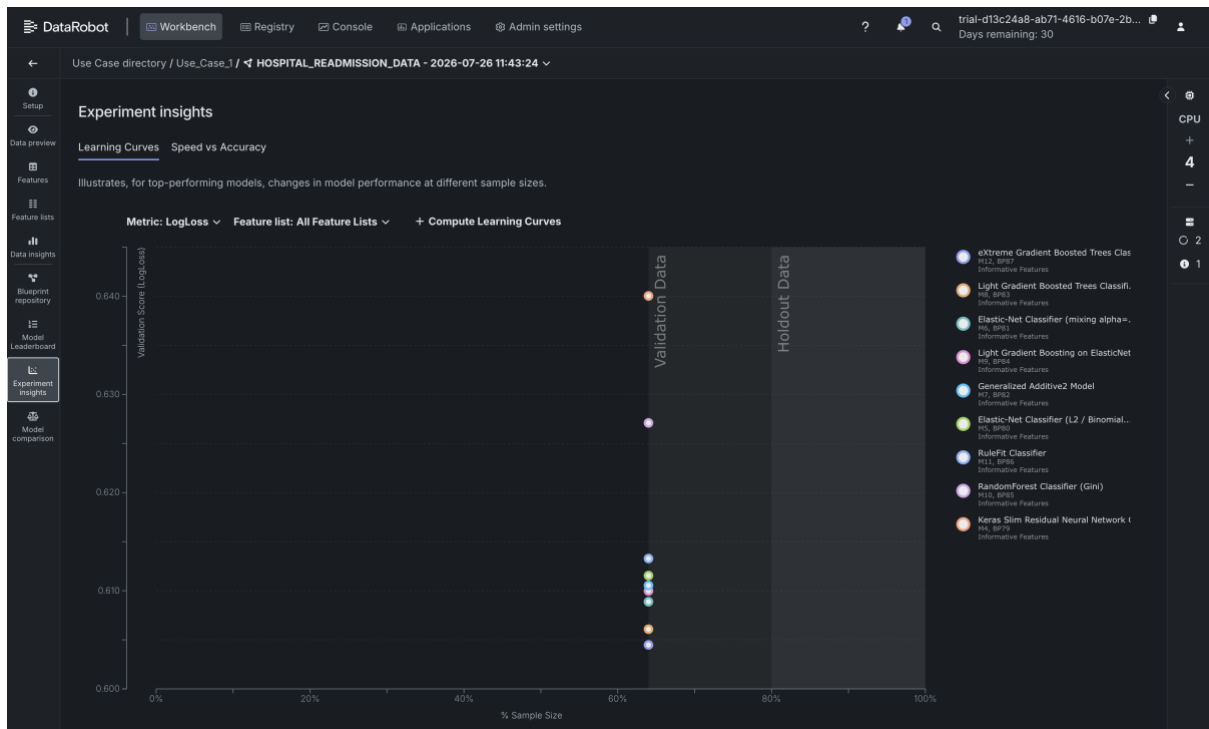
0 of 4 total workers

Jobs

- Setting target feature
- Creating CV and Holdout partitions
- Characterizing target variable
- Loading dataset and preparing data
- Saving target and partitioning information
- Analyzing features
- Generating blueprints

Models will start building soon...





10.6 Summary

These exercises demonstrated the basic workflow of using DataRobot for automated machine learning. We explored how to:

- Create a DataRobot account.
- Create a new use case.
- Add a dataset.
- Build machine learning models.
- Analyze model insights.
- Generate predictions.
- Produce compliance reports using the DataRobot platform.

11 Generative AI Tools for Data Scientists

11.1 Overview

Generative AI provides a wide range of tools that help data scientists automate repetitive tasks, improve productivity, accelerate machine learning workflows, and focus on higher-level problem-solving. These tools can be broadly categorized into **open-source tools** and **proprietary tools**.

11.2 Claude AI by Anthropic

One of the recommended generative AI tools is **Claude AI by Anthropic**. It can be used for a variety of data science and programming tasks, including:

- Understanding and debugging code found on **Stack Overflow** or in repositories shared by peers.
- Creating **sample synthetic data** for experimenting with new data science models.
- Automating repetitive tasks, such as extracting only a few email addresses from a list of **100 emails** in just a few seconds.

11.3 Categories of Generative AI Tools

There are two types of tools that we can use:

- **Open-source tools**
- **Proprietary tools**

11.4 Open-Source Generative AI Tools

11.4.1 Hugging Face Transformers

Hugging Face Transformers offers a wide variety of libraries of pre-trained models for:

- Text generation
- Translation
- Summarization
- Building applications powered by large language models (LLMs)

It supports models such as:

- OpenAI's ChatGPT
- LLaMA
- Mistral

11.4.2 Databricks with GenAI

Databricks provides many AI and data science products, including:

- Vector Search
- AutoML (Automated Machine Learning)

11.4.3 LangChain

LangChain helps build pipelines for text-based workflows, including:

- Summarization
- Retrieval-Augmented Generation (RAG) (a technique that combines information retrieval with large language models to generate more accurate and context-aware responses)

11.4.4 Hugging Face Datasets

Hugging Face Datasets is useful for:

- Data processing
- Data labeling

11.4.5 Label Studio

Label Studio is a tool used for data labeling and annotation.

11.4.6 DAML (Data Annotation for Machine Learning)

DAML stands for **Data Annotation for Machine Learning** and is useful for data processing and labeling tasks.

11.4.7 Albumentations

Albumentations is an open-source library commonly used for:

- Data augmentation (techniques used to increase the diversity of training data by creating modified versions of existing data)

11.4.8 Pandas Profiling

Pandas Profiling is useful for:

- Exploratory Data Analysis (EDA) (the process of analyzing datasets to summarize their main characteristics before building models)

11.4.9 TensorFlow

TensorFlow is essential for:

- Machine learning model development
- Model evaluation
- Model monitoring

11.4.10 PyTorch

PyTorch is widely used for:

- Machine learning modeling
- Model evaluation
- Model monitoring

11.4.11 MLflow

MLflow is essential for:

- Managing machine learning workflows
- Evaluating models
- Monitoring machine learning models

11.5 Proprietary Generative AI Tools

11.5.1 OpenAI GPT Models

OpenAI's GPT models are useful for:

- Generating insights from unstructured data
- Writing code
- Automating various data science tasks

11.5.2 ChatGPT

OpenAI's ChatGPT is described as a powerhouse for generating human-like text and can be fine-tuned for specific tasks.

11.5.3 Microsoft Copilot

Microsoft Copilot is very helpful for:

- Coding
- Productivity tasks
- Integrating with GitHub
- Assisting in writing code
- Assisting in documentation

11.5.4 GitHub Copilot

GitHub Copilot is recommended as a coding assistant because it:

- Helps develop applications
- Is trained on a diverse and large set of programming languages
- Integrates well with IDEs such as **Visual Studio Code (VS Code)**

11.5.5 IBM watsonx Code Assistant

IBM watsonx Code Assistant is recommended because it:

- Supports multiple programming languages
- Integrates well with IDEs such as **VS Code**
- Assists developers in writing code efficiently

11.5.6 Tableau GPT

Tableau GPT integrates generative AI to:

- Automatically create dashboards
- Suggest visualizations based on user queries

11.5.7 Power BI Copilot

Power BI Copilot leverages AI to:

- Build interactive visualizations directly from text prompts

11.5.8 DataRobot

DataRobot provides:

- Automated model building
- AI-powered feature impact analysis
- AI-driven insights

11.6 Cloud-Based AI Platforms

Cloud providers also offer proprietary AI platforms for machine learning development, including:

- Microsoft Azure Machine Learning Studio
- Google's Vertex AI
- AWS SageMaker

11.7 Additional AI Tools

11.7.1 Google's BOT Model

Google's BOT model is widely used for:

- Natural Language Processing (NLP)

11.7.2 HTU.ai

HTU.ai is an AutoML tool that integrates generative AI to:

- Speed up the model training process
- Assist with model fine-tuning

11.8 Key Takeaways

Generative AI tools help data scientists:

- Understand and debug code more efficiently.
- Generate synthetic data for experimentation.
- Automate repetitive tasks.
- Build applications using large language models.
- Perform data processing and data labeling.
- Conduct data augmentation and exploratory data analysis.
- Develop, evaluate, and monitor machine learning models.

- Build dashboards and interactive visualizations.
- Improve coding productivity with AI-powered coding assistants.
- Accelerate model training and fine-tuning using AutoML.
- Focus more on higher-level tasks by reducing manual effort.

12 Generative AI as a Useful Tool for Data Generation and Augmentation

12.1 Learning Objectives

After completing this topic, we will be able to:

- Explain how Generative AI is a useful tool for data generation and augmentation.
- Generate synthetic data using a Generative AI-based tool.
- Augment data using a Generative AI-based tool.

12.2 Introduction

Data augmentation is a powerful technique for improving the performance of machine learning models, especially when the training datasets are small or imbalanced.

It is the process of artificially increasing the size of a training dataset by creating modified data from the existing one.

Since data can be categorized into three categories—structured data, semi-structured data, and unstructured data—there are many ways to generate these categories of data or augment them.

Generative AI provides several tools that can be leveraged for data generation and augmentation across these different data types.

12.3 Generative AI for Different Types of Data

12.3.1 Structured Data

Structured data, often saved in tabular formats, serves as a foundation for many machine learning systems.

CTGAN (Conditional Generative Adversarial Network) and **Synthetic Data Vault (SDV)** are examples of Generative AI techniques that excel in augmenting structured datasets.

These tools generate synthetic data that mimics the statistical traits and correlations seen in the original data.

They are especially useful for:

- Handling data imbalances.
- Filling missing values.
- Addressing privacy problems.

12.3.2 Semi-Structured Data

Semi-structured data, such as text and code, poses unique challenges for data augmentation.

Generative AI tools such as **GPT** and **Copilot** can effectively augment semi-structured datasets by generating realistic text descriptions and code snippets.

This enhances the performance of machine learning models in:

- Natural Language Processing (NLP).
- Code generation tasks.

12.3.3 Unstructured Data

Unstructured data, including images and audio, is often difficult to augment because of its inherent complexity.

Generative AI tools such as **StyleGAN2** and **BigGAN** have demonstrated remarkable capabilities in augmenting image datasets by generating high-resolution, realistic images.

Generative AI can also augment audio datasets by synthesizing new audio samples. An example is **SoundGAN by NVIDIA**.

12.4 Data Generation Using Universal Data

The **Universal Data** website can be used to generate synthetic datasets.

Website:

generate.universaldata.io

12.4.1 Example Prompt

```
Patient dataset for the symptoms of diabetes
```

The process is as follows:

- Type the prompt.
- Skip specifying the fields (optional).
- Generate the data.
- Wait for the prompt to be processed.
- View the generated dataset.
- Copy or download the generated data.
- Selecting **Download** saves the dataset as a CSV file.

Generative AI tools may have different ways of creating datasets. Some may be efficient, while others may require additional work.

Generate

Create data on-the-fly using AI knowledge.

[new](#) Get the gsheets integration →

Title

patient dataset for the symptoms of diabetes

Specify Fields (optional)

Add field

Generate data

12.5 Data Generation Using ChatGPT

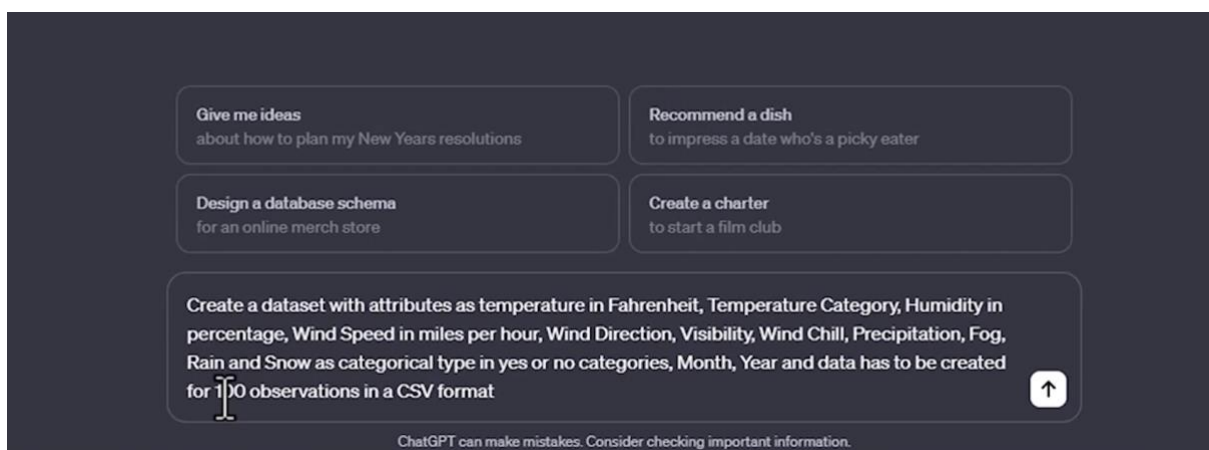
Another approach is to use **ChatGPT**, available free of charge, while specifying the required dataset attributes.

12.5.1 Example Prompt

Create a dataset with attributes as temperature in Fahrenheit, temperature category, humidity in percentage, wind speed in miles per hour, wind direction, visibility, windchill, precipitation, fog, rain, and snow as categorical type in yes or no categories, month, year, and specify the data has to be created for 100 observations in a CSV format.

After submitting the prompt:

- A sample dataset is generated.
- We can modify the values and attributes according to our needs.
- We can copy the generated code and run it in a Jupyter Notebook.



12.6 Data Generation Using BARD

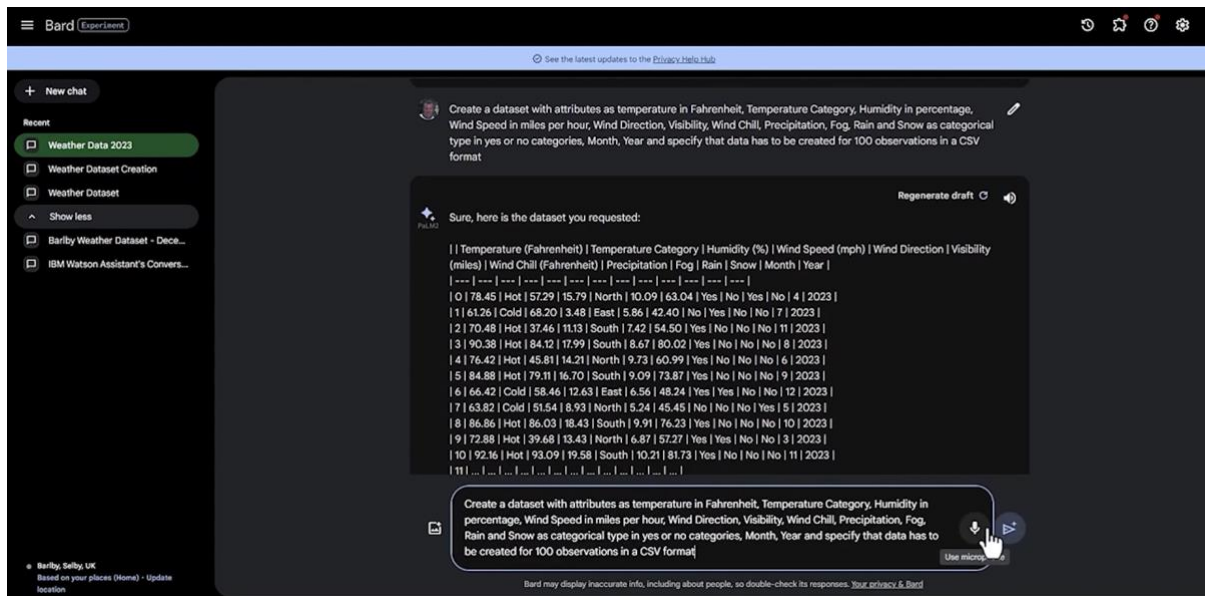
The same prompt can also be used with **BARD**.

Website:

bard.google.com

Steps:

- Open a new session.
- Submit the same prompt.
- Review the generated dataset.
- Review the additional draft versions generated by the tool.
- Copy the generated data for further use.



12.7 Data Generation Using MostlyAI

Another approach is to upload an existing dataset and generate synthetic data.

Website:

synthetic.mostly.ai

12.7.1 Steps

- Open the **Upload Files** tab.
- Upload the **Daily Car Sales** dataset.
- Click **Proceed**.
- Select a training goal.

Available training goals include:

- Accuracy
- Speed
- Turbo

Each option prioritizes one objective over another, causing the training size and the number of training epochs to change accordingly.

Next:

- Click **Create Synthetic Dataset**.
- Wait while the training epochs are executed.
- The job status changes from **Pending** to **In Progress**.
- Select the status link to monitor progress.
- Once completed, review the augmented dataset.

The generated dataset contains the same number of samples as the original dataset.

The synthetic dataset can then be downloaded and used for model training.

The screenshot shows the MOSTLY.AI interface for managing synthetic datasets. The top navigation bar includes 'Home', 'Synthetic datasets', 'Catalogs', and 'Connectors'. A 'Talk to Sales' button is in the top right. The main content area is titled 'Synthetic datasets / Summary' and shows a dataset named 'Daily_Car_Sales' with 167,934 generated rows. A table lists the steps of the generation process, including data fetching, analysis, encoding, and model training. The 'Train AI model' step is currently in progress at 75% completion, while subsequent steps like 'Generate data for Model QA' and 'Export data' are pending.

Model	Model Type	Step	Progress	Duration
Daily_Car_Sales	Flat	Fetch data	100%	1s
Daily_Car_Sales	Flat	Analyze data	100%	19s
Daily_Car_Sales	Flat	Encode data	100%	29s
Daily_Car_Sales	Flat	Train AI model	75% Running epoch 63	5m 9s
Daily_Car_Sales	Flat	Generate data for Model QA	Pending	0s
Daily_Car_Sales	Flat	Analyze data for Model QA	Pending	0s
Daily_Car_Sales	Flat	Generate data for delivery	Pending	0s
Daily_Car_Sales	Flat	Analyze data for Data QA	Pending	0s
Daily_Car_Sales	Flat	Post-process data	Pending	0s
Daily_Car_Sales	Flat	Export data	Pending	0s

12.8 Data Generation Using CTGAN in Google Colaboratory

Another method is to generate synthetic data programmatically.

Website:

collab.research.google.com

12.8.1 Dataset Used

For this demonstration, we use the sample dataset:

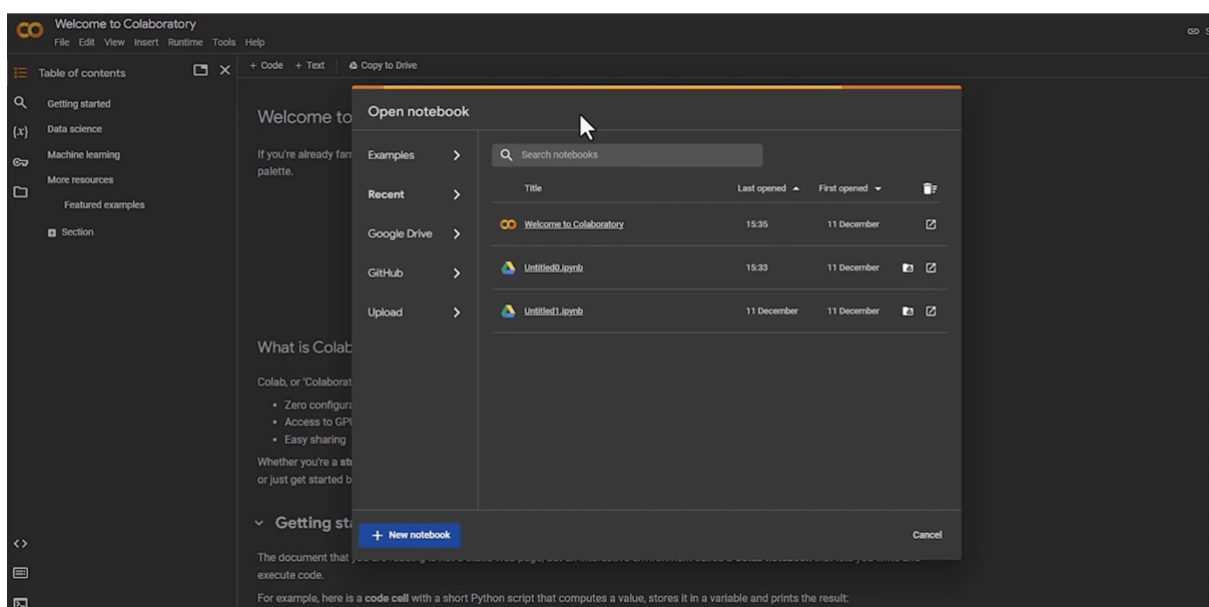
- CaliforniaHousingTrain.csv

The dataset can be accessed from:

- Files
- Sample Data

12.8.2 Steps

- Install the **CTGAN** module.
- Install **Table Evaluator**.
- Import the sample CSV file into the notebook using **Pandas**.
- Import the **CTGAN** library.
- Fit the CTGAN model on the dataset.
- If the dataset contains categorical features, pass them as a list to the **fit()** function.
- Run the training process for **200 epochs**.
- Save the generated samples.



```

Downloading scikit-plot-0.3.7-py3-none-any.whl (33 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas==2.0.*->table_evaluator) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas==2.0.*->table_evaluator) (2023.3.post1)
Collecting tzdata>=2022.1 (from pandas==2.0.*->table_evaluator)
  Downloading tzdata-2023.3-py2.py3-none-any.whl (341 kB)
  341.0/341.0 kB 33.3 MB/s eta 0:00:00
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (1.2.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (4.46.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (23.2)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (9.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib->table_evaluator) (3.1.1)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->table_evaluator) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->table_evaluator) (3.2.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas==2.0.*->table_evaluator) (1.16.0)
Installing collected packages: tzdata, pandas, scikit-plot, dython, table_evaluator
Attempting uninstall: pandas
  Found existing installation: pandas 1.5.3
  Uninstalling pandas-1.5.3:
    Successfully uninstalled pandas-1.5.3
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
  lida 0.0.10 requires fastapi, which is not installed.
  lida 0.0.10 requires kaleido, which is not installed.
  lida 0.0.10 requires python-multipart, which is not installed.
  lida 0.0.10 requires uvicorn, which is not installed.
  google-colab 1.0.0 requires pandas==1.5.3, but you have pandas 2.0.3 which is incompatible.
Successfully installed dython-0.7.3 pandas-2.0.3 scikit-plot-0.3.7 table_evaluator-1.6.1 tzdata-2023.3

[3] import pandas as pd
data = pd.read_csv('/content/sample_data/california_housing_train.csv')

from ctgan import CTGAN
ctgan = CTGAN(verbose=True)
ctgan.fit(data, epochs = 200)

... Gen. (-0.65) | Discrim. (-0.03): 100% | 199/200 [09:10<00:03, 3.48s/it]

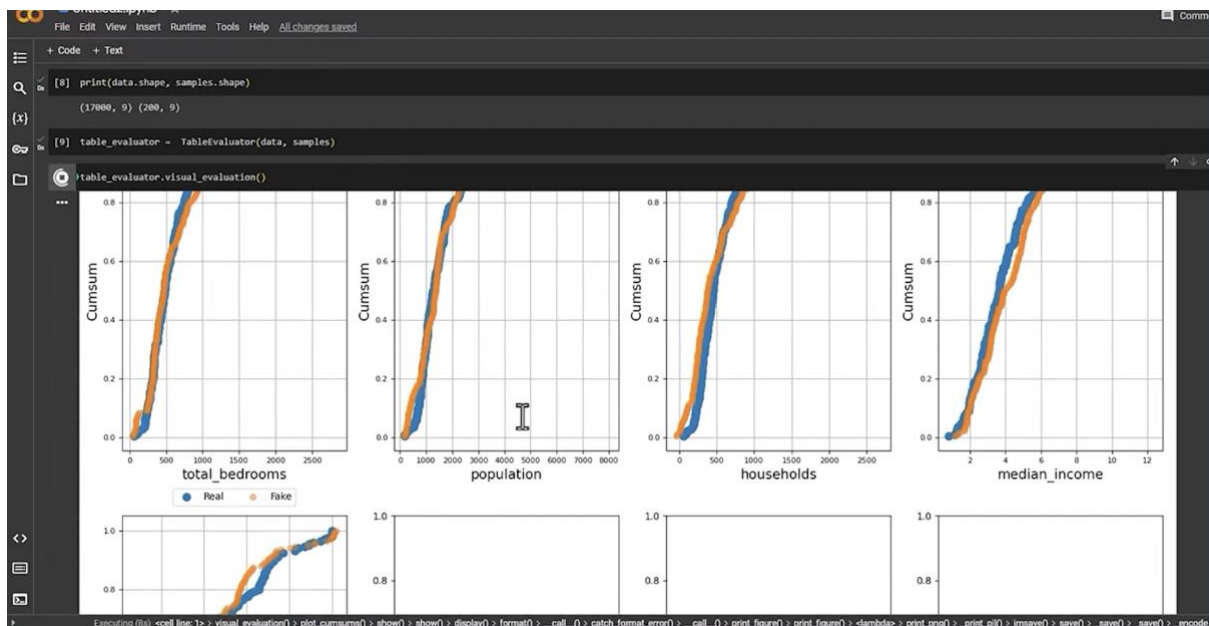
```

12.8.3 Evaluating the Generated Data

To evaluate how closely the generated data matches the original data:

- Import **Table Evaluator**.
- Print the shapes of the original dataset and the generated dataset.
- Compare the fake data with the original data.
- Generate evaluation visualizations.

The evaluation visualizations show that the generated synthetic data is fairly similar to the original real data.



12.9 Key Takeaways

- Data augmentation is the process of artificially increasing the size of a training dataset by creating modified data from existing data.
- **CTGAN** and **Synthetic Data Vault (SDV)** are Generative AI techniques that excel in augmenting structured datasets.
- **GPT** and **Copilot** effectively augment semi-structured datasets by generating realistic text and code.
- **StyleGAN2** and **BigGAN** augment image datasets by generating realistic, high-resolution images.
- **SoundGAN** by **NVIDIA** synthesizes new audio samples for augmenting audio datasets.
- Generative AI tools can generate synthetic datasets that help address data imbalance, missing values, and privacy concerns while improving machine learning model performance.

13 Generative AI for Data Generation and Augmentation

13.1 Learning Objectives

After completing these exercises, we will be able to:

- Understand the concept of synthetic data generation using Generative AI.
- Use tools such as **iLoveCSV** and **Tonic Fabricate** to generate synthetic data.
- Augment an existing CSV dataset while preserving its statistical characteristics.
- Explore how Generative AI enhances datasets for analysis, modeling, and other downstream data science tasks.

13.2 Introduction

One of the major advantages of Generative AI is its ability to generate realistic synthetic data. Synthetic data is created using pretrained generative models that generate new data samples or perform transfer learning on existing datasets.

These synthetic samples preserve the statistical distribution and interpretability of the original data while increasing the dataset size for machine learning and data analysis tasks.

In this topic, we explore how Generative AI can generate synthetic data samples and augment an existing dataset using two different tools.

13.3 Insurance Dataset Overview

The exercises use an **Insurance Dataset** containing insurance records. This dataset is a cleaned version of the original dataset, where errors, missing values, and inconsistencies have already been removed to ensure data quality.

14 Exercise 1: Data Augmentation Using iLoveCSV

The **iLoveCSV** tool provides a graphical interface for generating synthetic data while preserving the statistical properties of the original dataset.

14.1 Procedure

1. Open the **iLoveCSV** website.
2. Create an account or sign in.
3. Upload the **Insurance Dataset**.
4. Open the **Data Augmentation** tool under the **AI and Machine Learning** section.

14.2 Data Augmentation

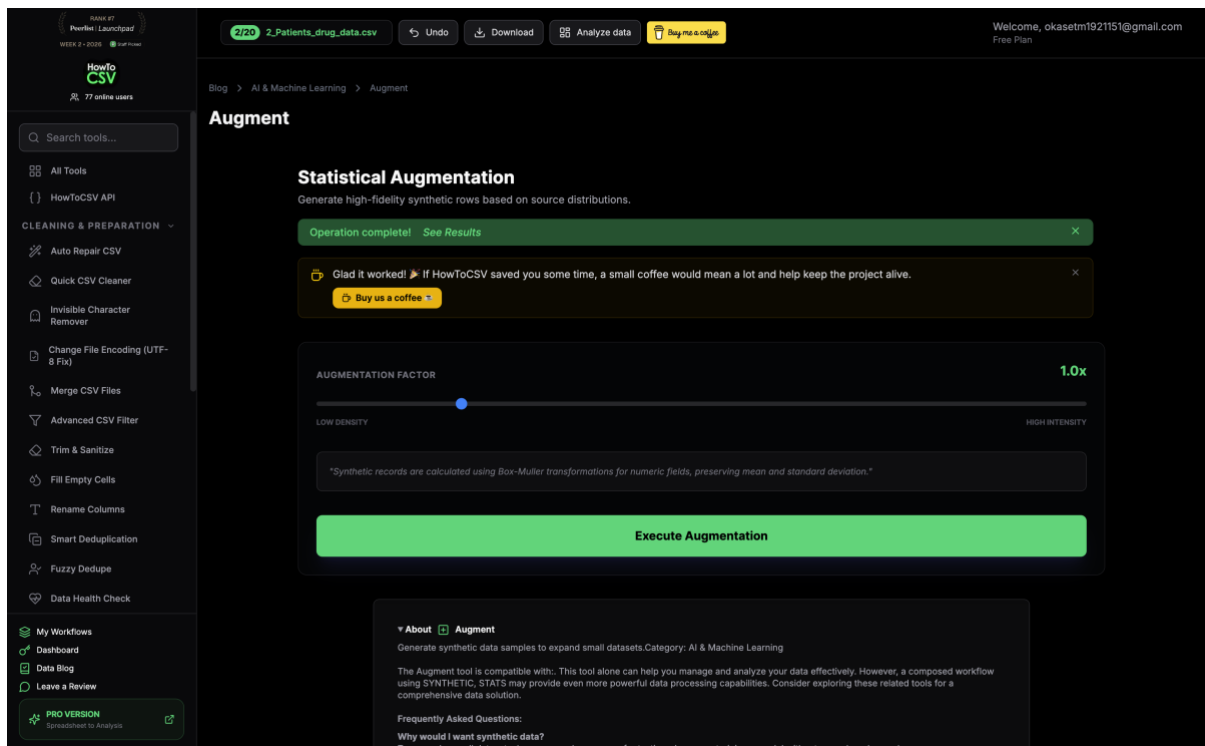
Statistical augmentation generates synthetic rows based on the existing dataset while maintaining its statistical characteristics.

- Set the **Augmentation Factor** according to the required dataset size (for example, **1×**).

- Execute the augmentation process.
- Download the generated synthetic dataset.

Note:

Using a **1× augmentation factor** doubles the dataset size. For example, the resulting dataset contains **2,676 rows**, providing additional synthetic samples suitable for data science and machine learning applications.



15 Exercise 2: Data Augmentation Using Tonic Fabricate

Tonic Fabricate is a prompt-based Generative AI tool that creates synthetic datasets while preserving schema, feature relationships, and statistical consistency.

15.1 Procedure

1. Open the **Tonic Fabricate** website.
2. Create an account or sign in.
3. Upload the **Dataset**.

15.2 Prompt for Synthetic Data Generation

Generate a synthetic dataset based on the uploaded data file. Preserve the original schema, column names, and data types. Maintain realistic distributions for key fields, for example like age, premium, policy type, claim amount, and tenure etc. Ensure logical relationships between variables (for example, higher age may correlate with higher premiums, and claim amounts should align with policy coverage). Avoid duplicates and unrealistic values. Generate clean, high-quality data suitable for analysis, ensuring synthetic data is statistically consistent with the source dataset and

achieving approximately 80% similarity/accuracy. Create a dataset that is 1× the size of the original while introducing meaningful variations.

Note:

Synthetic data generation may take some time depending on the dataset size and complexity.

15.3 Download the Synthetic Dataset

Once the generation process is complete, download the generated CSV dataset for further analysis or machine learning tasks.

The screenshot displays the Tonic Fabricate web application. On the left is a dark sidebar with navigation options: 'My Project', 'New database', 'Connect external database', 'PLANS', 'PROFILES', 'WORKFLOWS', and 'API'. The main area shows a 'Welcome to Fabricate!' message with a list of capabilities and a 'Take a tour of Fabricate' button. On the right, a chat window titled 'New Chat' contains a detailed report generated by an AI agent. The report describes the source data (200 rows, columns: Age, Sex, BP, Cholesterol, Na_to_K, Drug) and the process of generating 200 synthetic rows using bootstrap sampling and logical rules. It includes a summary of the synthetic dataset, the approach used for generation, and a comparison table of drug distributions between the original and synthetic datasets.

synthetic_patients_drug_data.csv
text/csv · 6.72 KB

Download

Done. Here's the summary:

synthetic_patients_drug_data.csv — 200 rows, same schema (Age, Sex, BP, Cholesterol, Na_to_K, Drug), no duplicates (verified against both the synthetic set and the original 200 rows).

Approach:

- Categoricals (Sex, BP, Cholesterol)** — sampled jointly from the empirical distribution of the source data, preserving their real correlations.
- Age & Na_to_K** — bootstrap-sampled from the original values with small random jitter (± 3 yrs / $\pm 5\%$), clipped to the observed ranges (Age 15–74, Na_to_K 6.27–38.25) to avoid unrealistic outliers.
- Drug** — derived using the same deterministic clinical logic present in the source data (Na_to_K > 14.829 → drugY; else if BP=HIGH → drugA/drugB split by Age ≤50/>50; else if BP=NORMAL → drugX; else if BP=LOW → drugC if Cholesterol=HIGH else drugX), so the logical relationships between variables are fully preserved.

Drug distribution comparison (original vs. synthetic):

Drug	Original	Synthetic
drugY	91	85
drugX	54	56
drugA	23	28
drugC	16	18
drugB	16	13

15.4 Analyze the Generated Report

Generative AI can also generate a detailed report describing how the synthetic dataset was created.

15.4.1 Prompt for Report Generation

Provide a detailed report on the model used for generating the synthetic data. Include information about the modeling approach, data distribution handling, and feature relationships learned. Also include performance metrics such as accuracy, AUC (Area Under the Curve), and any validation metrics used to assess the quality and similarity of the synthetic data relative to the original dataset. Present the report in a clear and structured format suitable for analysis.

15.5 Summary

These exercises demonstrated how Generative AI can generate and augment synthetic datasets while preserving the statistical properties of the original data.

We explored two different approaches:

- [iLoveCSV](#) for statistical data augmentation through a graphical interface.
- [Tonic Fabricate](#) for prompt-based synthetic data generation.

Together, these exercises illustrate how synthetic data can:

- Improve data availability.
- Support machine learning workflows.
- Enhance downstream data analysis.
- Maintain realistic feature distributions and relationships.

16 Generative AI for Data Preparation and Data Querying

16.1 Learning Objectives

After studying this topic, we will be able to:

- Summarize the challenges in data preparation and querying.
- List the Generative AI models that can help tackle these challenges.

16.2 Introduction

Generative AI models are instrumental in tackling data preparation and querying challenges. They excel in:

- Imputing (filling in) missing values.
- Detecting outliers.
- Reducing noise in datasets.
- Handling data translation tasks.
- Enabling natural language interactions in querying systems.
- Recommending queries.
- Optimizing query performance.

These capabilities enhance overall efficiency, improve accessibility, and help extract valuable insights from complex data.

16.3 Generative AI for Data Preparation

16.3.1 Handling Missing Values

Missing values are a pervasive (widely occurring) dataset issue and can pose significant challenges to data analysis and modeling.

Traditional imputation methods, such as:

- Mean imputation
- Median imputation

often lead to inaccurate results because they fail to capture the underlying data distribution and relationships between variables.

Generative AI models, particularly **Variational Autoencoders (VAEs)**, offer a promising solution by learning the intricate (complex) patterns within the data and generating plausible (realistic) values that align with the observed data.

16.3.2 Detecting Outliers

Outliers are data points that significantly deviate from the expected pattern. They can distort statistical analysis and lead to erroneous (incorrect) conclusions.

Generative AI models, notably **Generative Adversarial Networks (GANs)**, excel at outlier detection by learning the boundaries of the standard data distributions.

GANs consist of two competing neural networks:

- **Generator** – Strives to produce data that resembles the real data.
- **Discriminator** – Attempts to distinguish between real and generated data.

This adversarial process enables GANs to identify outliers.

16.3.3 Noise Reduction

Noise refers to unwanted random fluctuations that obscure the underlying patterns in a dataset. It can hinder data analysis and impede the extraction of meaningful insights.

Generative AI models, particularly **Autoencoders**, demonstrate remarkable capabilities in noise reduction.

Autoencoders learn the **latent code** (a compressed representation that captures the essential information of the data). This compressed representation encapsulates the core information while discarding the noise.

As a result, Autoencoders capture the essential features while removing unnecessary noise.

16.3.4 Data Translation

Data translation is the process of converting data from one format to another. It plays a crucial role in data exchange and communication, and inaccurate conversion can lead to inaccurate predictions.

Generative AI models, especially **Neural Machine Translation (NMT)**, have emerged as powerful tools for data translation.

NMT utilizes **Recurrent Neural Networks (RNNs)** to learn the complex relationships between languages.

RNNs are proficient at accurately translating text from one language to another.

The capabilities of NMT extend beyond language translation and include:

- Text-to-speech translation
- Image-to-text translation

16.4 Generative AI for Data Querying

16.4.1 Natural Language Querying

Natural language querying has revolutionized data exploration by enabling users to interact with data using natural language rather than complex SQL statements.

Generative AI models, particularly **Large Language Models (LLMs)**, act as intelligent intermediaries between users and data.

LLMs can:

- Interpret the nuances of natural language, including context, intent, and relationships between words.
- Translate natural language accurately into equivalent executable SQL statements.

16.4.2 Query Recommendation

Query recommendations suggest relevant queries to users based on their past questions and search context.

This enhances data exploration by guiding users toward insightful queries.

Generative AI models such as **Recurrent Neural Networks (RNNs)** excel at query recommendation.

RNNs model the sequential nature of users' queries by capturing the temporal relationship between questions, enabling them to predict the next logical query based on the user's current query and search history.

16.4.3 Query Optimization

Query optimization is the process of improving the performance of database queries to ensure efficient data retrieval.

Generative AI models, particularly **Graph Neural Networks (GNNs)**, demonstrate remarkable capabilities in query optimization.

GNNs represent data as a graph where:

- Data entities are represented as **nodes**.
- Relationships between entities are represented as **edges**.

This graph representation allows GNNs to understand the intricate connections between data entities, enabling them to identify the most efficient query execution plans that minimize query execution time.

16.5 Summary

In this topic, we learned that Generative AI models are instrumental in tackling several data preparation and querying challenges.

Key takeaways include:

- **Variational Autoencoders (VAEs)** are highly effective for imputing missing values.
- **Generative Adversarial Networks (GANs)** excel at outlier detection by learning the boundaries of standard data distributions.
- **Autoencoders** demonstrate remarkable capabilities in noise reduction.
- **Neural Machine Translation (NMT)** has emerged as a powerful tool for data translation.
- **Large Language Models (LLMs)** are highly effective at interpreting natural language queries and translating them into equivalent SQL statements.
- **Recurrent Neural Networks (RNNs)** excel at query recommendation by modeling the sequential nature of users' queries.
- **Graph Neural Networks (GNNs)** demonstrate remarkable capabilities in query optimization.

17 Generative AI for Streamlining the Data Preparation Process

17.1 Learning Objectives

After studying this topic, we will be able to:

- Explain how Generative AI streamlines the data preparation process.
- Describe how Generative AI assists in data cleaning, transformation, normalization, and augmentation.
- Understand the role of Generative AI in generating and optimizing complex data queries.
- Identify the potential pitfalls and limitations of using Generative AI during data preparation.

17.2 How Generative AI Streamlines Data Preparation

One game-changing technology that has significantly transformed the data preparation process is **Generative AI**. It automates several time-consuming tasks involved in preparing data for analysis and machine learning.

Generative AI can:

- Automate data cleaning by identifying and filling missing values.

- Correct errors in datasets.
- Remove duplicate records.
- Save a significant amount of manual effort and time.

17.2.1 ChatCSV for Data Cleaning

One example is the [ChatCSV Generative AI tool](#).

It can:

- Swiftly identify missing values in a dataset.
- Provide recommendations for replacing missing values.

17.3 Data Integration and Table Merging

Data preparation often involves merging multiple data tables to consolidate information and extract meaningful insights.

Generative AI simplifies this process by:

- Offering intuitive interfaces to join tables.
- Allowing users to define key columns.
- Helping users choose the appropriate type of join.

For example, [Tamato.ai](#) can assist with this process.

17.4 Natural Language to SQL

Generative AI helps automate the retrieval of data from databases using natural language.

Instead of writing complex SQL queries manually:

- We can provide a natural language prompt.
- A Large Language Model (LLM) transforms the prompt into an SQL query.
- The appropriate data is then retrieved automatically.

17.4.1 Knowledge-Augmented Foundation Models

If a foundation model is augmented with an extensive knowledge base, such as internal documentation, the model can effectively:

- Locate the required information.
- Retrieve relevant information from large databases.
- Reduce the time required for manual searching.

17.5 Data Cleaning and Data Quality Improvement

Generative AI can greatly streamline the data preparation process by automating tasks such as:

- Data cleaning
- Data transformation
- Data augmentation

For example, AI can:

- Correct formatting inconsistencies.
- Generate synthetic data to fill data gaps.

This significantly reduces manual effort.

Generative AI can also perform data cleaning tasks such as:

- Correcting typos.
- Fixing formatting issues.
- Handling datetime field inconsistencies.
- Identifying outliers.
- Handling missing data.

Handling missing values in numerical columns is relatively straightforward, whereas handling missing values in categorical columns is more challenging. Generative AI can make this process easier.

17.6 Data Normalization and Transformation

Generative AI can automate data normalization by:

- Scaling data.
- Transforming data into a standard format.
- Ensuring consistency across datasets.

It can also assist with data transformations such as:

- Standardizing data.
- Normalizing data.
- Encoding data using one-hot encoding.
- Applying other encoding techniques.

These transformations prepare the data for:

- Data analysis.
- Building machine learning models.

17.7 Potential Pitfalls of Generative AI

Although Generative AI offers many advantages, it also has several limitations and risks.

17.7.1 Lack of Domain Understanding

Generative AI may not understand domain-specific relationships within the data.

As a result, it may:

- Generate inappropriate transformations.
- Produce misleading outputs.

17.7.2 False Sense of Data Quality

AI-generated transformations can create a false sense of data quality by potentially overlooking:

- Edge cases.
- Hidden errors.

17.7.3 Overfitting Noise

Generative AI can overfit the noise present in data, which can negatively affect feature engineering.

17.7.4 Bias in Generated Data

We need to be very careful about the bias that Generative AI can introduce into generated data.

If the AI has been trained on biased data, it can:

- Produce skewed results.
- Perpetuate existing inequalities.

17.7.5 Importance of Human Oversight

Generative AI should **not** be considered a 100% automated system.

Human oversight is necessary at every step.

We should think of Generative AI as a tool that makes experts much more productive, rather than expecting a novice to use it to perform complex data science tasks independently.

Even though Generative AI can automate many tasks, oversight is still required to ensure:

- Data quality.
- Data accuracy.

Errors introduced during the data preparation stage can lead to poor machine learning model performance.

17.8 Generative AI for Complex Data Queries

Generative AI is highly effective in generating and optimizing complex data queries.

It accomplishes this by:

- Understanding patterns in data.
- Learning from previous query structures.

Compared to traditional methods that rely on manual coding and query tuning, AI-powered tools can:

- Suggest optimized SQL queries.
- Automatically construct queries for improved performance.

Since Generative AI models are trained on large amounts of internet data, they gain the ability to:

- Write complex code.
- Generate SQL queries.
- Suggest code completions.
- Handle complex programming tasks.
- Optimize queries.

Generative AI can also:

- Analyze existing data.
- Recommend indexes.
- Calculate execution times.
- Support speech-to-text functionality.
- Support text-to-SQL by using tailored prompts.

Natural language enables users to move from a simple request to retrieving the appropriate data without manually writing complex programs to pull and clean the data.

Additionally, techniques such as:

- Few-shot prompting.
- Retrieval-Augmented Generation (RAG) (enhancing responses by retrieving relevant external information before generating an answer).
- Agentic workflows (AI systems that autonomously plan and perform multi-step tasks to achieve a goal).

can further improve the quality of generated queries.

17.9 Gen SQL

A recent Generative AI tool known as **Gen SQL** has been introduced.

This tool enables database users to:

- Perform complicated statistical analyses.
- Make predictions.
- Detect anomalies.
- Estimate missing values.
- Fix errors.
- Generate synthetic data.

These tasks can be performed with just a few keystrokes.

Gen SQL is especially helpful for traditional database users and individuals who do not have SQL or other query language knowledge, making database querying much simpler and more accessible.

17.10 Key Takeaways

- Generative AI automates data cleaning, transformation, augmentation, and normalization.
- Tools such as **ChatCSV** help identify and replace missing values.
- **Tamato.ai** simplifies table joining and data integration.
- Large Language Models (LLMs) can convert natural language prompts into SQL queries.
- Knowledge-augmented foundation models can efficiently retrieve information from large internal knowledge bases.
- Generative AI supports standardization, one-hot encoding, outlier detection, typo correction, and synthetic data generation.
- Human oversight remains essential because AI may introduce bias, overlook edge cases, or generate inappropriate transformations.
- Generative AI significantly improves SQL generation and query optimization through natural language interfaces.
- Techniques such as **Few-shot Prompting**, **Retrieval-Augmented Generation (RAG)**, and **Agentic Workflows** further enhance query quality.
- **Gen SQL** simplifies statistical analysis and complex database operations for users with little or no SQL expertise.

18 Generative AI for Data Preparation

18.1 Learning Objectives

After completing this topic, we will be able to:

- Replace missing values in data.
- Find outliers in data.
- Merge multiple data tables by using a join.
- Filter data.
- Use an AI assistant to analyze data.
- Create an if-then rule for data preparation.

18.2 Introduction to Data Preparation

Data preparation is a crucial step in the process of data analytics, which involves cleaning, transforming, and organizing **raw data** to make it suitable for further analysis and modeling.

This step ensures that the data is:

- Accurate
- Reliable
- Consistent

Generative AI makes data preparation easier and quicker for data professionals by automating many common data cleaning and transformation tasks.

18.3 Demonstration 1: Data Preparation Using ChatCSV

The first demonstration uses the **ChatCSV** generative AI tool, which is a web-based tool that acts as a personal data analyst assistant. It enables us to attach any CSV file to a chat session and ask questions about the data.

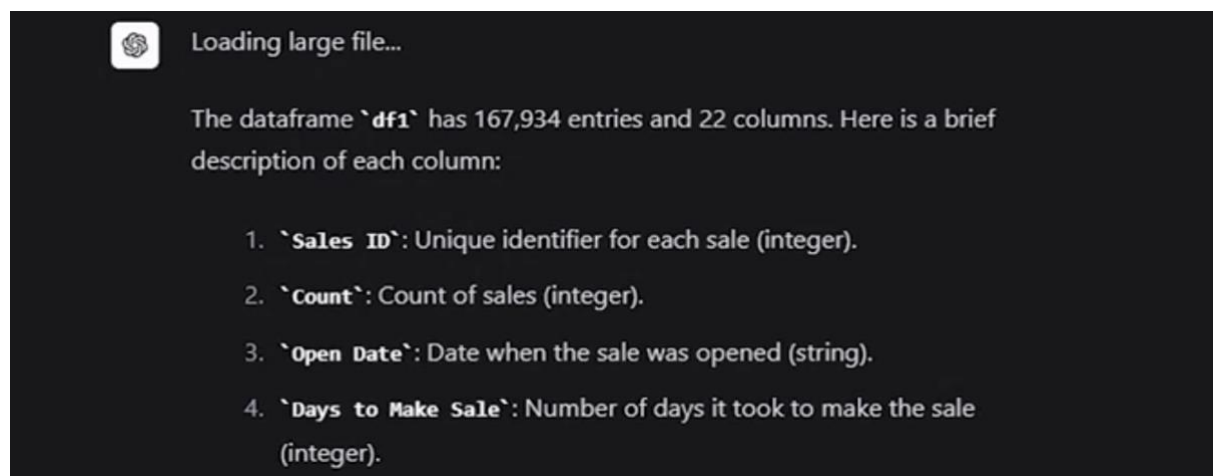
18.3.1 Uploading the Dataset

We begin by initiating a chat and attaching the dataset:

- **Dataset:** `Daily_Car_Sales.csv`

Once the dataset is uploaded, the GPT model displays:

- A description of the dataset.
- Basic information about the dataset.
- Information on columns containing missing values.



18.3.2 Replacing Missing Values

To handle missing values, we enter the following prompt:

Replace missing values of temperature F with the mean of temperature F.

The generative AI tool quickly updates the dataset by replacing the missing values in the **Temperature F** column with the average (mean) temperature.

updated_dataframe

Temperature (F)	Temperature Category	Weather Condition
28	21-30	Clear
30.9	31-40	Scattered Clouds
28	21-30	Clear
30.9	31-40	Scattered Clouds
52.20893643332317		Clear
30.9	31-40	Scattered Clouds
52.20893643332317		Clear
28	21-30	Clear
28.9	21-30	Clear
28.9	21-30	Clear
32	31-40	Scattered Clouds
32	31-40	Scattered Clouds
52.20893643332317		Clear
33.1	31-40	Overcast
30	21-30	Clear

Re-run response Share

18.3.3 Finding Outliers

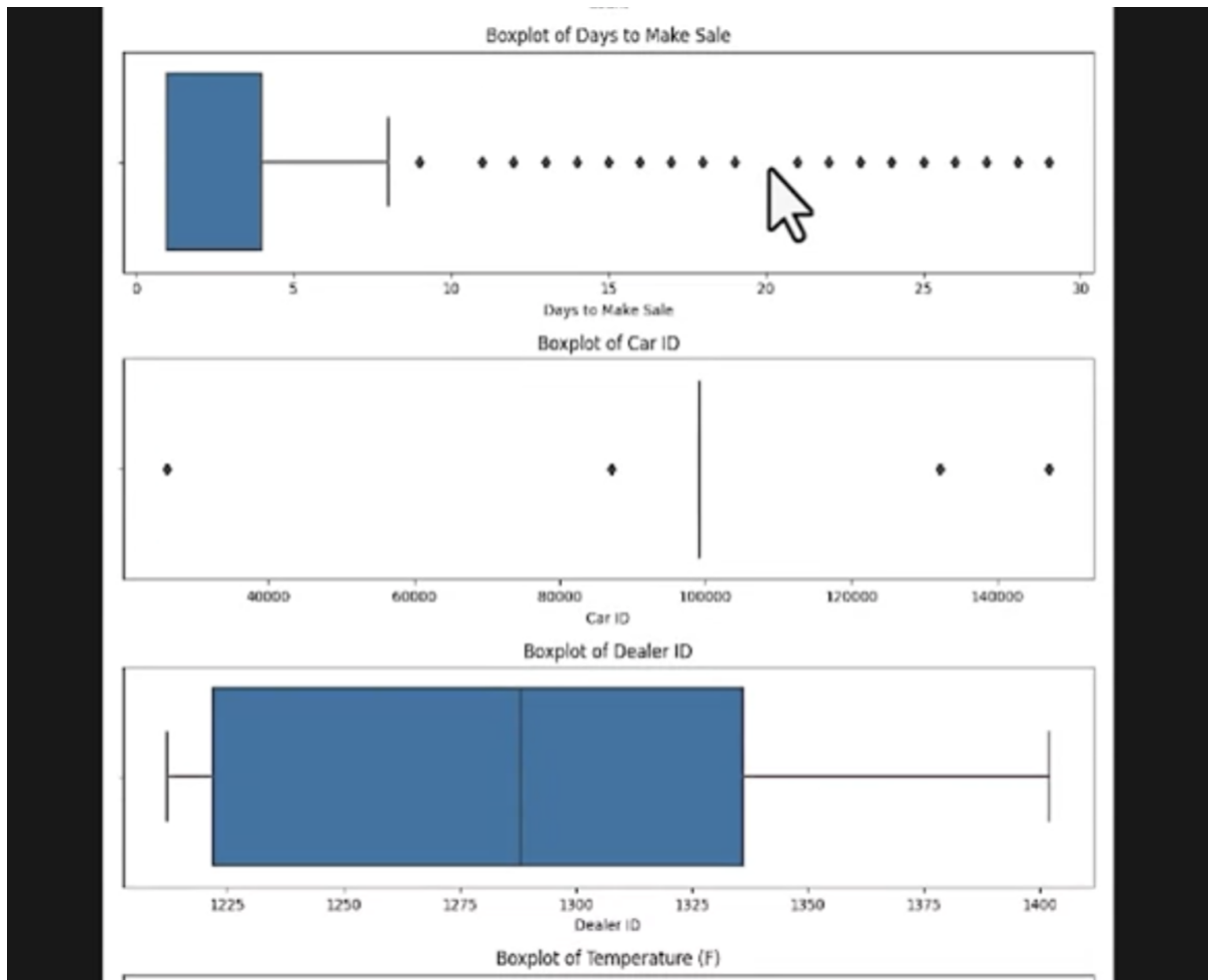
Next, we identify outliers in the dataset.

The GPT model automatically produces **box plots** (a visualization used to identify data distribution and outliers) for each column.

From these box plots, we observe:

- Several columns contain outliers.
- Outliers are represented by black dots.

This demonstrates how Generative AI simplifies common data preparation tasks.



18.4 Demonstration 2: Data Preparation Using Tomat.AI

The second demonstration uses the **Tomat.AI Community Edition**, which allows us to explore and analyze data without limitations.

18.4.1 Uploading the Dataset

To begin:

- Upload the CSV file by dragging and dropping it or browsing for it.
- Click **Add to Catalog** in the upper-right corner.

After uploading, we enter the **Explorer** view, where statistics for every column are displayed in the right-hand pane.

18.4.2 Calculating Average Values

To calculate the average temperature for each weather condition:

- Group the data by **Weather Condition**.
- Select the **Temperature F** column as the value.
- Choose **Average** as the aggregation method.

After closing the dialog box, the updated table displays the average temperature for each weather condition.

18.4.3 Creating a Data Flow

Next, we convert the prepared data into a **Flow** for additional transformations.

18.4.4 Merging Multiple Tables

We upload another dataset:

- **Dataset:** Dealer_ID_Names.csv

After uploading the file:

- Click **Create** to save the changes.
- Select **Join**.
- Connect both table icons to the Join node.

Configure the join as follows:

- **Join Type:** Left Join
- **Join Key (Table 1):** Dealer ID
- **Join Key (Table 2):** Dealer ID

Then create the join.

18.4.5 Filtering Data

We can also filter the data.

For example, we apply a filter where:

- **Weather Condition = Scattered Clouds**

After applying the filter, only rows containing **Scattered Clouds** remain visible.

18.5 Using the AI Assistant

The AI Assistant provides automatic suggestions for improving the dataset, including:

- Handling missing values.
- Other recommendations for different tables in the dataset.

We can also ask the AI Assistant specific questions.

Example:

How should we deal with missing values in the temperature column?

One suggested solution is to replace missing values with a constant value, such as the average.

18.6 Creating an If-Then Rule

To replace missing temperature values with the average value:

- Select **If..Then** from the **New** menu.
- Select the **Is Missing** condition for the **Temperature F** column.
- If the value is missing, assign the average temperature.
- Otherwise, keep the existing temperature value.

Finally:

- Overwrite the **Temperature** column.
- Create the rule.

An **If** node is then connected to the Join and Filter nodes in the data flow.

The screenshot shows the Tomat Flow Designer interface. The top menu bar includes 'tomat', 'File', 'Edit', 'View', 'Window', and 'Help'. Below the menu is a toolbar with various icons for data manipulation. The main workspace displays a flow diagram with two source nodes, 'Daily_Car_Sales' and 'Dealer_ID_Names', connected to a 'Join' node. The output of the 'Join' node is connected to an 'If' node, which is highlighted by a mouse cursor. The 'If' node is then connected to an 'Output' node. Below the flow diagram is a 'Preview' section showing a table of data.

#	Sales ID	Dealer ID	Dealer Name	Sales ID_Join	Count	Open Date	Days to Make Sale	Car ID
	546,467 - 713,261	1,336 - 1,481	2 Uniques	546,467 - 546,594	1	01 Jan - 02 Jan	1 - 4	99,110
1	713,261	1,481	Mr. John	546,467	1	2022-01-01	1	99,1
2	712,998	1,481	Mr. John	546,467	1	2022-01-01	1	99,1
3	712,845	1,481	Mr. John	546,467	1	2022-01-01	1	99,1
4	712,396	1,481	Mr. John	546,467	1	2022-01-01	1	99,1

18.7 Exporting the Prepared Dataset

To generate the processed CSV file:

- Add a **Local File** output.
- Specify the output location.
- Name the file:

prepared data

Once the flow is complete:

- Click the **Run Job** icon.

- Wait for the processing to finish.

After completion, a CSV file containing the prepared data is generated.

18.8 Key Takeaways

In this topic, we learned that Generative AI can simplify several data preparation tasks, including:

- Replacing missing values in data.
- Managing outliers in data.
- Finding average values for each data category.
- Applying filters.
- Merging multiple data tables using joins.
- Deleting columns.
- Creating **if-then** rules for data transformation.
- Generating a CSV file containing processed data.

19 Generative AI for Data Preparation

19.1 Learning Objectives

After completing these exercises, we will be able to:

- Handle missing values in a dataset.
- Correct the data types of dataset attributes.
- Perform standardization and normalization on selected attributes.
- Convert categorical data into numerical indicator variables.

19.2 Introduction

Data preparation is one of the first steps in a data science workflow. It involves cleaning and transforming data before analysis, including handling missing values, correcting data types, standardizing and normalizing numerical attributes, and converting categorical variables into numerical representations.

In these exercises, we use Generative AI to generate Python code that performs these common data preparation tasks on a real-world dataset.

19.3 Exercise 1: Import the Dataset

19.3.1 Prompt

Generate plain basic Python code without using definition or exceptions or if else statements. Response should be concise.

Write a Python code that can perform the following task:

- Read the CSV file, located on a given file path, into a Pandas DataFrame, assuming that the first rows of the file are the headers for the data.

19.4 Exercise 2: Handle Missing Data

19.4.1 Identify Missing Values

19.4.1.1 Prompt

Write a Python code that identifies the columns with missing values in a Pandas DataFrame and gives missing value counts per column.

The dataset contains missing values in the following attributes:

- **Screen_Size_cm** (categorical attribute)
- **Weight_kg** (continuous attribute)

19.4.1.2 Guidelines

- Replace missing values in categorical attributes with the most frequent value.
- Replace missing values in continuous attributes with the mean value.
- If the target column contains missing values, the corresponding row may need to be removed.

19.4.2 Replace Missing Values

19.4.2.1 Prompt

Write Python code to replace the missing values in a Pandas DataFrame, according to the following guidelines:

1. For the categorical attribute "**Screen_Size_cm**", replace the missing values with the most frequent value in the column.
2. For the continuous attribute "**Weight_kg**", replace the missing values with the mean value of the entries in the column.

19.5 Exercise 3: Modify Data Types

19.5.1 Prompt

Write a Python code snippet to change the data type of the attributes "**Screen_Size_cm**" and "**Weight_kg**" of a DataFrame to **float**.

19.6 Exercise 4: Standardization and Normalization

The dataset contains attributes that should be converted into standard measurement units.

19.6.1 Convert Measurement Units

Convert:

- **Screen_Size_cm** to **Screen_Size_inch**
- **Weight_kg** to **Weight_pounds**

19.6.1.1 Prompt

Write a Python code to modify the contents under the following attributes of the DataFrame as required:

1. Data under **"Screen_Size_cm"** is assumed to be in centimeters. Convert this data into inches. Modify the name of the attribute to **"Screen_Size_inch"**.
2. Data under **"Weight_kg"** is assumed to be in kilograms. Convert this data into pounds. Modify the name of the attribute to **"Weight_pounds"**.

19.6.2 Normalize CPU Frequency

The **CPU_frequency** attribute should be normalized with respect to its maximum value, and the original attribute should be updated instead of creating a new one.

19.6.2.1 Prompt

Write Python code to normalize the content under the attribute **"CPU_frequency"** in a DataFrame **df** with respect to its maximum value. Make changes to the original data, and do not create a new attribute.

19.7 Exercise 5: Convert Categorical Data to Numerical Data

For predictive modeling, categorical variables should be converted into indicator (one-hot encoded) variables. After creating the indicator variables, the original categorical attribute should be removed.

19.7.1 Prompt

Write Python code to perform the following tasks:

1. Convert the DataFrame **df** attribute **"Screen"** into indicator variables, saved as **df1**, with the naming convention **"Screen_"**.
2. Append **df1** to the original DataFrame **df**.
3. Drop the original **"Screen"** attribute from the DataFrame **df**.

19.8 Practice Exercises

- Create a prompt to generate Python code that converts the values under **Price** from **USD** to **Euros**.
- Modify the normalization prompt to perform **Min-Max Normalization** on the **CPU_frequency** attribute.

19.9 Summary

Through these exercises, we learned how to use Generative AI to generate Python code for common data preparation tasks, including:

- Handling missing values.
- Modifying attribute data types.
- Performing standardization and normalization.
- Converting categorical attributes into numerical indicator variables.

20 Generative AI for Querying Databases

20.1 Learning Objectives

After completing this topic, we will be able to:

- Demonstrate how we can quickly and easily extract insights from massive datasets using Generative AI.
- Use Generative AI tools to understand natural language queries.
- Convert natural language queries into SQL commands.

20.2 Introduction

Querying a database involves retrieving or manipulating data stored in a database.

A **database** refers to an organized collection of data, and **queries** refer to the primary means of interacting with this collection and extracting information from the database.

Structured Query Language (SQL) is the most common and standardized query language for interacting with relational databases. SQL queries consist of commands that specify:

- The data to be retrieved.
- The conditions to be applied.
- The sequence in which to display the data.

Generative AI has changed the way we interact with databases. By enabling users to query databases using natural language, Generative AI makes it easier to access and understand data. This capability can profoundly impact a wide range of industries, including:

- Finance
- Healthcare
- Education

20.3 Extracting Insights from Massive Datasets Using Generative AI

Traditionally, data professionals would spend hours crafting SQL queries to navigate through complex database structures.

This demonstration uses **SQL through AI**, a Generative AI model designed to understand natural language queries and convert them into SQL commands.

20.3.1 Setting Up the Database

To begin:

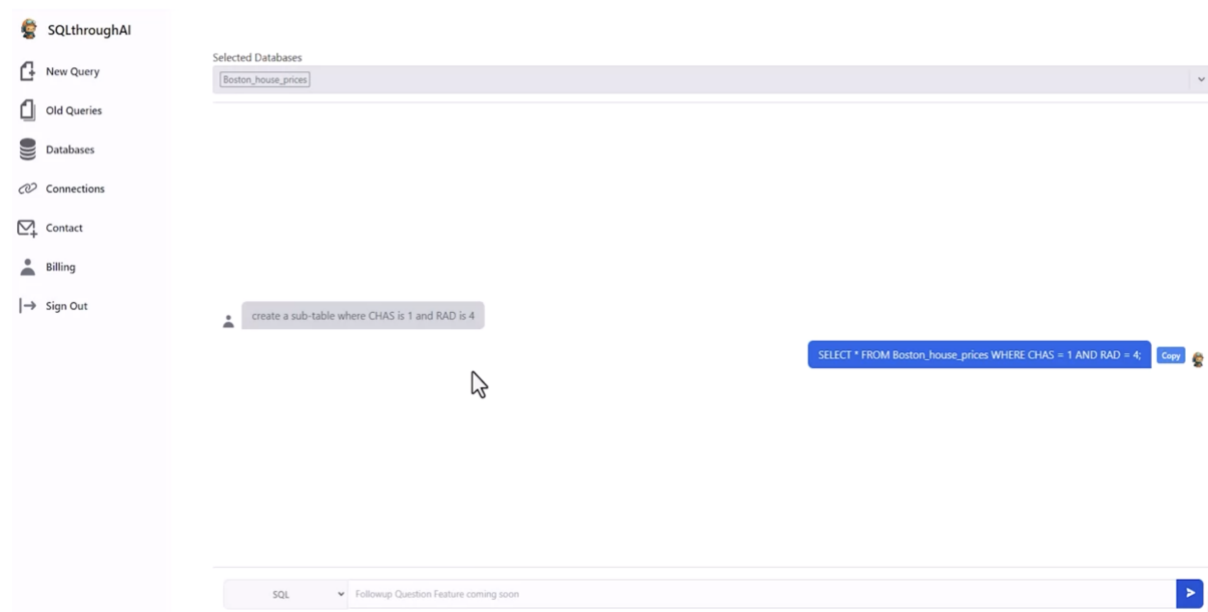
- Log in to the platform.
- Upload the **Boston Housing Price** dataset that we want to query.
- Give the dataset a name.
- Save the dataset.

Once the dataset is saved, we can begin querying the data.

The tool allows us to generate queries for multiple database management systems and languages, including:

- SQL
- MongoDB
- Other supported database systems

For this demonstration, SQL is selected.



20.4 Generating SQL Queries Using Natural Language

20.4.1 Viewing Column Names

Select the dataset and enter the following prompt:

What are the column names?

Within seconds, the SQL query is generated, which can then be copied and used in an application.

20.4.2 Counting the Number of Rows

Enter a new prompt requesting the count of rows in the dataset.

The generated SQL query is:

```
SELECT COUNT(*)
FROM Boston_house_prices;
```

20.4.3 Finding the Average Age

To determine the average age in the database, the generated SQL query is:

```
SELECT AVG(age)
FROM Boston_house_prices;
```

20.4.4 Finding Rows Based on a Tax Range

To find all rows where **tax is between 210 and 250**, the generated SQL query is:

```
SELECT *
FROM Boston_house_prices
WHERE tax >= 210
AND tax <= 250;
```

20.4.5 Replacing Zero Values in the ZN Column

To replace all zero values in the **ZN** column with a constant value of **5**, the generated SQL query is:

```
UPDATE Boston_house_prices
SET ZN = 5
WHERE ZN = 0;
```

20.4.6 Sorting the Table by the MEDV Column

To sort the table in ascending order based on the **MEDV** column, the generated SQL query is:

```
SELECT *
FROM Boston_house_prices
ORDER BY MEDV ASC;
```

20.4.7 Inserting New Rows

The tool can also generate SQL statements for inserting new rows.

The generated response may not always be exactly as expected. We need to replace the **NULL** values with the actual values required for each field before executing the query.

20.4.8 Generating a Condition-Based Query

To find rows where:

- **RAD = 5**
- **AGE is between 50 and 55**

The generated SQL query is:

```
SELECT *
FROM Boston_house_prices
WHERE RAD = 5
AND AGE BETWEEN 50 AND 55;
```

20.4.9 Creating a Sub-Table

To create a sub-table where:

- **CHAS = 1**
- **RAD = 4**

The generated SQL statement is:

```
SELECT *
FROM Boston_house_prices
WHERE CHAS = 1
AND RAD = 4;
```

20.5 Importance of Querying Databases

Querying databases is an essential part of working with data.

By understanding how to write SQL queries, we can effectively retrieve and manipulate data to:

- Gain valuable insights.
- Analyze datasets efficiently.
- Make informed decisions.

Generative AI plays a vital role in query generation through a simple natural language processing (NLP) interface, saving both time and effort.

20.6 Key Takeaways

In this topic, we learned that Generative AI can generate SQL queries from natural language prompts, allowing us to:

- Query specific data from a dataset, such as column names, row counts, and average age.
- Find specific rows using filtering conditions.
- Replace values in a column.
- Sort a table in ascending order.
- Insert new rows.
- Generate condition-based queries.
- Create sub-tables using SQL statements.

21 Generative AI for Querying Databases

21.1 Learning Objectives

After completing these exercises, we will be able to:

- Use Generative AI to create efficient SQL queries for a dataset.

21.2 Introduction

Processed data stored in database tables can be accessed using SQL queries. Writing efficient queries is an essential skill for data professionals. In these exercises, we use Generative AI to generate optimized SQL queries by providing sufficient context about the dataset and its attributes.

21.3 Exercise 1: Provide Context to the Generative AI Model

Before generating SQL queries, we provide the model with a detailed description of the dataset and its attributes so it can generate accurate and optimized queries.

21.3.1 Prompt Instructions

We have a **Heart Disease Prediction** dataset with a single table that contains the following attributes.

Attribute	Description
age	Age in years
gender	Gender (1 = male; 0 = female)
cp	Chest pain type
trestbps	Resting blood pressure (mm Hg)
chol	Serum cholesterol (mg/dl)
fbs	Fasting blood sugar > 120 mg/dl (1 = true; 0 = false)
restecg	Resting electrocardiographic results
thalach	Maximum heart rate achieved
exang	Exercise-induced angina (1 = yes; 0 = no)
oldpeak	ST depression induced by exercise relative to rest
slope	Slope of the peak exercise ST segment
ca	Number of major vessels (0–3) colored by fluoroscopy
thal	Thallium stress test result
num	Target attribute (Heart disease status)

21.3.2 Attribute Value Definitions

21.3.2.1 Chest Pain Type (cp)

Value	Description
1	Typical angina

2	Atypical angina
3	Non-anginal pain
4	Asymptomatic

21.3.2.2 Resting Electrocardiographic Results (*restecg*)

Value	Description
0	Normal
1	ST-T wave abnormality
2	Left ventricular hypertrophy

21.3.2.3 Slope of Peak Exercise ST Segment (*slope*)

Value	Description
1	Upsloping
2	Flat
3	Downsloping

21.3.2.4 Thallium Stress Test (*thal*)

Value	Description
3	Normal
6	Fixed defect
7	Reversible defect

21.3.2.5 Target Attribute (*num*)

Value	Description
0	<50% diameter narrowing
1	>50% diameter narrowing

21.4 Exercise 2: Generate SQL Queries Using Prompts

21.4.1 Age Distribution

Prompt

Write an SQL query to find the minimum, maximum, and average age of patients in the dataset.

```
SELECT
    MIN(age) AS min_age,
    MAX(age) AS max_age,
    AVG(age) AS avg_age
FROM heart_disease_prediction_dataset;
```

21.4.2 Gender Analysis

Prompt

Write an SQL query to count the number of male and female patients in the dataset.

```
SELECT
    SUM(CASE WHEN gender = 1 THEN 1 ELSE 0 END) AS male_count,
    SUM(CASE WHEN gender = 0 THEN 1 ELSE 0 END) AS female_count
```



```
FROM heart_disease_prediction_dataset;
```

21.4.3 Chest Pain Type Frequency

Prompt

Write an SQL query to determine the frequency of each type of chest pain (typical angina, atypical angina, non-anginal pain, and asymptomatic) among patients.

```
SELECT
    CASE cp
        WHEN 1 THEN 'typical_angina'
        WHEN 2 THEN 'atypical_angina'
        WHEN 3 THEN 'non_anginal_pain'
        WHEN 4 THEN 'asymptomatic'
        ELSE 'unknown'
    END AS chest_pain_type,
    COUNT(*) AS frequency
FROM your_table_name
GROUP BY
    CASE cp
        WHEN 1 THEN 'typical_angina'
        WHEN 2 THEN 'atypical_angina'
        WHEN 3 THEN 'non_anginal_pain'
        WHEN 4 THEN 'asymptomatic'
        ELSE 'unknown'
    END
ORDER BY frequency DESC;
```

21.4.4 Age Group Analysis and Target Variable

Prompt

Write an SQL query to investigate the distribution of the target variable (presence or absence of heart disease) within different age groups (e.g., 20–30, 30–40, etc.).

```
SELECT
    CASE
        WHEN age IS NULL THEN 'Unknown'
        WHEN age >= 20 AND age < 30 THEN '20-29'
        WHEN age >= 30 AND age < 40 THEN '30-39'
        WHEN age >= 40 AND age < 50 THEN '40-49'
        WHEN age >= 50 AND age < 60 THEN '50-59'
        WHEN age >= 60 AND age < 70 THEN '60-69'
        WHEN age >= 70 AND age < 80 THEN '70-79'
        ELSE '80+'
    END AS age_group,
    num AS heart_disease_status,
    COUNT(*) AS count
FROM your_table_name
GROUP BY
    CASE
        WHEN age IS NULL THEN 'Unknown'
        WHEN age >= 20 AND age < 30 THEN '20-29'
        WHEN age >= 30 AND age < 40 THEN '30-39'
        WHEN age >= 40 AND age < 50 THEN '40-49'
        WHEN age >= 50 AND age < 60 THEN '50-59'
        WHEN age >= 60 AND age < 70 THEN '60-69'
        WHEN age >= 70 AND age < 80 THEN '70-79'
```

```

        ELSE '80+'
    END,
    num
ORDER BY age_group, heart_disease_status;

```

21.5 Practice Exercises

Practice the following SQL exercises using Generative AI.

1. **Cholesterol Range**
 - Find the range of cholesterol levels among patients (minimum and maximum).
2. **Age Range and Gender Analysis**
 - Determine the age range (youngest and oldest) for male and female patients separately.
3. **Age Group Analysis and Target Variable**
 - Investigate the distribution of the target variable (presence or absence of heart disease) within different age groups (e.g., 20–30, 30–40, etc.).
4. **Maximum Heart Rate by Age Group**
 - Find the maximum heart rate achieved during exercise for different age groups (e.g., 30–40, 40–50, etc.).
5. **Percentage of Patients with High Blood Sugar**
 - Calculate the percentage of patients with fasting blood sugar greater than **120 mg/dl**.
6. **Ratio of Patients with Resting Electrocardiographic Abnormality**
 - Find the ratio of patients with abnormal resting electrocardiographic results to those with normal results.
7. **Number of Patients with Reversible Thalassemia**
 - Count the number of patients with reversible thalassemia detected by thallium stress testing.
8. **Average Age of Patients with Chest Pain**
 - Calculate the average age of patients who experienced chest pain during diagnosis.
9. **Distribution of Patients by Number of Major Vessels**
 - Investigate the distribution of patients based on the number of major vessels colored by fluoroscopy (0–3).

21.6 Summary

Through these exercises, we learned how to use Generative AI to generate efficient SQL queries for retrieving information from a database.

We also learned that providing detailed context about the dataset and its attributes enables the model to generate accurate, optimized, and ready-to-use SQL queries.

22 Module 1 Summary: Data Science and Generative AI

22.1 Module Completion

At this point, we know that:

- The four common AI models are:

- **Generative Adversarial Networks (GANs)**
 - **Variational Autoencoders (VAEs)**
 - **Autoregressive Models**
 - **Flow-based Models**
- **Generative Adversarial Networks (GANs)** are particularly effective for **data augmentation**.
- **Variational Autoencoders (VAEs)** are well suited for:
 - Anomaly detection
 - Data compression
 - Collaborative filtering
 - Style transfer
- **Autoregressive Models** are commonly used for:
 - Text generation
 - Speech synthesis
 - Time series forecasting
 - Machine translation
- **Flow-based Models** are suitable for:
 - Image generation
 - Data generation
 - Density estimation
- Generative AI can tackle complex problems across a wide range of industries.
- Generative AI models are instrumental in addressing several data preparation and querying challenges, including:
 - Imputation of missing values
 - Detecting outliers
 - Reducing noise
 - Translating natural language queries into equivalent SQL statements

23 Module 1 Cheatsheet: Data Science and Generative AI

23.1 Popular Generative AI Tools

Name of Model	Usage	Link
Data Robot	A simple tool useful for data analysis and model building operations.	https://www.datarobot.com/
Mostly.AI	Synthetic data generation.	https://mostly.ai/
ChatGPT	GPT-based model used for text and code generation based on natural language queries.	https://openai.com/chatgpt
DB Sensei	Generate SQL queries for databases using natural language queries.	https://dbsensei.com/

23.2 Important Prompts for Data Preparation

Task	Prompt
Read a CSV data file and load it to a data frame.	Write a Python code that can perform the following tasks Read the CSV file, located on a given file path, into a Pandas data frame, assuming that the first rows of the file are the headers for the data.

Data cleaning: Identify and replace missing values per the following guidelines.	Write a Python code to perform the following tasks: 1. Identify the attributes with missing values. 2. Segregate these attributes into categorical and continuous valued attributes. 3. Drop the entire row if the value is missing in the target variable. 4. If the value is missing in a categorical attribute, replace the missing values with the most frequent value in the column. 5. If the value is missing in a continuous value attribute, replace the missing values with the mean value of the entries in the column.
Data Normalization: Normalize an attribute to its maximum value.	Write a Python code to normalize the content under a given attribute in a data frame df to its maximum value. Make changes to the original data, and do not create a new attribute.
Converting categorical variables into indicator variables	Write a Python code to perform the following tasks: 1. Convert a data frame df attribute into indicator variables, saved as df1, with the naming convention <code>Name_<unique value of the attribute></code>. 2. Append df1 into the original data frame df. 3. Drop the original attribute from the data frame df.

Module 2: Use of Generative AI for Data Science

24 Generative AI for Drawing Data Insights

24.1 Learning Objectives

After completing this topic, we will be able to:

- Use generative AI to create Python code that performs various operations relevant to drawing insights from a given dataset.
- Use hal9's free plan to generate statistical data analysis.
- Find missing values in a dataset using generative AI tools.

24.2 Using Generative AI to Generate Python Code for Data Analysis

Assume that our cleaned data is available in a CSV file. We can then use a generative AI model like **GPT-3.5** to create Python code that generates the analysis we require.

For example, a simple prompt for the tool would be:

Create Python code to generate the statistical description of cleaned data available in a CSV file.

The response of the model would generate valid Python code that uses the **pandas** library to produce statistical descriptions of the dataset using the `describe()` function.

24.3 Enhancing Data Analysis with Advanced Statistical Insights

The same prompt can be enhanced to include more advanced aspects for creating data insights.

For example, we can use the following prompt:

```
Create Python code to perform univariate, bivariate, and multivariate analysis of data available in a CSV file.
```

The generated code will produce a detailed statistical description of the data.

It will perform:

- **Univariate analysis** for any selected attribute.
- **Bivariate analysis** for any pair of attributes.
- **Multivariate analysis** by generating a **pair plot** (an ensemble plot consisting of bivariate analysis of all pairs of data attributes).

24.4 Extending the Analysis with Feature Selection and Feature Engineering

We can further extend the generated code by asking the model to include additional data analysis tasks.

For example, if we want to select the five best features that have the strongest relationship with the target attribute and engineer new features, we can continue the conversation using the following prompt:

```
In the code above, add the aspects of selecting the five best features that fit the target attribute as well as the aspect of engineering new features for the same.
```

The generated code will:

- Use the **SelectBest** method from the **scikit-learn** library to identify the five best features for the dataset.
- Create new polynomial features using the **PolynomialFeatures** method from the **scikit-learn** library.

24.5 Using hal9 to Generate Data Insights

Another way to generate insights quickly and effectively is by using [hal9's free plan](#).

For this demonstration, the dataset used is **student-mat.csv**, which contains student performance data collected from:

- School reports
- Questionnaires

The dataset includes:

- Student grades
- Demographic information
- Social features
- School-related features

After uploading the dataset and selecting **Create**, the platform suggests prompts that can be used to generate insights.

24.6 Distribution of Student Ages Across Schools

One suggested analysis is to determine the distribution of student ages across schools.

The generated output is a graphical representation of the age distribution.

For example:

- School **GP** has **57 students** aged **18**.
- School **MS** has **25 students** aged **18**.

The generated responses can also be saved for future use.

24.7 Finding Missing Values

Another useful analysis is checking for missing values in the dataset.

The generated response includes:

- Sample rows from the dataset.
- A tabular summary of column names.
- The count of missing values for each column.

In this demonstration, the output shows that **no values are missing**.

24.8 Generating Statistical Insights

The platform can also generate statistical insights for the complete dataset.

The statistical summary includes the following for quantitative columns:

- Count
- Mean
- Standard Deviation (STD)
- Minimum value (Min)
- Quartiles (25%, 50%, and 75%)
- Maximum value (Max)

For categorical columns, it provides:

- Unique values

- Count of each unique value

For example, the **school** attribute contains the unique values:

- GP
- MS

24.9 Key Takeaways

- Generative AI tools can generate Python code to perform various operations for drawing insights from a dataset.
- We can enhance generated code to perform **univariate**, **bivariate**, and **multivariate** analysis.
- We can extend the generated code by adding **feature selection** and **feature engineering** capabilities.
- The **SelectBest** method in **scikit-learn** can be used to identify the five best features related to the target attribute.
- The **PolynomialFeatures** method in **scikit-learn** can be used to engineer new polynomial features.
- **hal9's free plan** can generate statistical representations of datasets and identify missing values.
- Statistical summaries include descriptive statistics for quantitative columns and unique value counts for categorical columns.

25 Generative AI for Data Insights

25.1 Learning Objectives

After completing these exercises, we will be able to:

- Generate a statistical description of all the features in a dataset.
- Generate regression and box plots to compare feature distributions against a target attribute.
- Evaluate the correlation value, Pearson coefficient, and p-values of different attributes with the target attribute.
- Create pivot tables for grouped attributes and visualize them using pcolor plots.

25.2 Introduction

Drawing statistical and correlative insights is an essential step in understanding and interpreting data. In these exercises, we use Generative AI to generate Python code for exploratory data analysis (EDA), including descriptive statistics, data visualization, correlation analysis, and pivot table visualization using a dataset stored in a CSV file.

25.3 Exercise 1: Generate a Statistical Description of the Dataset

The dataset is assumed to be clean and ready for exploratory data analysis, with the first row of the CSV file containing the column headers.

25.3.1 Prompt

Write Python code to perform the following actions:

- Import a dataset from a CSV file. The headers for the dataset must be in the first row of the CSV file.
- Generate the statistical description of all the features used in the dataset, including **object** data types.

25.4 Exercise 2: Create Parameter Visualizations

To understand the relationship between different attributes and the target attribute (**Price**), we create:

- Regression plots for continuous attributes.
- Box plots for categorical attributes.

25.4.1 Continuous Attributes

- CPU_frequency
- Screen_Size_inch
- Weight_pounds

25.4.2 Categorical Attributes

- Category
- GPU
- OS
- CPU_core
- RAM_GB
- Storage_GB_SSD

25.4.3 Prompt

Write Python code to perform the following actions:

- Create regression plots for the attributes **"CPU_frequency"**, **"Screen_Size_inch"**, and **"Weight_pounds"** against **"Price"**.
- Create box plots for the attributes **"Category"**, **"GPU"**, **"OS"**, **"CPU_core"**, **"RAM_GB"**, and **"Storage_GB_SSD"** against the attribute **"Price"**.

25.5 Exercise 3: Evaluate Dependence

Not every attribute has the same level of influence on the target variable. To determine which attributes are most useful for predictive modeling, we evaluate the correlation value, Pearson coefficient, and p-values (the probability value used to determine the statistical significance of a relationship) for each numerical attribute against the target attribute.

25.5.1 Prompt

Write Python code to perform the following actions:

- Evaluate the correlation value, Pearson coefficient, and p-values for all numerical attributes against the target attribute **"Price"**.
- Do not include the values evaluated for the target variable against itself.
- Print these values as part of a single DataFrame against each individual attribute.

25.6 Exercise 4: Grouping and Pivot Tables

Another exploratory data analysis technique is grouping multiple attributes to create a pivot table. The pivot table summarizes the average value of the target attribute for different combinations of grouped variables and can be visualized using a pcolor plot (a color-coded matrix plot used to visualize values in a two-dimensional table).

In this exercise, we group the following attributes:

- GPU
- CPU_core
- Price

25.6.1 Prompt

Write Python code that performs the following actions:

- Group the attributes **"GPU"**, **"CPU_core"**, and **"Price"**, as available in a DataFrame named **df**.
- Create a pivot table for this group, assuming the target variable is **"Price"** and the aggregation function is **mean**.
- Plot a pcolor plot for this pivot table.

25.7 Summary

Through these exercises, we learned how to use Generative AI to generate Python code for exploratory data analysis. We covered:

- Generating statistical descriptions of dataset features.
- Creating regression plots and box plots for data visualization.
- Evaluating correlations using correlation values, Pearson coefficients, and p-values.
- Creating pivot tables and visualizing them using pcolor plots to gain meaningful insights from the data.

26 Generative AI for Data Visualization

26.1 Learning Objectives

After completing this topic, we will be able to:

- Explore datasets using Generative AI tools.
- Generate charts from datasets.
- Obtain exploratory insights into the relationships between variables.

26.2 Introduction

When it comes to developing or creating visualizations from data, there are many tools available besides those provided by major companies such as **IBM, Google, Microsoft, and Salesforce**.

In this topic, we will explore some of the Generative AI tools that are available either free of charge or on a trial basis. Data professionals can use these tools to create visualizations quickly and easily without spending hours writing code.

26.3 Creating Visualizations with Columns.ai

In this demonstration, we explore [Columns.ai](#), an AI-assisted storytelling platform, and see how it can easily generate charts.

26.3.1 Uploading and Exploring a Dataset

To begin:

- Initiate a new story.
- Upload the **student-mat.csv** dataset.
- Submit the dataset.
- Review the preview of the uploaded data along with the available configuration options.
- Save the dataset.

Once the dataset is uploaded, we can explore it immediately. The platform automatically generates charts and statistical summaries.

For example, the **Age** attribute shows:

- Minimum student age: **15**
- Maximum student age: **22**

26.3.2 Generating a Bar Chart

To determine the total number of male and female students, we enter the appropriate prompt.

The generated bar chart displays the distribution of **395** students.

By hovering over each bar, we observe:

- Female students: **208**
- Male students: **187**

This shows that there are slightly more female students than male students in the dataset.

Once satisfied with the result, we click **Apply** to use the generated chart.

26.3.3 Creating a Pie Chart

We can also use the built-in wizard to create charts.

In this example, we create a **pie chart** representing the **average weekly alcohol consumption**, identified by the **Walc** value, for each gender.

To improve the presentation:

- Click **Summary**.
- Select **Smart Title**.

The platform automatically generates an appropriate chart title.

26.3.4 Generating Insights

After creating the chart, we can generate AI-powered insights.

The generated insight can also be edited and annotated according to our requirements.

26.3.5 Customizing and Exporting Charts

We can easily customize the chart's appearance.

After completing the customization, we can download the chart as:

- PNG image
- SVG image
- CSV data
- JSON data

26.4 Exploring Data with Akkio

For the next demonstration, we use [Akkio's free trial version](#).

The dataset used is:

- **retail sales data.csv** (one of the sample datasets provided by Akkio)

26.4.1 Creating a Scatter Plot

To begin:

- Create a new project.
- Import the dataset into the project.

Next, use **Chat Explorer** to generate an exploratory insight into the relationship between **marketing spend** and **sales** by creating a **scatter plot**.

The platform also provides details about the approach used to generate the chart.

The chart can also be downloaded.

26.4.2 Generating a Bar Chart

Using a simple prompt, we generate a **bar chart** showing the **average sales by area**.

The chart is created automatically with minimal effort.

26.4.3 Creating a Correlation Matrix

To understand the relationships among data attributes, generate a **correlation matrix** displayed as a **heat map**.

The color of each square represents both the **strength** and **direction** of the correlation.

Interpretation of the heat map:

- Lighter colors represent stronger correlations.
- Darker colors represent weaker correlations.
- Positive correlations are represented using one color.
- Negative correlations are represented using another color.

This enables us to identify which attributes are strongly correlated and which are weakly correlated.

26.4.4 Identifying Outliers

To verify potential outliers, generate **box plots**.

Using a single prompt, the tool automatically creates box plots for all applicable attributes in the dataset.

26.4.5 Understanding Data Distribution

We can also generate **histograms** for all attributes to understand the spread (distribution) of values.

26.5 Key Takeaways

- Generative AI tools can generate charts quickly and efficiently.
- Generative AI tools can analyze datasets and generate insights.
- Generative AI tools help us explore relationships between variables.
- Generative AI tools can generate:
 - Correlation matrices (heat maps)

- Box plots
- Histograms
- AI-powered visualization tools significantly reduce the time and coding effort required for exploratory data analysis (EDA).

27 Generative AI for Storytelling

27.1 Learning Objectives

After completing these exercises, we will be able to:

- Sign up for **Zoho Analytics** and upload a dataset.
- Generate reports by prompting **Zia AI**.

27.2 Introduction

Generative AI can be used to create interactive reports by analyzing datasets and generating charts with meaningful insights. In this topic, we use **Zoho Analytics** and its AI assistant **Zia AI** to explore a dataset, generate visualizations, create a dashboard, and prepare a report for sharing.

27.3 Exercise 1: Sign Up for Zoho Analytics

- Sign up for a **Zoho Analytics** account.

27.4 Exercise 2: Upload a Dataset

- Upload the **electronics_dataset.csv** dataset into **Zoho Analytics**.

27.5 Exercise 3: Use Zia AI to Explore Data and Generate Reports

Generate the following visualizations by entering each prompt into **Zia AI**.

27.5.1 Prompts

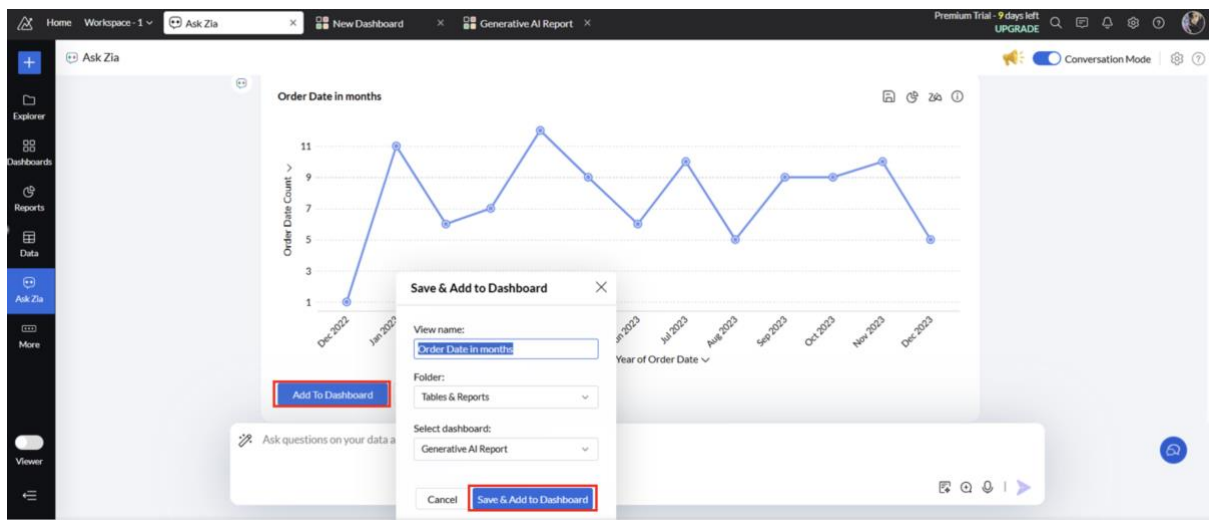
- **Prompt 1:** Show a line chart of total revenue grouped by month using the **Order Date** column.
- **Prompt 2:** Create a bar chart of the top **10 products** by total revenue.
- **Prompt 3:** Create a pie chart showing each product's contribution to total revenue.
- **Prompt 4:** Create a pivot table with **Product Name**, **Total Quantity Ordered**, and **Total Revenue**.

After generating the charts:

- Create a new dashboard.
- Add all generated visualizations to the dashboard.

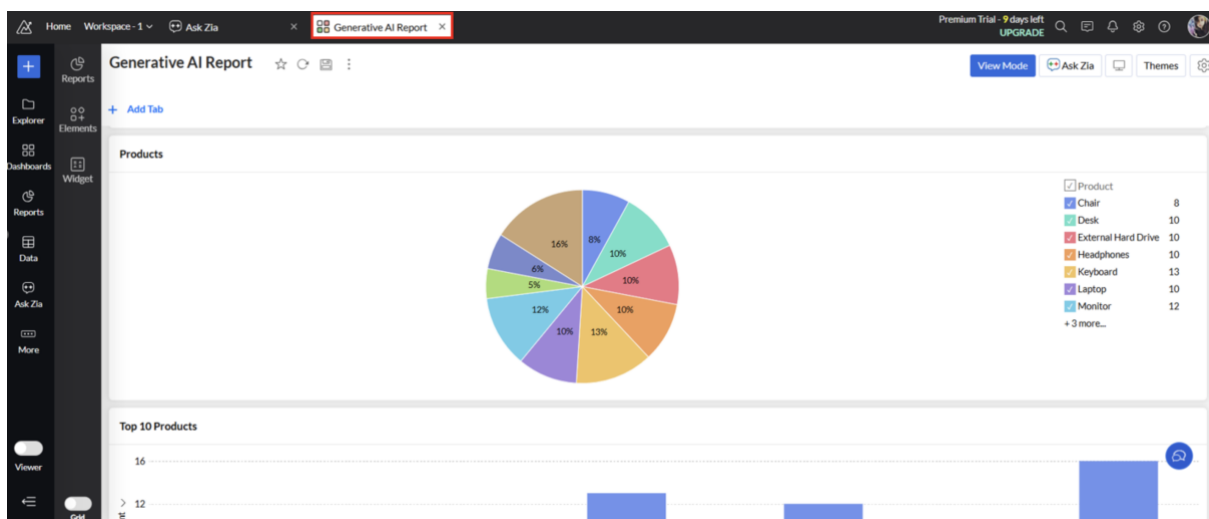
27.6 Exercise 4: Finalize the Dashboard

- Save each generated chart.
- Add all charts to the dashboard.
- Verify that all visualizations appear correctly in the dashboard.



27.7 Exercise 5: Save and Share the Report

- Export the completed dashboard.
- Share the dashboard with colleagues or team members when required.



27.8 Summary

Through these exercises, we learned how to use **Zoho Analytics** and **Zia AI** to generate AI-powered reports. We uploaded a dataset, created multiple visualizations using natural language prompts, combined them into an interactive dashboard, and exported and shared the final report.

28 Generative AI Tools for Model Development

28.1 Learning Objectives

After studying this topic, we will be able to:

- Explain the different Generative AI tools used for model development.
- Discuss the merits and demerits of each tool.

28.2 Introduction

Predictive modeling is experiencing a transformative shift driven by the emergence of powerful Generative AI tools.

These tools are not merely automating existing processes but are changing how data scientists build, train, and deploy predictive models, unlocking unprecedented accuracy, efficiency, and potential **interpretability (the ability to understand how a model makes its predictions)**.

In this evolving landscape, data scientists can leverage Generative AI to:

- Accelerate model development.
- Explore innovative model architectures.
- Gain deeper insights into data.
- Improve decision-making across industries and applications.

Data scientists use several Generative AI tools to develop predictive models.

Some widely used tools include:

- DataRobot
- AutoGluon
- H2O Driverless AI
- Amazon SageMaker Autopilot
- Google Vertex AI

These tools are widely used for building and deploying machine learning models.

28.3 DataRobot

DataRobot is a leading enterprise AI platform that automates and accelerates the process of building, deploying, and managing Machine Learning (ML) models.

28.3.1 Merits

- Automates the end-to-end machine learning workflow.
- Supports various machine learning algorithms.
- Provides a user-friendly interface.
- Offers both cloud-based and on-premises deployment options.

28.3.2 Demerits

- Can be expensive for small businesses or individual users.
- Offers limited customization compared to open-source tools.
- The black-box nature (internal decision-making process is not easily visible) of some algorithms might hinder interpretability.

28.4 AutoGluon

AutoGluon is an open-source Automated Machine Learning (AutoML) library that simplifies the development and deployment of machine learning models by automating:

- Model selection
- Hyperparameter tuning
- Model training

It also supports various interpretability and explainability features.

28.4.1 Merits

- Open-source AutoML framework.
- Automates model selection and hyperparameter tuning.
- Supports interpretability and explainability features.

28.4.2 Demerits

- Requires basic Python programming knowledge.
- Has limited customization compared to some other tools.
- May not be suitable for large or complex datasets.

28.5 H2O Driverless AI

H2O Driverless AI is a cloud-based Automated Machine Learning platform that supports automatic model building, deployment, and monitoring, enabling organizations to harness the power of AI without requiring extensive data science expertise.

28.5.1 Merits

- Supports automatic model building, deployment, and monitoring.
- Provides an intuitive drag-and-drop interface.
- Includes built-in explainability tools for understanding model predictions.

28.5.2 Demerits

- Can be expensive for individual users because it primarily caters to enterprise needs.
- Less flexible than open-source alternatives.
- Provides limited access to the underlying code and algorithms.

28.6 Amazon SageMaker Autopilot

Amazon SageMaker Autopilot is a fully managed AutoML service that automates the process of building, training, and deploying machine learning models, making it accessible to users with varying levels of expertise.

28.6.1 Merits

- Fully integrated with the AWS ecosystem for seamless data management and deployment.
- Supports automatic model building, tuning, and model selection.
- Offers a pay-as-you-go pricing model for cost-effective usage.

28.6.2 Demerits

- Vendor lock-in to the AWS platform, making migration to another cloud provider or platform less straightforward.
- Limited customization compared to other tools.
- Some Autopilot models have a black-box nature, requiring additional analysis.

28.7 Google Vertex AI

Google Vertex AI is a fully managed cloud-based machine learning platform that integrates with Google Cloud services.

It supports advanced deep learning models and custom algorithms while providing Explainable AI features for understanding model predictions.

28.7.1 Merits

- Fully managed cloud platform.
- Integrates with Google Cloud services.
- Supports advanced deep learning models.
- Supports custom algorithms.
- Provides Explainable AI features.

28.7.2 Demerits

- Can be complex for beginners because it requires expertise in Google Cloud.
- Can become expensive due to cloud resource consumption.
- Limited support for non-Google machine learning libraries and frameworks.

28.8 Using ChatGPT and Google Bard for Model Development

While dedicated Generative AI tools are excellent for building and deploying machine learning models, Generative AI alternatives such as ChatGPT and Google Bard offer unique capabilities.

Developers who are comfortable with Python can leverage these tools for AI-powered script generation, helping streamline the model-building process.

28.8.1 Example Prompt

Write a Python script to build a predictive model for a dataset available in a Pandas DataFrame.

Include the following steps:

1. Find the top five features that affect the target parameter the most.
2. Use linear and polynomial models on these parameters to find the most optimal model and use Ridge Regression to evaluate.
3. Use techniques like Grid Search to identify the optimum hyperparameters.

This prompt generates detailed Python code that performs all the requested tasks.

The generated code enables data scientists with beginner-level Python skills to:

- Use the generated solution.
- Personalize the code.
- Optimize it according to their application requirements.

28.8.2 Important Note

It is essential to note that using AI-generated code verbatim (exactly as generated without modification) may constitute plagiarism.

Instead, we should:

- Use prompts to generate generic code.
- Understand the generated code.
- Personalize and modify it according to our application requirements.
- Create original solutions for our prospective applications.

28.9 Key Takeaways

- Predictive modeling is undergoing a transformative shift due to the emergence of powerful Generative AI tools.
- DataRobot automates the end-to-end machine learning process.
- AutoGluon is an open-source tool that automates model selection.
- H2O Driverless AI is a cloud-based platform that supports automatic model building.

- Amazon SageMaker Autopilot is integrated with the AWS ecosystem and offers a pay-as-you-go pricing model.
- Google Vertex AI is a scalable cloud-based platform that supports advanced deep learning models and custom algorithms.
- ChatGPT and Google Bard can be leveraged for AI-powered script generation to streamline the model-building process.

29 Generative AI for Models Development

29.1 Learning Objectives

After completing these exercises, we will be able to:

- Use linear regression with a single variable to train a predictive model.
- Use linear regression with multiple variables to train a predictive model.
- Use polynomial regression with a single variable.
- Create a machine learning pipeline that performs polynomial feature scaling and linear regression.
- Apply Grid Search with Cross-Validation and Ridge Regression to determine the optimal model hyperparameters.

29.2 Introduction

Predictive models can be developed efficiently with the help of Generative AI by generating Python code for different machine learning techniques. In this topic, we use Generative AI to create, evaluate, and compare multiple regression models, build machine learning pipelines, and optimize model performance using Grid Search and Ridge Regression.

29.3 Import the Dataset

Read the CSV file, located on a given file path, into a Pandas DataFrame, assuming that the first row of the file can be used as the headers for the data.

29.3.1 Prompt

Write Python code that performs the following task:

- Read the CSV file from a specified file path into a Pandas DataFrame.
- Assume that the first row of the CSV file contains the column headers.

29.4 Linear Regression with a Single Variable

Develop a linear regression model using a single feature to predict the target variable and evaluate the model using Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination).

29.4.1 Prompt

Write Python code that performs the following tasks:

- Develop and train a linear regression model that uses one attribute of a DataFrame as the source variable and another attribute as the target variable.
- Calculate and display the Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination) for the trained model.

29.5 Linear Regression with Multiple Variables

Train a linear regression model using multiple input features to predict the target variable and evaluate its performance.

29.5.1 Prompt

Write Python code that performs the following tasks:

- Develop and train a linear regression model that uses multiple attributes of a DataFrame as the source variables and one attribute as the target variable.
- Calculate and display the Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination) for the trained model.

29.6 Polynomial Regression

Polynomial regression extends linear regression by generating higher-order features, allowing the model to capture more complex relationships between variables.

29.6.1 Prompt

Write Python code that performs the following tasks:

- Develop and train multiple polynomial regression models with polynomial degrees (orders) **2, 3, and 5**, using one attribute of a DataFrame as the source variable and another attribute as the target variable.
- Calculate and display the Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination) for each trained model.
- Compare the performance of the polynomial regression models.

29.7 Create a Machine Learning Pipeline

A machine learning pipeline automates a sequence of preprocessing and modeling steps, including feature scaling, polynomial feature generation, and linear regression.

29.7.1 Prompt

Write Python code that performs the following tasks:

- Create a machine learning pipeline that performs:
- Parameter scaling
- Polynomial feature generation
- Linear regression
- Use the same set of multiple input features as in the previous exercise.

- Calculate and display the Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination) for the trained pipeline model.

29.8 Grid Search and Ridge Regression

Grid Search combined with Cross-Validation identifies the optimal hyperparameter values for a Ridge Regression model, resulting in improved predictive performance.

29.8.1 Prompt

Write Python code that performs the following tasks:

- Use polynomial features for selected attributes of a DataFrame.
- Perform Grid Search on a Ridge Regression model using different values of the hyperparameter **alpha** and polynomial features as input.
- Use Cross-Validation during the Grid Search process.
- Evaluate the resulting model by calculating and displaying the Mean Squared Error (MSE) and R^2 Score (Coefficient of Determination).

29.8.2 Model Configuration

Source Variables: CPU_frequency, RAM_GB, Storage_GB_SSD, CPU_core, OS, GPU, Category

- **Target Variable:** Price
- **Alpha Values:** 0.0001, 0.001, 0.01, 0.1, 1, 10
- **Cross-Validation:** 4-fold
- **Polynomial Degree:** 2

29.9 Summary

In this topic, we learned how to use Generative AI to generate Python code for developing and evaluating predictive models. We explored:

- Single-variable linear regression
- Multiple-variable linear regression
- Polynomial regression
- Machine learning pipelines
- Hyperparameter optimization using Grid Search with Cross-Validation and Ridge Regression

These techniques help build more effective predictive models by improving model training, feature engineering, and hyperparameter selection.

30 Generative AI for Understanding Data and Model Development

30.1 Learning Objectives

After studying this topic, we will be able to:

- Summarize the techniques that help in **Exploratory Data Analysis (EDA)**.
- Discuss the advantages of using **Generative AI** for model development.

30.2 Generative AI for Exploratory Data Analysis (EDA)

Generative AI is a powerful tool for **Exploratory Data Analysis (EDA)**, offering a range of techniques that enhance data understanding, uncover hidden patterns, and generate new insights.

30.2.1 Statistical Data Description

Generative AI models such as **Variational Autoencoders (VAEs)** can:

- Generate descriptive statistics for numerical and categorical data.
- Capture the underlying data distribution.
- Generate output that resembles the original data distribution.

30.2.2 Univariate Analysis

Generative AI models such as **Generative Adversarial Networks (GANs)** can facilitate univariate analysis.

Data scientists can train GANs to generate synthetic data that mimics the distribution of a particular variable. They can use this generated data to:

- Detect outliers.
- Understand the distribution of the variable.

30.2.3 Bivariate Analysis

Data scientists use generative AI models to explore relationships between pairs of variables.

For instance, **Copulas** (statistical models that capture the dependency between variables) can model the joint distribution of two variables to reveal their potential correlation or conditional dependency.

30.2.4 Multivariate Analysis

Generative AI can uncover complex relationships among multiple variables.

For example, **VAEs** can reduce the dimensionality of high-dimensional data while preserving the underlying relationships between variables. This ability helps analyze complex data relationships.

30.2.5 Feature Engineering

Data scientists can use generative AI to enhance feature engineering by generating new features that capture the underlying structure of the data.

For instance, **GANs** can create new samples that resemble the original data distribution, providing a richer data representation.

30.2.6 Hypothesis Generation

Generative AI can assist in hypothesis generation by identifying potential patterns and relationships in the data that may warrant further investigation.

For example, **VAEs** can reveal anomalies or outliers that could provide new insights for subsequent modeling and decision-making processes.

30.3 Advantages of Generative AI for Model Development

There are several advantages of using generative AI in building a predictive model, even after completing data augmentation and feature engineering.

30.3.1 Model Architecture Selection

Generative AI models can help select the most appropriate model architecture for a given dataset.

For instance, **VAEs** can generate **latent data representations** (lower-dimensional versions of the original data that capture the underlying structure of the data).

Data scientists can use these latent representations to evaluate the performance of various machine learning algorithms, such as:

- Linear models
- Decision trees
- Neural networks

This helps identify the most effective model architecture for the given dataset.

30.3.2 Feature Selection

Data scientists leverage generative AI models such as **Mutual Information Neural Networks (MINNs)** to systematically assess the importance of different features in a dataset for predicting the target variable.

MINNs measure the **mutual information** between features and the target variable.

- High mutual information values indicate features that are likely more essential for accurate predictions.

30.3.3 Ensemble Model Generation

Ensemble models are a combination of multiple predictive models created to predict outcomes more accurately.

Generative AI can create ensemble models by generating diverse representations of the data.

For example, **GANs** consist of two neural networks:

- **Generator** – Generates new representations of the data.
- **Discriminator** – Distinguishes between real and generated representations.

This adversarial process forces the model to produce increasingly realistic data representations.

30.3.4 Model Interpretability

Generative AI can interpret the predictions made by predictive models by generating representations of the data that highlight the most influential features of a given prediction.

By reconstructing the input data from the model's latent representations, generative AI can explain the model's decision-making process.

For example, **interpretable autoencoders** can generate explanations about how the model makes a particular prediction.

30.3.5 Improving Generalization

Generative AI is highly effective for improving the **generalization ability** of predictive models.

Generalization refers to the ability of a model to perform well on data that it has not been trained on.

Generative AI can also help prevent **overfitting**, which occurs when a model becomes too specific to the training data and fails to generalize well to unseen data.

For example, **denoising autoencoders** enhance generalization by learning robust representations of the underlying structure in the data while avoiding overfitting to specific details in the training data.

30.4 Key Takeaways

- Generative AI enhances **Exploratory Data Analysis (EDA)** through:
 - Statistical data description
 - Univariate analysis
 - Bivariate analysis

- Multivariate analysis
- Feature engineering
- Hypothesis generation
- Generative AI provides several advantages during predictive model development by:
 - Selecting suitable model architectures
 - Identifying important features
 - Generating ensemble models
 - Improving model interpretability
 - Improving generalization
 - Preventing overfitting

31 Generative AI for Identifying Patterns and Insights in Large Datasets

31.1 Learning Objectives

After studying this topic, we will be able to:

- Explain how generative AI identifies patterns and insights from large datasets that conventional methods might miss.
- Describe the advantages of using generative AI for model building and validation in predictive analytics.
- Discuss the limitations of using generative AI in predictive analytics.

31.2 How Generative AI Identifies Patterns and Insights

Generative AI can process and analyze large datasets at a scale and speed beyond human capability. It excels at uncovering hidden patterns, correlations, or anomalies that conventional methods might overlook.

For example, AI algorithms can:

- Detect trends in customer behavior.
- Predict market shifts.
- Highlight inefficiencies in operations.
- Examine vast amounts of data across multiple variables.

A key advantage of generative AI is its ability to continuously learn and refine insights as more data becomes available, making the process more dynamic and iterative compared to traditional methods.

31.2.1 Processing Large Volumes of Data

Generative AI excels at processing vast amounts of data quickly and efficiently using advanced algorithms.

This enables it to:

- Detect subtle patterns.

- Identify hidden correlations.
- Discover insights that traditional analytical methods may overlook.

31.2.1.1 Healthcare Example

In the healthcare sector, AI models such as **Google's DeepMind** have been used to analyze vast amounts of medical data.

This helps uncover patterns that support:

- Early disease detection.
- Personalized treatment plans.

31.2.2 Foundation Models vs. Traditional Machine Learning Models

Foundation models are known for their ability to generalize to new, unseen data.

As a result, they can uncover patterns and insights that might not be captured by traditional machine learning models.

Traditional machine learning models, such as:

- Support Vector Machines (SVMs)
- Random Forests

are typically trained on a specific task using specific data.

In contrast, foundation models are trained on a wide and diverse set of knowledge across multiple domains and tasks. Because of this broader knowledge base, they are able to uncover more comprehensive insights.

31.2.3 The Role of the Attention Mechanism

The **attention mechanism** is a key concept in generative AI.

Before the introduction of attention mechanisms, researchers attempted to identify patterns in sequential data using:

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory (LSTM) networks
- Convolutional Neural Networks (CNNs)

Long Short-Term Memory (LSTM) models typically failed in many cases to identify past or long-term patterns, making long-term forecasting unsuccessful.

Attention mechanisms overcome this limitation by allowing the model to focus on different parts of the data without memory concerns. As a result, forecasting long-term prediction sequences becomes possible.

31.2.4 Learning Complex Relationships

Generative AI has the ability to learn complex relationships within data very easily and very quickly.

Compared to human analysis, its processing speed is remarkable.

Generative AI can help uncover:

- Hidden connections.
- Hidden anomalies.

These are patterns that a human analyst might otherwise miss.

This advantage becomes especially significant when working with:

- Complex datasets.
- Unstructured datasets.

The difference between human analysis and generative AI analysis becomes much more apparent in these situations.

31.3 Advantages of Generative AI for Model Building and Validation

Generative AI offers significant advantages in predictive analytics by speeding up the model-building process, automating hyperparameter tuning, and testing different model architectures.

It can generate accurate models faster than traditional methods and quickly validate them by running simulations.

31.3.1 Synthetic Data Generation

One major benefit of generative AI is its ability to generate synthetic data.

This is especially valuable when:

- Rare-event data is available.
- Only a limited amount of data exists.

31.3.2 Automation and Efficiency

Generative AI can automate many aspects of the data science workflow, including:

- Data processing.
- Feature selection.
- Feature engineering.
- Identifying complex patterns.

These capabilities help accelerate model building and validation.

31.3.3 Adaptive Learning

Generative AI can continuously learn from new data, allowing predictive models to adapt and evolve over time.

31.3.4 Doing More with Less

One major advantage of generative AI is that we can accomplish more with fewer resources.

Instead of retraining a new model with new data for every use case, the same generative AI model can perform a wide variety of tasks across multiple use cases.

In the long term, this helps reduce:

- Time required to gather new data.
- Costs associated with collecting data.
- Time required to retrain new models.

31.4 Limitations of Generative AI in Predictive Analytics

Although generative AI offers numerous advantages, several limitations must also be considered.

31.4.1 High Computational Cost

Generative AI models can be computationally very expensive.

Many models require substantial computing resources for both training and inference.

31.4.2 Black-Box Nature

Many generative AI models are highly complex and function as **black-box models** (models whose internal decision-making process is difficult to interpret).

This makes it difficult to understand how they arrive at particular results or conclusions.

Compared with traditional machine learning, generative AI currently offers fewer straightforward methods for achieving the same level of transparency and explainability.

31.4.3 Model Bias

Before deploying a generative AI model, we should consider whether the model is unbiased.

Bias in training data or model behavior can negatively affect predictions and decision-making.

31.4.4 Dependence on Data Quality

Although generative AI can generate predictive models, the quality of those models depends heavily on the quality of the input data.

Poor-quality data often leads to poor-quality predictions.

31.4.5 Computational Hardness

Another limitation is **computational hardness** (high computational complexity requiring significant processing power and time).

Training and running large generative AI models can be resource-intensive.

31.4.6 Lack of Domain Context

Generative AI may lack sufficient domain context.

Data scientists should carefully identify and focus on the appropriate domain-specific context at the beginning of a project.

31.4.7 Risk of Overfitting

There is a potential risk of overfitting if the model learns too closely from the training data.

An overfitted model may perform well on training data but poorly on unseen data.

31.4.8 Challenges in Regulated Industries

In highly regulated industries or sensitive applications, transparency and explainability remain critical challenges.

These requirements often make it difficult to rely solely on black-box generative AI models.

31.5 Key Takeaways

- Generative AI processes and analyzes large datasets faster than human capability.
- It uncovers hidden patterns, correlations, and anomalies that conventional methods may overlook.
- Foundation models generalize across diverse domains, enabling broader insight discovery than traditional machine learning models.
- The attention mechanism enables effective long-term sequence prediction without the memory limitations of earlier models such as LSTMs.
- Generative AI excels at learning complex relationships within structured, semi-structured, and unstructured data.
- Major advantages include synthetic data generation, automation, adaptive learning, hyperparameter tuning, faster model development, and reusable models across multiple use cases.

- Key limitations include high computational cost, black-box behavior, model bias, dependence on data quality, computational complexity, lack of domain context, risk of overfitting, and explainability challenges in regulated industries.

32 Considerations While Using Generative AI in Industries

32.1 Learning Objectives

After studying this topic, we will be able to:

- Summarize the key considerations while using Generative AI in various industries.
- List the industry-specific considerations of using Generative AI.

32.2 Introduction

The application of Generative AI is not without due consideration. When using Generative AI in data science, several considerations are crucial to ensure the effectiveness, fairness, and responsible use of the technology across various applications.

The key considerations while using Generative AI include:

- **Data considerations**
- **Model considerations**
- **Ethical considerations**

32.3 Data Considerations

Generative AI models are trained on large datasets. The effectiveness of these models depends on the quality of the training data.

If the training data is inaccurate or biased, the generated output is likely to amplify those biases. Therefore, we should thoroughly evaluate the quality and representativeness of the data and eliminate bias when training Generative AI models.

32.4 Model Considerations

The choice of Generative AI model and its training parameters determines the explainability and interpretability of the generated output.

- **Explainability** refers to the model's ability to provide clear insights into its decision-making process.
- **Interpretability** refers to the ease with which we can understand the model's output.

We should select a model that provides both explainability and interpretability.

To improve interpretability, we should use techniques such as:

- Feature attribution (identifying which input features contribute most to a prediction)

- Partial dependence plots (visualizations showing how one or more features influence the model's predictions)

32.5 Ethical Considerations

The ethical implications of using Generative AI must be carefully considered.

Generative AI models can be misused for malicious activities such as:

- [Creating deepfakes](#)
- [Manipulating public opinion](#)
- [Spreading misinformation](#)

We must establish ethical guidelines for the use of Generative AI and ensure that models are used responsibly without contributing to malicious activities.

32.6 Industry-Specific Considerations

32.6.1 Finance Industry

32.6.1.1 Data Considerations

The finance industry deals with:

- Sensitive financial data
- Regulatory requirements
- Potential impacts on financial markets

We should:

- Use encryption techniques to comply with data privacy regulations.
- Establish clear data access protocols.

32.6.1.2 Model Considerations

Generative AI models used in finance must be robust against adversarial attacks (intentional attempts to manipulate model inputs to produce misleading outputs).

We should:

- Use interpretability techniques such as feature attribution and partial dependence plots.
- Ensure robust optimization to improve the model's resilience against adversarial attacks.

32.6.1.3 Ethical Considerations

Generative AI models can perpetuate biases present in training data, leading to discriminatory financial decisions such as biased loan approvals.

We should:

- Check the data for potential biases.
- Use fairness metrics (measurements used to evaluate whether AI decisions are fair across different groups).
- Apply adversarial training (training the model using intentionally modified inputs to improve robustness and fairness).

32.6.2 Healthcare Industry

32.6.2.1 Data Considerations

The healthcare industry generates highly sensitive data, including:

- Medical records
- Imaging data
- Genetic information
- Clinical trial data

We should:

- Carefully evaluate the quality and representativeness of the data.
- Ensure the data accurately reflects the patient population.
- Ensure the data is free from bias and errors.

32.6.2.2 Model Considerations

Generative AI models used in healthcare must:

- Comply with HIPAA (Health Insurance Portability and Accountability Act) and other regulations.
- Be highly accurate.
- Be interpretable.
- Be robust against adversarial attacks that could lead to:
 - Misdiagnosis
 - Inaccurate treatment recommendations
 - Manipulation of patient data

We should select models that can:

- Anonymize patient data (remove personally identifiable information while preserving data usefulness).
- Control data access.

32.6.2.3 Ethical Considerations

Ethical considerations in healthcare must include transparency and consent.

We should:

- Inform patients about the use of Generative AI in their care.
- Explain the limitations and potential risks of the model.

- Obtain informed consent from patients.
- Ensure patients have the right to access and review their AI-generated data.
- Check the data for potential biases.
- Use techniques to mitigate bias.

32.6.3 Retail Industry

32.6.3.1 Data Considerations

Retail data includes:

- Customer purchase history
- Product specifications
- Market trends
- Social media interactions

We should:

- Select Generative AI models that generate accurate product designs.
- Develop effective marketing strategies.
- Use data augmentation techniques to ensure augmented data retains the underlying patterns and distribution of the original data.

32.6.3.2 Model Considerations

The choice of Generative AI model architecture depends on the task.

Examples include:

- **Generative Adversarial Networks (GANs):** Well suited for generating realistic product images.
- **Recurrent Neural Networks (RNNs):** Effective in predicting purchase patterns.

We should use interpretability techniques to create accurate model predictions.

32.6.3.3 Ethical Considerations

Ethical considerations in retail include handling customer data such as:

- Personal information
- Purchase history

Generative AI models can perpetuate biases present in training data, leading to unfair product recommendations.

We should:

- Use techniques to mitigate bias.
- Ensure compliance with privacy regulations.
- Implement robust data security measures.

- Obtain informed consent from customers before using their data.

32.7 Key Takeaways

- While using Generative AI models, we need to consider **data, model, and ethical considerations**.
- The effectiveness of generated output depends on the quality of the training data.
- We should select models that provide both **explainability** and **interpretability**.
- Generative AI models can be misused for malicious activities if appropriate safeguards are not implemented.
- Different industries have unique data, model, and ethical considerations that must be addressed before deploying Generative AI solutions.

33 Challenges While Using Generative AI

33.1 Learning Objectives

After completing this topic, we will be able to:

- Summarize the challenges while using Generative AI in Data Science.
- Discuss the three key challenges while using Generative AI in Data Science.

33.2 Overview

Data professionals face various challenges when using Generative AI across multiple industries. These challenges go beyond the well-known concerns of bias, robustness, privacy, consent, and ethics.

The challenges can be broadly categorized into three areas:

- Technical challenges
- Organizational challenges
- Cultural challenges

Addressing these challenges requires the responsible deployment of Generative AI technologies, along with strategic planning, transparency, and continuous improvement.

33.3 Technical Challenges

The use of Generative AI presents several technical challenges, including:

- Quality and availability of training data.
- Striking a balance between model complexity and interpretability.
- The resource-intensive nature of training large-scale computational models.
- Lack of standardization and tools.

33.3.1 Training Data Quality and Availability

Generative AI models are highly data-driven.

The quality of the training data significantly impacts the performance of these models. It is often challenging for data professionals to find relevant, high-quality, and well-labeled data, especially for niche applications or when dealing with sensitive data.

33.3.2 Model Interpretability

Generative AI models can be difficult to interpret, making it challenging to:

- Understand the model's decision-making process.
- Assess its reliability.
- Identify potential biases.

33.3.3 AI Hallucinations

Data professionals may face **AI hallucinations** (the generation of inaccurate or illogical information by Generative AI models) due to:

- Flawed training data.
- Inappropriate model architectures.
- Inadequate evaluation methods.

33.3.4 Computational Resource Requirements

Training and running Generative AI models can be computationally expensive, requiring specialized hardware and software infrastructure.

This constraint can pose a significant barrier for organizations with limited resources or those operating in cloud environments.

33.3.5 Lack of Standardization

Generative AI technology is still in its early stages and requires more standardization regarding:

- Model architectures.
- Training procedures.
- Evaluation frameworks.

This lack of standardization makes it difficult for data professionals to effectively compare different models and tools.

33.4 Organizational Challenges

Implementing Generative AI in organizations comes with its own set of challenges, including:

- Copyright and intellectual property issues.

- Skill shortages.
- Integration with existing systems.
- Change management.
- Measuring Return on Investment (ROI).

Overcoming these challenges requires a strategic approach, including process adjustments, skill development, and commitment to ethical practices.

33.4.1 Copyright and Intellectual Property Issues

Some companies currently disallow the use of public Generative AI models to create content because it can create copyright and intellectual property issues.

To overcome these issues, organizations can develop their own customized Generative AI models.

With time, more specific guidelines about the use of Generative AI will likely be established.

33.4.2 Skill Gaps

The demand for machine learning engineers and data scientists with expertise in Generative AI is growing consistently.

However, the supply of skilled professionals is limited, making it challenging for organizations to build and maintain in-house Generative AI capabilities.

33.4.3 Integration with Existing Systems

Integrating Generative AI into existing data architectures and workflows can be complex and time-consuming.

This integration often requires changes to:

- Data pipelines.
- Decision-making processes.
- Risk management frameworks.

33.4.4 Change Management

Introducing Generative AI into organizations can be met with resistance from stakeholders concerned about:

- Job displacement.
- Data privacy.
- The potential impact on decision-making processes.

Effective change management strategies are crucial for successful adoption.

33.4.5 Measuring Return on Investment (ROI)

Determining the Return on Investment (ROI) for projects involving Generative AI can be challenging because:

- Some applications may take a longer time to show tangible results or benefits.
- Many valuable outcomes are difficult to express in monetary terms.

Data professionals must develop robust evaluation frameworks to assess the impact of Generative AI initiatives.

33.5 Cultural Challenges

Organizations also face several cultural challenges while implementing Generative AI.

These include:

- Risk aversion.
- Data sharing and collaboration issues.
- Trust and transparency concerns.
- Continuous learning and adaptation.

Successfully navigating these challenges involves promoting secure data practices, enhancing transparency, and fostering a continuous learning mindset.

33.5.1 Risk Aversion

Generative AI is a relatively new technology, and many organizations may be hesitant to embrace it without clear assurance about its impact.

This risk aversion can hinder innovation using Generative AI applications.

33.5.2 Data Sharing and Collaboration

Generative AI models require large amounts of data for training.

However, organizations' concerns about the security of their proprietary data make them reluctant to share it.

This lack of data sharing can limit collaboration and hinder the development of more robust and generalizable models.

33.5.3 Trust and Transparency

Generative AI models are powerful but complex and opaque (difficult to understand how they produce outputs).

Because of this opaqueness, stakeholders may not find the outputs reliable.

Building trust and transparency requires:

- Open communication.
- Explainable AI techniques.
- Transparent governance frameworks.

33.5.4 Continuous Learning and Adaptation

Generative AI models must be continuously updated and adapted to changing data distributions and business requirements.

This requires:

- A commitment to continuous learning.
- A culture of data-driven decision-making.

33.6 Key Takeaways

- Data professionals face various challenges when using Generative AI models across multiple industries.
- Technical challenges include:
 - Data quality.
 - Model interpretability.
 - Computational resources.
 - Lack of standardization.
- Organizational challenges include:
 - Skill gaps.
 - Integration with existing systems.
 - Change management.
 - Measuring ROI.
- Cultural challenges include:
 - Risk aversion.
 - Data sharing and collaboration.
 - Trust and transparency issues.
 - Continuous learning.
- Successfully navigating these challenges requires a strategic approach to ensure the responsible deployment of Generative AI technologies while maintaining transparency and ethical practices.

34 Responsible Generative AI for Data Professionals

34.1 Introduction

Generative AI, including models like **GPT-3** and **GPT-4**, presents both opportunities and challenges. Some of the major challenges include:

- Bias and fairness
- Ethical use of AI
- Explainability (the ability to understand how AI makes decisions)

- Privacy concerns
- Environmental impact

To address these challenges, **Responsible AI** (also referred to as **Ethical AI** or **Trustworthy AI**) should be implemented. Responsible AI includes practices such as:

- Debiasing AI models (reducing unfair bias)
- Diverse dataset curation (using representative and inclusive datasets)
- Continuous monitoring for biases throughout the AI lifecycle

34.2 Responsible AI

IBM's AI systems are built on **five pillars** that guide the development, deployment, and usage of AI systems:

- Explainability
- Fairness
- Robustness
- Transparency
- Privacy

These pillars are essential for building trustworthy and reliable AI systems.

Responsible AI encourages **transparency** and **accountability**, allowing users to understand AI decision-making processes and fostering trust in the technology. It also establishes mechanisms to hold developers, organizations, and AI systems accountable for their actions and impacts.

Responsible AI practices include:

- Developing and enforcing ethical guidelines.
- Establishing industry standards.
- Investing in research to improve model interpretability (making AI decisions easier to understand).
- Providing explanations for generated outputs.
- Addressing privacy concerns by implementing privacy-preserving techniques such as:
 - Federated Learning (training AI models on edge devices, without sharing raw data to the cloud)
 - Differential Privacy (protecting individual data by adding precise, controlled statistical noise to datasets, queries, or model updates, which blurs exact individual data points so that an outside observer cannot tell whether a specific person's information was included in the training set.)

Collaboration among the following stakeholders is crucial for creating responsible AI guidelines and policies:

- AI researchers
- Ethicists
- Policymakers
- The public

Responsible AI also emphasizes:

- User empowerment by allowing users to control AI systems and make informed decisions.
- Education and awareness about the ethical implications of Generative AI.
- Building public trust, which is essential for the widespread adoption of AI technologies.

34.3 Legal and Regulatory Compliance

Responsible AI helps ensure that AI systems comply with existing laws and regulations, preventing issues associated with unethical AI practices.

Responsible AI also:

- Supports compliance with current legal requirements.
- Helps guide future AI regulations and industry standards.

34.4 Social Impact

Responsible AI plays an essential role in addressing the ethical, social, and practical concerns associated with the development and deployment of artificial intelligence.

It encourages the development of AI systems that:

- Contribute positively to society.
- Address real-world challenges.
- Promote the well-being of individuals and communities.
- Prevent harm.
- Respect human rights.
- Ensure fair treatment for all individuals.

Responsible AI also promotes:

- Inclusive design.
- Avoiding discrimination.
- Encouraging diversity.

Another important benefit is its positive **environmental impact**, as Responsible AI encourages sustainable and eco-friendly practices in AI research and deployment.

Long-term viability is equally important because unethical or irresponsible AI practices can result in:

- Public backlash.
- Legal actions.
- Regulatory restrictions.

These consequences can hinder the long-term success and adoption of AI technologies.

34.5 Responsible AI Practices

Responsible AI includes the following key practices:

- Transparency
- Inclusivity
- Continuous monitoring
- Collaboration
- User empowerment
- Education and awareness
- Legal and regulatory compliance

Key recommendations include:

- Being transparent about the capabilities and limitations of Generative AI models.
- Ensuring inclusive representation in training datasets.
- Actively involving stakeholders during AI development and deployment.

Adhering to existing and emerging AI regulations, while advocating for responsible AI legislation, is essential for developing and deploying Generative AI models that align with ethical principles and societal values.

34.6 Further Reading

To learn more about Responsible AI, visit the following IBM resources:

34.6.1 IBM Responsible AI

Three key reasons your organization needs Responsible AI

This IBM resource explains why implementing Responsible AI is essential for organizations.

34.6.2 Government Guidance on Responsible AI

10 Things Governments Should Know About Responsible AI

This IBM publication provides guidance for governments and policymakers on AI-related initiatives.

PDF Link:

<https://www.ibm.com/downloads/cas/NK0J1VEN>

35 Case Study: Considerations While Using Generative AI in Healthcare

35.1 Overview

A team of data professionals at a leading healthcare company harnessed the power of generative AI to address critical challenges in **data augmentation, diagnostics, drug design, and personalized healthcare**. Each application presented unique considerations and challenges. The following sections describe the issues encountered and the approaches taken to address them.

35.2 Data Augmentation

Data augmentation is used to generate additional data while preserving the characteristics of the original dataset. In healthcare, the following considerations were important.

35.2.1 Key Considerations

- **Data quality and relevance:** Generative AI models require high-quality, relevant data to generate realistic and meaningful augmented data. The team established rigorous data quality control measures and collaborated with external sources to ensure data integrity.
- **Data diversity:** To avoid overfitting (when a model performs well on training data but poorly on new data) and enhance generalizability, the team employed generative AI models capable of capturing the diversity and nuances of the original data. They implemented techniques such as **adversarial training** (a technique that trains models to generate more realistic and diverse data) to generate data with diverse patterns and characteristics.
- **Privacy considerations:** Preserving patient privacy is paramount in healthcare data augmentation. The team implemented **federated learning** (a technique where models are trained across multiple devices or organizations without sharing raw data) and **differential privacy** (a method that protects individual data by adding controlled randomness) to protect sensitive patient information while enabling data sharing and collaboration.

35.2.2 Data Professionals' Approach

- **Maintaining data fidelity:** The team carefully evaluated the generated data to ensure it preserved the statistical properties and characteristics of the original data, preventing the introduction of artificial biases or distortions.
- **Domain expertise:** The team collaborated closely with domain experts, such as clinicians and researchers, to ensure the generated data aligned with clinical relevance and real-world scenarios.

35.3 Diagnostics

Generative AI supports disease diagnosis by integrating multiple healthcare data sources and improving diagnostic accuracy.

35.3.1 Key Considerations

- **Data integration and harmonization:** Disease diagnosis often involves analyzing diverse data sources, including medical images, patient history, and laboratory results. The team developed data harmonization and integration frameworks to streamline data access and analysis.
- **Explainability and interpretability:** Explainable AI algorithms are essential for building trust in AI-powered diagnostic tools. The team employed techniques such as **saliency maps** (visualizations highlighting the most influential parts of the input) and **LIME (Local Interpretable Model Explanations)** to provide insights into the AI models' decision-making process.
- **Clinical validation:** AI-based diagnostic tools must be validated against traditional diagnostic methods to ensure accuracy and reliability. The team conducted rigorous clinical trials to evaluate the performance of their AI models and demonstrate their effectiveness.

35.3.2 Data Professionals' Approach

- **Data bias mitigation:** The team carefully selected and curated datasets to minimize biases that could lead to inaccurate diagnoses.
- **Continuous monitoring and improvement:** The team established an ongoing monitoring and improvement process to update and refine AI models as new data became available and clinical practices evolved.

35.4 Drug Design

Generative AI accelerates drug discovery by generating potential drug candidates and predicting their properties.

35.4.1 Key Considerations

- **Molecular complexity and dynamics:** Drug design involves understanding and manipulating complex molecular structures and their dynamic interactions. The team employed generative AI models capable of representing and simulating molecular structures and their interactions.
- **Target specificity and efficacy:** Effective drugs must selectively target disease-causing molecules without harming healthy cells. The team used generative AI algorithms to design drug candidates with high target specificity and optimal efficacy.
- **Predictive modeling and virtual screening:** Predicting drug properties and potential side effects is crucial before clinical trials. The team developed generative AI models to predict drug properties and perform virtual screening to identify promising drug candidates.

35.4.2 Data Professionals' Approach

- **Data availability and quality:** The team collaborated with pharmaceutical companies and research institutions to access high-quality molecular data and ensure its relevance to the drug discovery process.

- **Computational efficiency:** Drug design is computationally intensive. The team optimized AI models to reduce computational time and accelerate the drug discovery pipeline.

35.5 Personalized Healthcare

Generative AI enables healthcare solutions tailored to individual patients while ensuring privacy, fairness, and clinician involvement.

35.5.1 Key Considerations

- **Patient data privacy and security:** Patients must have confidence that their health data is being used responsibly and securely. The team implemented transparent data governance policies, robust data security measures, and clear communication channels with patients to address privacy concerns.
- **Patient-centered approach:** Personalized healthcare requires understanding individual patient preferences and needs. The team involved patients in designing and developing AI-powered tools, ensuring patient-centricity and fostering trust.
- **Clinical decision support and interpretability:** AI models should complement, not replace, clinical expertise. The team developed explainable AI algorithms to provide clinicians with insights into the rationale behind AI-generated recommendations, facilitating informed decision-making.

35.5.2 Data Professionals' Approach

- **Addressing data bias and fairness:** The team carefully evaluated AI models to identify and mitigate potential biases that could lead to unfair or discriminatory treatment recommendations.
- **Human-AI collaboration and integration:** The team emphasized the importance of human-AI collaboration, ensuring that AI-generated recommendations were integrated into clinical workflows while maintaining clinician oversight and expertise.

35.6 Key Takeaways

- High-quality, diverse, and privacy-preserving data is essential for successful healthcare data augmentation.
- Explainability, interpretability, and clinical validation are critical for trustworthy AI-powered diagnostics.
- Generative AI accelerates drug discovery through molecular modeling, predictive modeling, and virtual screening.
- Personalized healthcare depends on secure patient data management, fairness, patient involvement, and clinician oversight.
- Human expertise remains essential, with generative AI serving as a decision-support tool rather than a replacement for healthcare professionals.

36 Responsible Use of Generative AI in Real-World Applications

36.1 Overview

In this topic, we assess and reinforce our understanding of the key principles and considerations for using Generative AI responsibly in real-world applications. The scenarios focus on **fairness, transparency, accountability, and reliability** when deploying AI systems in production.

36.2 Learning Objectives

After completing these exercises, we will be able to:

- Maintain transparency and fairness in AI systems.
- Ensure accountability in the deployment of AI chatbots.
- Enhance the reliability of Generative AI models to produce accurate and trustworthy outputs.

36.3 Exercise 1

36.3.1 Scenario

We are developing a Generative AI system that creates personalized content recommendations for users. The system appears to consistently recommend content that reflects certain cultural and demographic biases. Users from diverse backgrounds have raised concerns about the lack of transparency and fairness in the recommendations.

36.3.2 Question

How do we maintain transparency and fairness in our AI system?

36.3.3 Solution

To maintain transparency and fairness, we should:

- Conduct regular bias assessments to identify and measure biases in the model.
- Improve the diversity and representativeness of the training data.
- Apply bias mitigation techniques during model training and evaluation.
- Incorporate Explainable AI (XAI) (techniques that help users understand how and why an AI model makes its decisions) so users understand why recommendations are generated.
- Clearly communicate the limitations and capabilities of the AI system.
- Establish user feedback mechanisms to report unfair or biased recommendations.
- Continuously monitor model performance and retrain the model as new data becomes available.
- Perform periodic fairness audits to ensure compliance with ethical AI principles.

36.3.4 Additional Information

Some specialized tools that can be used to measure and quantify biases:

- **Holistic AI Library:** This open-source library offers a range of metrics and mitigation strategies for various AI tasks, including content recommendation. It analyzes data for bias across different dimensions and provides visualizations for clear understanding.
- **Fairness 360:** IBM's Fairness 360 toolkit provides various tools like Aequitas and What-If Tool to analyze bias in data sets, models, and decision-making processes. It offers metrics like statistical parity, differential odds ratio, and counterfactual fairness. IBM moved AI Fairness 360 to LF AI in July 2020.

36.4 Exercise 2

36.4.1 Scenario

Our company has deployed a Generative AI chatbot to interact with customers. Occasionally, the chatbot produces inappropriate or offensive responses, resulting in customer dissatisfaction.

36.4.2 Question

How do we take responsibility for these incidents and ensure accountability in the deployment of the AI chatbot?

36.4.3 Solution

To ensure accountability, we should:

- Investigate and analyze the root cause of inappropriate responses.
- Respond promptly to reported incidents and take corrective action.
- Continuously monitor chatbot interactions to detect harmful or offensive outputs.
- Implement human oversight for sensitive or high-risk conversations.
- Update and retrain the model using appropriate datasets and safety guidelines.
- Maintain detailed logs and audit trails (records that track system activities and decisions for review and accountability) to support accountability and troubleshooting.
- Communicate openly with users regarding system limitations and improvements.
- Establish clear governance policies that define roles and responsibilities for AI deployment and monitoring.

36.5 Exercise 3

36.5.1 Scenario

Our company has developed a Generative AI model that automatically generates product descriptions for an e-commerce platform. Users have reported that some descriptions contain inaccurate information, causing customer confusion and dissatisfaction.

36.5.2 Question

How do we enhance the reliability of our AI model to ensure accurate product descriptions?

36.5.3 Solution

To improve the reliability of the AI model, we should:

- Perform comprehensive quality assurance testing before deployment.
- Train the model using high-quality, accurate, and domain-specific datasets.
- Continuously retrain and fine-tune the model based on user feedback and newly available data.
- Validate generated product descriptions against trusted product information before publication.
- Integrate human review for high-value or sensitive product listings.
- Monitor model performance regularly and address recurring errors.
- Establish continuous evaluation processes using reliability and accuracy metrics.
- Maintain a feedback loop to identify issues and continuously improve model performance.

36.6 Key Takeaways

- Transparency and fairness require diverse training data, bias mitigation techniques, Explainable AI (XAI), continuous monitoring, and user feedback.
- Accountability involves prompt incident response, human oversight, governance policies, audit trails, and transparent communication with users.
- Reliability is achieved through high-quality training data, quality assurance testing, validation against trusted sources, continuous retraining, performance monitoring, and human review when appropriate.

37 Module 2 Summary: Use of Generative AI for Data Science

At this point, we know that:

- Generative AI can help in **Exploratory Data Analysis (EDA)** using various techniques, such as:
 - Statistical data description
 - Univariate analysis
 - Bivariate analysis
 - Multivariate analysis
 - Feature engineering
 - Hypothesis generation
- Generative AI plays a significant role in developing a predictive model using various techniques, such as:
 - Selecting model architecture
 - Selecting important features
 - Generating ensemble models
 - Improving model interpretability
 - Improving model generalization
 - Preventing overfitting
- While using Generative AI models, we need to consider:

- Data considerations
 - Model considerations
 - Ethical considerations
- Data professionals face various challenges when using Generative AI across multiple industries, including:
 - Technical challenges
 - Organizational challenges
 - Cultural challenges
- Becoming a successful data scientist requires several essential skills, including:
 - Mathematical skills
 - Statistical skills
 - Knowledge of programming languages
 - Understanding of machine learning principles
- Generative AI can help us develop and strengthen these skills.

38 Module 2 Cheat Sheet: Use of Generative AI for Data Science

38.1 Popular Generative AI Tools

Name of Model	Usage	Link
Hal9	EDA (Exploratory Data Analysis) tool to identify key insights on data.	https://www.hal9.com/
Columns.ai	Data visualization tool to create useful charts.	https://columns.ai/
Akkio	Data visualization tool to create data plots such as regression plots, box plots, correlation heatmaps, and more.	https://www.akkio.com/

38.2 Important Prompts for Generating Data Insights and Visualizations

Task	Prompt
Generate a statistical description of data.	Write a Python code to generate the statistical description of all the features used in the data set. Include " object " data types as well.
Create regression plots between a target variable and a continuous-valued source variable.	Write a Python code to generate a regression plot between a target variable and a source variable of a data frame.
Create box plots between a target variable and a categorical source variable.	Write a Python code to generate a box plot between a target variable and a source variable of a data frame.

Evaluate parametric interdependence using correlation, p-value, and Pearson coefficient.	Write a Python code to evaluate correlation, Pearson coefficient, and p-values for all attributes of a data frame against the target attribute.
Group variables to create pivot tables. Create a p-color plot for the pivot table.	Write a Python code that performs the following actions: 1. Groups three attributes as available in a data frame df. 2. Creates a pivot table for this group, using a target attribute and aggregation function as mean. 3. Plots a pcolor plot for this pivot table.

38.3 Important Prompts for Model Development and Refinement

Task	Prompt
Linear regression between a single source attribute and target attribute and evaluate it.	Write a Python code that performs the following tasks: 1. Develops and trains a linear regression model that uses one attribute of a data frame as the source variable and another as the target variable. 2. Calculates and displays the MSE and R² values for the trained model.
Linear regression between multiple source attributes and a target attribute and evaluate it.	Write a Python code that performs the following tasks: 1. Develops and trains a linear regression model that uses some attributes of a data frame as the source variables and one of the attributes as the target variable. 2. Calculates and displays the MSE and R² values for the trained model.
Polynomial regression model with a single source and target variable.	Write a Python code that performs the following tasks: 1. Develops and trains multiple polynomial regression models with orders 2, 3, and 5 that use one attribute of a data frame as the source variable and another as the target variable. 2. Calculates and displays the MSE and R² values for the trained models. 3. Compares the performance of the models.
Pipeline creation for scaling, polynomial feature creation, and linear regression.	Write a Python code that performs the following tasks: 1. Creates a pipeline that performs parameter scaling, polynomial feature generation, and linear regression. Use the same set of multiple features as before to create this pipeline. 2. Calculates and displays the MSE and R² values for the trained model.

Grid search with ridge regression and cross-validation.	Write a Python code that performs the following tasks: 1. Uses polynomial features for some of the attributes of a data frame. 2. Performs a grid search on a ridge regression model for a set of values of hyperparameter alpha and polynomial features as input. 3. Uses cross-validation in the grid search. 4. Evaluates the resulting model's MSE and R² values.
---	--

39 Review What We Learned

39.1 Overview

In this topic, we explored **Generative AI techniques** and their applications in **Data Science**. We learned how to integrate Generative AI into data science projects, including **data augmentation** and **feature engineering**. We also learned how to use and apply generative models for tasks such as **data generation**, **data augmentation**, **data preparation**, **data querying**, and more.

39.2 Key Takeaways

- Generative AI empowers data scientists to generate entirely new data, unlocking a universe of possibilities and tackling previously insurmountable challenges.
- There are **four standard Generative AI models**:
 - Generative Adversarial Networks (GANs)
 - Variational Autoencoders (VAEs)
 - Autoregressive Models
 - Flow-Based Models
- **Generative Adversarial Networks (GANs)** are particularly effective for **data augmentation**.
- **Variational Autoencoders (VAEs)** are well suited for:
 - Anomaly detection
 - Data compression
 - Collaborative filtering
 - Style transfer
- **Autoregressive Models** are commonly used for:
 - Text generation
 - Speech synthesis
 - Time series forecasting
 - Machine translation
- **Flow-Based Models** are suitable for:
 - Image generation
 - Data generation
 - Density estimation (estimating how data is distributed)
- Generative AI can tackle complex problems across a wide range of industries.
- Generative AI models are instrumental in addressing several **data preparation** and **data querying** challenges, including:
 - Imputing missing values (filling in missing data)
 - Detecting outliers
 - Reducing noise (removing irrelevant or unwanted variations in data)

- Translating natural language queries into equivalent SQL statements
- Generative AI can support **Exploratory Data Analysis (EDA)** through techniques such as:
 - Statistical data description
 - Univariate analysis
 - Bivariate analysis
 - Multivariate analysis
 - Feature engineering
 - Hypothesis generation
- Generative AI plays a significant role in developing predictive models by:
 - Selecting suitable model architectures
 - Selecting essential features
 - Generating ensemble models
 - Improving model interpretability
 - Improving model generalization
 - Preventing overfitting
- While using Generative AI models, data scientists must carefully consider:
 - Data-related considerations
 - Model-related considerations
 - Ethical considerations
- Data professionals encounter several challenges when adopting Generative AI across industries, including:
 - Technical challenges
 - Organizational challenges
 - Cultural challenges
- Becoming a successful data scientist requires a combination of:
 - Mathematical skills
 - Statistical skills
 - Knowledge of programming languages
 - Understanding of machine learning principles

Module 3: Final Project

40 Final Project Overview

40.1 Introduction

In this project, we use a real-world dataset and practice using Generative AI to create Python code that performs:

- Data preparation
- Exploratory data analysis (EDA)
- Machine learning model development

40.2 Project Scenario

We have been employed as a **Data Scientist** by a consultancy firm. The firm has a client who is a resale car dealer specializing in **Ford** cars. The client wants us to design a machine learning model that can predict the **optimum quotation price** for the cars in their inventory.

To accomplish this task, the client has provided historical sales data collected over the past few years. The dataset contains various features of Ford cars along with the prices at which they were sold.

40.3 Project Objectives

The following tasks are assigned as part of the project:

- Clean the dataset by identifying and handling duplicate entries.
- Identify and handle missing values in the dataset.
- Perform **Exploratory Data Analysis (EDA)** to gain meaningful insights into the data.
- Analyze the impact of different features on the selling price of the cars.
- Build and compare multiple regression models to determine the best-performing model.
- Optimize the selected model using **Grid Search** to identify the best hyperparameter values.

40.4 Client-Specific Analysis Requirements

The client has requested the following analyses:

- Identify the number of car sales for each **fuel type**.
- Identify which **transmission type** contains more **price outliers**.

40.5 Model Development Requirements

Develop and compare the following regression models:

- Linear Regression
- Polynomial Regression
- Ridge Regression

The comparison should include both:

- Single-variable regression models.
- Multiple-variable regression models.

The objective is to determine the **best-performing model**.

After selecting the Ridge Regression model, perform a **Grid Search** to identify the optimum hyperparameter for achieving the best model performance.

40.6 About the Dataset

This dataset contains resale prices of **Ford** cars.

It is a public dataset available on **Kaggle** under the name **Ford Car Pricing Dataset**, licensed under the **CC0: Public Domain License**.

For this project, the dataset has been **slightly modified**.

40.7 Dataset Attributes

Variable	Description
model	Car model name
year	Year of car manufacture
transmission	Type of transmission (Automatic, Manual, or Semi-Auto)
mileage	Number of miles traveled
fuelType	Type of fuel used (Petrol, Diesel, Hybrid, Electric, or Other)
tax	Annual tax payable in USD
mpg	Miles per gallon (fuel efficiency)
engineSize	Engine size of the car
price	Selling price of the car in USD

40.8 Project Tasks

The project consists of the following major stages:

- Data Preparation
- Data Analysis
- Model Development

Based on the provided dataset, we must write **Generative AI prompts** that generate Python code for performing the required tasks.

41 Data Loading & Preparation – Part 1

41.1 Learning Objectives

After completing these exercises, we will be able to:

- Import a dataset into a Pandas DataFrame.
- Clean the dataset by handling missing values and duplicate records.
- Perform exploratory data analysis and data visualization.
- Develop and evaluate multiple machine learning models.
- Optimize model performance using Ridge Regression and Grid Search.

41.2 Introduction

In these exercises, we use Generative AI to assist with the complete data science workflow for a used Ford car pricing dataset. The workflow includes importing and preparing the dataset, exploring and visualizing the data, developing predictive models, evaluating their performance, and optimizing the best model using Grid Search with Ridge Regression.

41.3 Exercise 1: Import the Dataset

41.3.1 Objective

Import the used Ford car pricing dataset into a Pandas DataFrame and verify that it has been loaded correctly.

41.3.2 Prompt

Generate plain basic Python code without using definitions or, exceptions or if-else statements. Response should be concise.

Write Python code that can perform the following tasks:

1. Read the CSV file, located on a given file path, into a Pandas DataFrame, assuming that the first row of the file can be used as the headers for the data.
2. Print the first five rows of the DataFrame to verify correct loading.

41.3.3 Python Code

```
import pandas as pd

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

print(df.head())
```

41.4 Exercise 2: Data Preparation

41.4.1 Data Cleaning

Before performing analysis or building predictive models, we clean the dataset by replacing missing values with the mean of their respective columns and removing duplicate records.

41.4.2 Prompt

Generate plain, basic Python code without using definitions or, exceptions or if-else statements. Response should be concise.

Write Python code that performs the following tasks:

1. Identify the columns of a DataFrame with missing values.
2. Replace the missing values with the mean value of their respective columns.
3. Delete duplicate entries from the DataFrame.

41.4.3 Python Code

```
import pandas as pd

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

print("Missing Values:")
print(df.isnull().sum())
```

```

numeric_columns = df.select_dtypes(include="number").columns

df[numeric_columns] = df[numeric_columns].fillna(df[numeric_columns].mean())

df = df.drop_duplicates()

print("\nDataset Shape After Cleaning:")
print(df.shape)

print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

```

41.5 Summary

In this part, we completed the initial stages of the data science workflow. We learned how to:

- Import a CSV dataset into a Pandas DataFrame.
- Verify that the dataset loaded correctly.
- Identify missing values.
- Replace missing numerical values with the mean of their respective columns.
- Remove duplicate records to prepare the dataset for further analysis.

42 Data Insights and Visualization – Part 2

42.1 Introduction

Exploratory Data Analysis (EDA) helps us understand the characteristics of the dataset before building predictive models. In these exercises, we use Generative AI to generate Python code for identifying highly correlated features, analyzing categorical data, and creating visualizations that reveal patterns and relationships within the dataset.

42.2 Exercise 3: Data Insights and Visualization

42.2.1 Identify the Top 5 Correlated Features

42.2.1.1 Objective

Identify the five attributes that have the highest correlation with the target attribute in the dataset.

42.2.1.2 Prompt

Generate plain, basic Python code without using definitions or, exceptions or if-else statements. Response should be concise.

Write Python code that identifies the top 5 attributes with the highest correlation with the target attribute in a DataFrame.

42.2.1.3 Python Code

```

import pandas as pd

url = "https://cf-courses-data/used_car_price_analysis.csv"

```

```
df = pd.read_csv(url)

correlation = df.corr(numeric_only=True) ["price"].drop("price")

top5 = correlation.abs().sort_values(ascending=False).head(5)

print(top5)
```

42.2.2 Count the Number of Cars by Fuel Type

42.2.2.1 Objective

Determine how many vehicles belong to each fuel type category in the dataset.

42.2.2.2 Prompt

Generate plain basic Python code without using definitions or exceptions or if-else statements. Response should be concise.

Write Python code that counts the number of entries in a DataFrame with each unique value of a specific attribute.

42.2.2.3 Python Code

```
import pandas as pd

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

fuel_counts = df["fuelType"].value_counts()

print(fuel_counts)
```

42.2.3 Create a Box Plot

42.2.3.1 Objective

A box plot helps visualize the distribution of car prices across different transmission types and identifies potential price outliers.

42.2.3.2 Prompt

Generate plain basic Python code without using definitions or, exceptions or if-else statements. Response should be concise.

Write Python code that creates a Seaborn box plot between a source attribute and a target attribute of a DataFrame.

42.2.3.3 Python Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)
```



```
sns.boxplot(x="transmission", y="price", data=df)

plt.title("Transmission vs Price")
plt.xlabel("Transmission")
plt.ylabel("Price")

plt.show()
```

42.2.4 Create a Regression Plot

42.2.4.1 Objective

A regression plot visualizes the relationship between **mpg** and **price**, helping us understand the direction and strength of their correlation.

42.2.4.2 Prompt

Generate plain basic Python code without using definitions or exceptions or if-else statements. Response should be concise.

Write Python code that creates a regression plot between a source attribute and a target attribute.

42.2.4.3 Python Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

sns.regplot(x="mpg", y="price", data=df)

plt.title("MPG vs Price")
plt.xlabel("Miles Per Gallon (MPG)")
plt.ylabel("Price")

plt.show()
```

42.3 Summary

Through these exercises, we used Generative AI to generate Python code for exploratory data analysis and visualization. We:

- Identified the five attributes most strongly correlated with the target variable.
- Analyzed the distribution of vehicles by fuel type.
- Created a box plot to compare price distributions across transmission types.
- Generated a regression plot to examine the relationship between **mpg** and **price**.

These analyses provide valuable insights that support feature selection and model development.

43 Model Development and Evaluation – Part 3

43.1 Introduction

After preparing and exploring the dataset, the next step is to build predictive models for estimating used Ford car prices. In these exercises, we use Generative AI to generate Python code for developing, evaluating, and optimizing different regression models using the selected features.

43.2 Exercise 4: Model Development and Evaluation

43.2.1 Linear Regression Using a Single Feature

43.2.1.1 Objective

In this exercise, we use **mpg** as the source feature to predict the **price** of a car. The model is evaluated using **Mean Squared Error (MSE)** and the **R² Score**.

43.2.1.2 Prompt

Generate plain, basic Python code without using definitions, exceptions, or if-else statements. Response should be concise.

Write Python code to fit a linear regression model on a target variable on the basis of a source variable. Calculate the **R²** value and **MSE** value for this model.

43.2.1.3 Python Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

X = df[["mpg"]]
y = df["price"]

model = LinearRegression()
model.fit(X, y)

predictions = model.predict(X)

print("MSE:", mean_squared_error(y, predictions))
print("R2 Score:", r2_score(y, predictions))
```

43.2.2 Linear Regression Using Multiple Features

43.2.2.1 Objective

Train a Linear Regression model using the following features:

- year
- mileage

- tax
- mpg
- engineSize

43.2.2.2 Prompt

Generate plain basic Python code without using definitions, exceptions, or if-else statements. Response should be concise.

Write Python code to fit a linear regression model on a target variable on the basis of a set of source variables. Calculate the **R²** value and **MSE** value for this model.

43.2.2.3 Python Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

features = ["year", "mileage", "tax", "mpg", "engineSize"]

X = df[features]
y = df["price"]

model = LinearRegression()
model.fit(X, y)

predictions = model.predict(X)

print("MSE:", mean_squared_error(y, predictions))
print("R2 Score:", r2_score(y, predictions))
```

43.2.3 Create a Machine Learning Pipeline

43.2.3.1 Objective

A Machine Learning Pipeline automates preprocessing and model training by combining **StandardScaler**, **PolynomialFeatures**, and **LinearRegression** into a single workflow.

43.2.3.2 Prompt

Generate plain basic Python code without using definitions, exceptions or if-else statements. Response should be concise.

Write Python code that can perform the following tasks:

1. Create a pipeline with StandardScaler, second-degree PolynomialFeatures, and a LinearRegression model.
2. Fit this pipeline for a target variable using a set of source attributes from a DataFrame.
3. Evaluate the **R²** and **MSE** values for the trained model.

43.2.3.3 Python Code

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

features = ["year", "mileage", "tax", "mpg", "engineSize"]

X = df[features]
y = df["price"]

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("poly", PolynomialFeatures(degree=2)),
    ("model", LinearRegression())
])

pipeline.fit(X, y)

predictions = pipeline.predict(X)

print("MSE:", mean_squared_error(y, predictions))
print("R2 Score:", r2_score(y, predictions))
```

43.3 Summary

In these exercises, we developed predictive models using both single and multiple input features. We also created a Machine Learning Pipeline that combines preprocessing and model training into a single workflow, making model development more efficient and reproducible.

Through these exercises, we learned how to:

- Build a Linear Regression model using a single feature.
- Train a Linear Regression model using multiple features.
- Evaluate models using **Mean Squared Error (MSE)** and **R² Score**.
- Create a Machine Learning Pipeline using **StandardScaler**, **PolynomialFeatures**, and **LinearRegression** to automate preprocessing and model training.

44 Model Optimization – Part 4

44.1 Introduction

Model optimization improves predictive performance by reducing overfitting and identifying the best hyperparameters. In these exercises, we apply Ridge Regression, Polynomial Feature Transformation, and Grid Search with Cross-Validation to build a more robust prediction model.

44.2 Exercise 5: Model Optimization

44.2.1 Ridge Regression

44.2.1.1 Objective

Create and evaluate a Ridge Regression model using a training and testing dataset split.

44.2.1.2 Prompt

Generate plain basic Python code without using definitions, exceptions or if-else statements. Response should be concise.

Write Python code that can perform the following tasks:

1. Assuming that a subset of the attributes of a DataFrame are source attributes and one of the attributes is a target attribute, split the data into training and testing data, assuming the testing data to be 20%.
2. Create and fit a Ridge Regression model using the training data, setting the regularization parameter to **0.1**.
3. Calculate the **MSE** and **R²** values for the Ridge Regression model using the testing data.

44.2.1.3 Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

features = ["year", "mileage", "tax", "mpg", "engineSize"]

X = df[features]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Ridge(alpha=0.1)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, predictions))
print("R2 Score:", r2_score(y_test, predictions))
```

44.2.2 Polynomial Ridge Regression

44.2.2.1 Objective

Improve the Ridge Regression model by applying second-degree Polynomial Feature Transformation before model training.

44.2.2.2 Prompt

Generate plain basic Python code without using definitions, exceptions or if-else statements. Response should be concise.

Write Python code that can perform the following tasks:

1. Split the data into training and testing sets (20% testing).
2. Apply second-degree polynomial scaling.
3. Create and fit a Ridge Regression model using the training data with **alpha = 0.1**.
4. Calculate the **MSE** and **R²** values for the testing data.

44.2.2.3 Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

features = ["year", "mileage", "tax", "mpg", "engineSize"]

X = df[features]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = Ridge(alpha=0.1)

model.fit(X_train_poly, y_train)

predictions = model.predict(X_test_poly)

print("MSE:", mean_squared_error(y_test, predictions))
print("R2 Score:", r2_score(y_test, predictions))
```

44.2.3 Grid Search with Ridge Regression

44.2.3.1 Objective

Use Grid Search with Cross-Validation to identify the optimal **alpha** value for a Ridge Regression model after applying Polynomial Feature Transformation.

44.2.3.2 Prompt

Generate plain basic Python code without using definitions, exceptions or if-else statements. Response should be concise.

Write Python code that can perform the following tasks:

1. Split the data into training and testing sets (20% testing).
2. Apply second-degree polynomial scaling.
3. Perform Grid Search on Ridge Regression using **alpha** values {0.01, 0.1, 1, 10, 100} with **4-fold cross-validation**.
4. Calculate the **MSE** and **R²** values for the optimized model.

44.2.3.3 Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

url = "https://cf-courses-data/used_car_price_analysis.csv"

df = pd.read_csv(url)

features = ["year", "mileage", "tax", "mpg", "engineSize"]

X = df[features]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

parameters = {"alpha": [0.01, 0.1, 1, 10, 100]}

grid = GridSearchCV(Ridge(), parameters, cv=4)

grid.fit(X_train_poly, y_train)

predictions = grid.predict(X_test_poly)

print("Best Alpha:", grid.best_params_["alpha"])
print("MSE:", mean_squared_error(y_test, predictions))
print("R2 Score:", r2_score(y_test, predictions))
```

44.3 Summary

Through these exercises, we used Generative AI to generate Python code for developing, evaluating, and optimizing machine learning models. We:

- Implemented Linear Regression models using both single and multiple features.
- Created a Machine Learning Pipeline that combines preprocessing and model training into a single workflow.
- Trained and evaluated Ridge Regression models.
- Applied second-degree Polynomial Feature Transformation to capture more complex relationships in the data.
- Performed Grid Search with 4-fold Cross-Validation to identify the optimal **alpha** hyperparameter for Ridge Regression.

- Evaluated model performance using **Mean Squared Error (MSE)** and **R² Score**.

44.4 Final Project Summary

In this final project, we used Generative AI to support the complete data science workflow for predicting used Ford car prices. We completed the following tasks:

- Imported a CSV dataset into a Pandas DataFrame.
- Cleaned the dataset by handling missing values and removing duplicate records.
- Performed Exploratory Data Analysis (EDA) to understand the dataset.
- Created visualizations to identify patterns and relationships between variables.
- Built Linear Regression models using single and multiple features.
- Developed a Machine Learning Pipeline to automate preprocessing and model training.
- Optimized model performance using Ridge Regression.
- Applied Polynomial Feature Transformation to improve model capability.
- Used Grid Search with Cross-Validation to determine the optimal hyperparameters.
- Evaluated model performance using **Mean Squared Error (MSE)** and **R² Score**.

These exercises demonstrate how Generative AI can accelerate every stage of the data science workflow, from data preparation and exploration to model development, evaluation, and optimization, while reducing the amount of manual coding required.

45 How Generative AI Could Transform Data Science Over the Next Five Years

45.1 Overview

Generative AI is expected to significantly revolutionize the field of Data Science over the next five years by automating increasingly complex tasks, enabling us to focus more on strategic thinking and high-value problem-solving.

45.2 How Generative AI Will Transform Data Science

Generative AI is expected to automate more complex tasks such as:

- Data cleaning
- Model generation
- Hypothesis testing

AI-driven tools will make it easier for us to generate insights more quickly, allowing us to focus on strategic decision-making rather than repetitive tasks.

Additionally, AI will likely play a bigger role in predictive analytics, improving accuracy and efficiency across various sectors, including:

- Healthcare
- Finance

Generative AI is expected to evolve from a complementary tool into a central component of the Data Science ecosystem.

Its capabilities include:

- Automation of tasks
- Suggestion-based code generation
- No-code and low-code implementation
- Assisting in solving Data Science problems
- Power prediction
- Forecasting
- Image creation

These capabilities will enhance the fields of Data Science, Artificial Intelligence, and many business domains.

45.3 Automation of Repetitive Data Science Tasks

Many routine tasks involved in data preparation are expected to become mature enough that they can be handled almost entirely by Generative AI.

These tasks include:

- Data extraction
- Data transformation
- Data normalization
- Data imputation

Generative AI can also facilitate more insightful:

- Data exploration
- Data visualization

This automation allows us to spend more time solving important business problems instead of performing repetitive manual work.

45.4 More Abstraction in Data Science Workflows

In the coming years, Generative AI has the potential to completely transform the way we work by creating additional layers of abstraction.

Certain Data Science workflows can be abstracted away using Generative AI models, reducing the need for us to directly interact with lower-level implementation details.

This shift will simplify many workflows while improving productivity.

45.5 Democratization of Data Science

Generative AI is expected to democratize Data Science through increasingly powerful zero-code and low-code tools such as AutoML (Automated Machine Learning).

These tools will enable non-experts to create powerful machine learning models without requiring extensive technical knowledge.

Generative AI can automate many tedious tasks, including:

- Data processing
- Feature engineering
- Hyperparameter tuning

As a result, we can devote more attention to:

- Solving complex problems
- Drawing meaningful insights

Many repetitive coding tasks can also be generated automatically by Generative AI.

45.6 Example: Synthetic Tabular Data Generation

Consider the example of creating synthetic tabular data.

45.6.1 Traditional Data Science Approach

With a traditional Data Science approach, we would write a Python script that defines the required data.

For example, we could specify:

- The required number of columns
- The statistical distribution for each column
- Other data characteristics

After running the script, we would obtain the synthetic dataset.

45.6.2 Generative AI Approach

With the Generative AI paradigm, certain parts of this process can be abstracted away.

Instead of manually writing the Python script, we can simply provide a natural language prompt describing the required dataset.

Behind the scenes, an agentic workflow (an AI system that performs multiple connected tasks automatically) could:

- Write the Python script

- Execute the script
- Generate the synthetic data

As the final output of this abstraction layer, we receive the synthetic dataset.

45.7 Synthetic Data and Data Augmentation

Generative AI is expected to create growing opportunities in sectors where collecting real-world data is difficult, expensive, or risky.

Generative AI will promote innovation through:

- Data augmentation (creating additional training examples from existing data)
- Synthetic data generation (artificially generated data that mimics real-world data)

These capabilities will:

- Improve model training
- Reduce bias
- Enable greater experimentation
- Reduce privacy concerns

45.8 Staying Updated with Generative AI

Data professionals should continuously stay informed about the rapid advancements in Generative AI.

One effective approach is following leading AI companies and influential AI professionals on LinkedIn.

Examples include:

- IBM
- Meta
- OpenAI
- Anthropic
- Mistral
- Hugging Face
- NVIDIA
- Google
- AWS

It is also valuable to follow their executives and AI leadership teams while keeping track of their:

- Posts
- Product announcements
- News releases

45.9 Learn Through Hands-On Practice

After discovering new AI tools and models, practical experimentation is essential.

Useful activities include:

- Exploring open-source GitHub repositories
- Using released SDKs (Software Development Kits)
- Experimenting with newly released tools

Writing our own code helps us:

- Understand the practical usage of Generative AI models
- Stay technically up to date

Learning modern AI frameworks is considered essential, including:

- TensorFlow
- PyTorch

For further low-code research, learning C++-based languages or Rust may further improve the capabilities of Data Science professionals.

45.10 Continuous Learning Resources

Continuous learning is one of the most important ways to remain current in Generative AI.

Recommended learning platforms include:

- Coursera
- edX
- Udemy
- NVIDIA
- Microsoft
- Google
- AWS

These platforms offer certifications and educational resources for developing Generative AI skills.

45.11 Read Research Papers

Regularly reading research papers helps us remain aware of the latest developments.

Important AI conferences include:

- NeurIPS
- ICML
- CVPR

It is also important to follow research publications and subscribe to journals such as:

- [arXiv \(an open-access repository for scientific research papers\)](#)

45.12 Participate in the AI Community

Professional growth can also be supported by:

- Attending webinars
- Participating in workshops
- Attending AI and Data Science conferences

Following blogs and newsletters from leading AI researchers and organizations provides continuous updates and valuable insights.

Examples include:

- [Towards Data Science \(on Medium\)](#)
- [AI newsletters](#)

Joining AI-focused communities and participating in open-source projects also helps us remain hands-on with the latest tools and techniques.

45.13 Continue Experimenting

To keep pace with advancements in Generative AI, we should continuously:

- Learn through online platforms such as Coursera and edX.
- Participate in webinars.
- Attend AI and Data Science conferences.
- Follow key research publications.
- Subscribe to AI research journals such as arXiv.
- Join AI-focused communities.
- Contribute to open-source projects.
- Experiment with new tools and techniques.

Practical experience remains one of the best ways to understand Generative AI and stay connected with ongoing advancements.

45.14 Key Takeaway

Always remember the [compounding effect](#).

[The things we learn today may continue compounding over time, leading to significantly greater knowledge and wealth in the future.](#)

46 Generative AI for Data Science Skills

46.1 Learning Objectives

After completing this topic, we will be able to:

- List the skills that data scientists require to be successful in their field.
- Explain how Generative AI can help data scientists develop and improve these skills.

46.2 Skills Required to Become a Successful Data Scientist

Being a successful data scientist requires a combination of technical knowledge, analytical thinking, practical experience, and continuous learning. Generative AI can assist in developing many of these essential skills.

46.2.1 Mathematical and Statistical Foundation

A robust foundation in mathematics and statistics is essential for data science. Important concepts include:

- [Statistics](#)
- [Probability](#)
- [Linear Algebra](#)
- [Central Tendencies](#)
- [Standard Deviation](#)
- [Probability Distributions](#)

Generative AI models such as GPT can ease the learning process by quickly skimming through available resources and providing concise information based on our requirements.

46.2.2 Programming Skills

Knowledge of programming languages such as **Python** is a key requirement in the field of data analysis and predictive modeling.

Generative AI can significantly assist by:

- Guiding us in writing efficient code.
- Suggesting the optimum approach for solving programming problems.
- Helping prepare code that performs the required tasks.

46.2.3 Data Preparation

Data preparation is a key asset for all data professionals.

It includes tasks such as:

- Cleaning data
- Handling missing values

- Standardizing data
- Normalizing data
- Structuring data for analysis or model training

Data scientists perform data preparation to ensure that the data is in a format that is usable and appropriate for the intended task, whether it is analysis, machine learning, or another application.

Generative AI helps perform these tasks more efficiently and optimally.

46.2.4 Statistical Analysis

Statistical analysis is another important skill in the arsenal of a good data scientist.

It is required to:

- Identify features.
- Engineer features that best describe the data for a target attribute.

Generative AI enables us to:

- Perform statistical analysis much faster.
- Create flexible code that can be modified to develop customized solutions for specific problems.

Note: We must consider the ethical and professional aspects of Generative AI while generating code.

46.2.5 Machine Learning Knowledge

Different types of data, including:

- Numerical data
- Text data
- Time-series data

require different analysis and predictive modeling processes.

Therefore, data scientists must have a good understanding of machine learning principles, including:

- Knowledge of various machine learning algorithms.
- Understanding the strengths and weaknesses of different algorithms.
- Knowing how algorithms can be applied to different types of problems.

Generative AI models can help by:

- Explaining the applicability of different machine learning models.

- Reducing the time and effort required to perform machine learning-based analysis on a given dataset.

46.2.6 Hands-on Project Experience

The success of data scientists is often measured by the projects they have undertaken.

Many datasets are freely available on machine learning and data science community websites such as **Kaggle**, where data professionals can:

- Gain experience with real-world datasets.
- Develop solutions to practical problems.

Generative AI can make the development of appropriate models for these datasets much easier and faster.

46.2.7 Staying Updated with Industry Developments

The data science landscape is constantly evolving.

The integration of Generative AI into data science is one of the most prominent and significant developments in recent years.

Data professionals should:

- Stay updated with the latest developments.
- Learn how to use Generative AI efficiently.
- Use Generative AI productively.
- Use Generative AI ethically.

These are considered essential skills for modern data professionals.

46.2.8 Practice and Continuous Learning

Like any other skill, data science requires regular practice.

We should:

- Develop hands-on skills in data science.
- Use Generative AI to improve our performance.
- Take advantage of freely available learning resources.
- Practice consistently to strengthen our skill set.

Remember:

- Consistency and persistence are key.
- Start by learning the basics.
- Gradually move toward more complex concepts.
- Continue practicing and learning.

Hands-on experience and continuous learning will significantly contribute to our growth as data scientists.

46.3 Key Takeaways

- Being a successful data scientist requires several technical and analytical skills.
- Generative AI can assist in developing and improving these skills.
- Data scientists require a robust foundation in mathematics and statistics.
- Knowledge of programming languages, especially Python, is a key requirement.
- Data preparation is a critical skill for all data professionals.
- Statistical analysis helps identify and engineer meaningful features for predictive modeling.
- A strong understanding of machine learning principles is essential.
- Hands-on projects using real-world datasets, such as those available on Kaggle, help build practical experience.
- Staying current with developments in Generative AI and learning to use it efficiently, productively, and ethically is essential.
- Hands-on experience and continuous learning significantly contribute to the growth of a data scientist.